



Your Tree

Autor: **Nicolas Gianfranco Pezo Ccama**

Tutor: **Juan Carlos Redondo Bastante**

Fecha: 27/03/2026

Versión: 2.0.0

GitHub: <https://github.com/nicowooow/Proyecto-FP>

Curso académico: 2025 / 2026



TABLA DE CONTENIDO

1	INTRODUCCIÓN	11
2	RESUMEN Y ABSTRACT	12
2.1	RESUMEN (CASTELLANO)	12
2.2	ABSTRACT (INGLÉS).....	13
3	GLOSARIO.....	14
4	TECNOLOGÍAS ACTUALES.....	20
4.1	COMPARATIVA DE TECNOLOGÍAS ACTUALES	20
4.1.1	<i>Frontend – comparativa de frameworks.....</i>	<i>20</i>
4.1.2	<i>Backend – comparativa de frameworks</i>	<i>21</i>
4.1.3	<i>Base de datos - comparativa de SGBD</i>	<i>22</i>
4.2	COMPARATIVA CON LA COMPETENCIA	24
4.2.1	<i>Aplicaciones Link-in-BIO</i>	<i>24</i>
4.2.1.1	Milkshake.app.....	24
4.2.1.2	LinkTr.ee	25
4.2.1.3	Beacons.ai	26
4.2.1.4	Campsite.bio	26
4.2.2	<i>Comparativa gráfica.....</i>	<i>27</i>
5	JUSTIFICACIÓN	30
5.1	INSPIRACIÓN DEL PROYECTO	30
5.1.1	<i>Nombre del proyecto y del dominio.....</i>	<i>31</i>
6	COSTES DEL PROYECTO	31
6.1	ESTUDIO ECONÓMICO Y PRESUPUESTO	31
6.2	PLAN DE EMPRESA	32
6.2.1	<i>Propuesta de Valor</i>	<i>32</i>
6.2.2	<i>Perfil de Usuario</i>	<i>32</i>
6.2.3	<i>Modelo de Servicio y Funcionalidades.....</i>	<i>33</i>
6.2.4	<i>Estrategia de Monetización</i>	<i>33</i>
6.2.5	<i>Análisis de Costes y Sostenibilidad</i>	<i>33</i>
7	OBJETIVOS	34
7.1	OBJETIVO GENERAL.....	34
7.2	OBJETIVOS ESPECÍFICOS	34
8	DESARROLLO DEL PROYECTO.....	34

8.1	TECNOLOGÍAS ESCOGIDAS.....	34
8.1.1	<i>Frontend</i>	34
8.1.2	<i>Backend</i>	35
8.1.3	<i>Base de datos</i>	35
8.1.4	<i>Despliegue e infraestructura</i>	36
8.2	METODOLOGÍA DE TRABAJO.....	36
8.2.1	<i>METODOLOGÍAS COMPARADAS</i>	36
8.2.2	<i>Diagrama de gantt</i>	41
8.2.3	<i>Configuración del entorno</i>	42
8.3	ESPECIFICACIÓN DE REQUISITOS	44
8.3.1	<i>Análisis de requisitos</i>	44
8.3.1.1	Requisitos funcionales	44
8.3.1.2	Requisitos no funcionales	45
8.3.2	<i>Definición de funcionalidades</i>	45
8.3.2.1	Gestión de Identidad y Seguridad	45
8.3.2.2	Gestión de perfil público "Link in Bio" (YourTree)	48
8.3.2.3	Sistema de Comunidad y Foro	50
8.3.2.4	Monitorización y Optimización SEO	52
8.4	DISEÑO DE LA APLICACIÓN.....	53
8.4.1	<i>Diseño conceptual</i>	53
8.4.2	<i>Modelado de la base de datos</i>	57
8.4.2.1	Diagrama Entidad Relación (ER)	58
8.4.2.2	Diagrama lógico	59
8.4.2.3	Diagrama físico	60
8.4.2.3.1	Indices realizados	60
8.4.3	<i>Prototipado digital</i>	61
8.4.3.1	Home.....	62
8.4.3.2	Recent pages	62
8.4.3.3	YourTree	63
8.4.3.4	Profile	63
8.4.3.5	Forums	64
8.4.3.6	Sign Up	65
8.4.3.7	Sign In	65
8.4.3.8	YourTreeUser	66
8.4.4	<i>Diagrama de secuencia</i>	66
8.4.5	<i>Diagrama de clases</i>	67
8.4.6	<i>Diagrama de casos de uso</i>	67
8.4.7	<i>Mapa de navegación</i>	68
8.5	IMPLEMENTACIÓN Y ARQUITECTURA.....	68
8.5.1	<i>Arquitecturas utilizadas</i>	68

8.5.1.1	Backend.....	68
8.5.1.1.1	Arquitectura Control-Service-Repository	69
8.5.1.1.2	Arquitectura por capas.....	69
8.5.1.2	Frontend	71
8.5.1.2.1	Public	71
8.5.1.2.2	Src.....	71
8.5.2	<i>Patrones utilizados</i>	72
8.5.2.1	Patrones en el Backend	72
8.5.2.1.1	Patrón Middleware	72
8.5.2.1.2	Patron de repositorio.....	73
8.5.2.1.3	Patron Singleton	73
8.5.2.2	Patrones en el Frontend	73
8.5.2.2.1	Patrón de Composición de Componentes	73
8.5.2.2.2	Patrón Proveedor	73
8.5.2.2.3	Patrón de Enrutamiento del lado del Cliente	73
8.5.3	<i>Desarrollo del backend</i>	74
8.5.3.1	Elección del backend	74
8.5.3.1.1	Comandos para iniciar	74
8.5.3.1.2	Ficheros necesarios	75
8.5.3.2	Directorios y ficheros utilizados	76
8.5.3.2.1	Config.....	76
8.5.3.2.2	Controllers.....	76
8.5.3.2.3	Db	77
8.5.3.2.4	Middlewares	78
8.5.3.2.5	Models.....	78
8.5.3.2.6	Public	79
8.5.3.2.7	Repository	79
8.5.3.2.8	Routes	80
8.5.3.2.9	Services	82
8.5.3.2.10	Upload.....	83
8.5.3.2.11	Utils	83
8.5.4	<i>Desarrollo del frontend</i>	84
8.5.4.1	Elección del frontend	84
8.5.4.1.1	Comandos para iniciar	84
8.5.4.1.2	Ficheros necesarios	85
8.5.4.2	Directorios y ficheros utilizados	86
8.5.4.2.1	Public	86
8.5.4.2.2	Src.....	88
8.5.5	<i>Descripción de los componentes (Frontend)</i>	98
8.5.5.1	Componentes de lógica y seguridad	98
8.5.5.1.1	Auth.jsx	98

8.5.5.1.2	Token.jsx	99
8.5.5.1.3	UsePassword.jsx	99
8.5.5.2	Componentes de Optimización y Servicios	100
8.5.5.2.1	GA4.jsx	100
8.5.5.2.2	SEO.jsx	100
8.5.5.3	Componentes de estructura e interfaz	101
8.5.5.3.1	Header.jsx	101
8.5.5.3.2	Collapsible.jsx	101
8.5.6	<i>Estructura del proyecto</i>	102
8.5.6.1	Capas de responsabilidad	102
8.5.6.2	Capa de Interfaz (Frontend)	102
8.5.6.3	Capa de Lógica de Negocio (Backend)	102
8.5.6.4	Capa de Persistencia (Base de Datos)	102
8.5.6.5	Estructura del repositorio	102
8.5.6.5.1	Directorio raíz (/)	102
8.5.6.5.2	Frontend	106
8.5.6.5.3	Backend	111
8.5.7	<i>Herramientas utilizadas en el proyecto</i>	117
8.5.7.1	Entorno de desarrollo y gestión de código	117
8.5.7.2	Gestión de datos y bases de datos	118
8.5.7.3	Pruebas, depuración y seguridad	118
8.5.7.4	Infraestructura y despliegue (DevOps)	119
8.5.7.5	Creación de maquetados	120
8.6	PLAN DE PRUEBAS	121
8.6.1	<i>Análisis de Vulnerabilidades con OWASP ZAP</i>	121
8.6.2	<i>Medidas de Seguridad Activa e Infraestructura</i>	122
8.6.3	<i>Visualización de datos</i>	123
8.7	DESPLIEGUE Y MANTENIMIENTO	124
8.7.1	<i>Infraestructura de servidores</i>	124
8.7.1.1	Separación de Servicio	124
8.7.1.2	Capa de Proxy y Seguridad	125
8.7.2	<i>Diagrama de flujo de peticiones</i>	125
8.7.3	<i>Monitoreo y SEO</i>	126
8.7.3.1	Monitorización de errores	126
8.7.3.2	Optimización SEO y rastreo	126
9	CONCLUSIÓN	126
9.1	CONCLUSIÓN GENERAL DEL PROYECTO	126
9.2	CONCLUSIÓN DE OBJETIVOS	127
9.3	VALORACIÓN DE METODOLOGÍA Y PLANIFICACIÓN	127
9.4	VISIÓN A FUTURO	127

10	REFERENCIAS.....	128
11	ANEXOS	136
11.1	PLAN DE EMPRESA FUTURO	136
11.2	MANUAL DE USUARIO	136
11.2.1	<i>Ingreso a la página web</i>	136
11.2.2	<i>Secciones que se repiten</i>	136
11.2.2.1	Header.....	136
11.2.2.2	Buy me a coffee.....	137
11.2.3	<i>Páginas y uso respectivo</i>	137
11.2.3.1	Página base o Home.....	137
11.2.3.2	Sign in	138
11.2.3.3	Sign up	139
11.2.3.4	YourTree	139
11.2.3.4.1	Interfaz de Gestión y Configuración	139
11.2.3.4.2	Módulo de Vista Previa (Preview)	140
11.2.3.5	Recent Pages	141
11.2.3.5.1	Sin perfil público	141
11.2.3.5.2	Con perfil público.....	142
11.2.3.6	Forums	142
11.2.3.6.1	Creación de un foro.....	143
11.2.3.7	@username o perfil	144
11.2.3.8	Log out	145
11.3	HISTORIAL DE VERSIONES	146
11.3.1	<i>Versión 0.1.0</i>	146
11.3.2	<i>Versión 0.1.1</i>	146
11.3.3	<i>Versión 0.1.2</i>	146
11.3.4	<i>Versión 0.1.3</i>	147
11.3.5	<i>Versión 0.1.4</i>	148
11.3.6	<i>Versión 0.2.0</i>	149
11.3.7	<i>Versión 1.0.0</i>	150
11.3.8	<i>Versión 1.0.1</i>	151
11.3.9	<i>Versión 1.3.0</i>	152
11.3.10	<i>Versión 1.5.0</i>	154
11.4	INFORME DE AUDITORÍA DE SEGURIDAD (OWASP ZAP).....	157
11.4.1.1	Acceso al reporte detallado.....	158
11.4.1.2	Descarga y ejecución local	158
11.5	FRAGMENTOS DE CÓDIGO IMPORTANTES	159
11.5.1	<i>PutLinks</i>	159
11.5.2	<i>Collapsible</i>	160

11.5.3	<i>verifyToken</i>	160
11.5.4	<i>refreshAccessToken</i>	161

TABLA DE ILUSTRACIONES

ILUSTRACIÓN 1 REFERENCIA DE MILKSHAKE FREE	25
ILUSTRACIÓN 2 REFERENCIA DE MILKSHAKE FREE	25
ILUSTRACIÓN 3 REFERENCIA DE MILKSHAKE PAID	25
ILUSTRACIÓN 4 REFERENCIA DE LINKTREE FREE	25
ILUSTRACIÓN 5 REFERENCIA DE LINKTREE FREE	25
ILUSTRACIÓN 6 REFERENCIA DE LINKTREE PAID	25
ILUSTRACIÓN 7 REFERENCIA DE BEACONS FREE	26
ILUSTRACIÓN 8 REFERENCIA DE BEACONS PAID	26
ILUSTRACIÓN 9 REFERENCIA DE BEACONS PAID	26
ILUSTRACIÓN 10 REFERENCIA DE CAMPSITE FREE.....	27
ILUSTRACIÓN 11 REFERENCIA DE CAMPSITE FREE.....	27
ILUSTRACIÓN 12 REFERENCIA DE CAMPSITE PAID.....	27
ILUSTRACIÓN 13 REFERENCIA DE DEMO.TREEDLINK COMUNITARIO.....	29
ILUSTRACIÓN 14 REFERENCIA DE DEMO.TREEDLINK 3	29
ILUSTRACIÓN 15 REFERENCIA DE DEMO.TREEDLINK 2	29
ILUSTRACIÓN 16 REFERENCIA DE DEMO.TREEDLINK 1	29
ILUSTRACIÓN 17 BACKEND EN JIRA.....	38
ILUSTRACIÓN 18 FRONTEND EN JIRA.....	40
ILUSTRACIÓN 19 BASE DE DATOS Y DOCKER EN JIRA.....	41
ILUSTRACIÓN 20 DIAGRAMA DE GANTT	41
ILUSTRACIÓN 21 DISEÑO CONCEPTUAL PAPEL 1.....	53
ILUSTRACIÓN 22 DISEÑO CONCEPTUAL PAPEL 2.....	54
ILUSTRACIÓN 23 DISEÑO CONCEPTUAL PAPEL 3.....	55
ILUSTRACIÓN 24 DISEÑO CONCEPTUAL PAPEL 4.....	56
ILUSTRACIÓN 25 DISEÑO CONCEPTUAL PAPEL 5.....	56
ILUSTRACIÓN 26 DIAGRAMA ENTIDAD RELACIÓN (ER)	59
ILUSTRACIÓN 27 DIAGRAMA LOGICO	60
ILUSTRACIÓN 28 VISTA GENERAL DEL ENTORNO DE ILLUSTRATOR.....	61
ILUSTRACIÓN 29 PROTOTIPO DE LA WEB (HOME)	62
ILUSTRACIÓN 30 PROTOTIPO DE LA WEB (RECENT PAGES)	63
ILUSTRACIÓN 31 PROTOTIPO DE LA WEB (YOURTREE)	63
ILUSTRACIÓN 32 PROTOTIPO DE LA WEB (PROFILE).....	64
ILUSTRACIÓN 33 PROTOTIPO DE LA WEB (FORUMS)	64
ILUSTRACIÓN 34 PROTOTIPO DE LA WEB (SIGN UP).....	65
ILUSTRACIÓN 35 PROTOTIPO DE LA WEB (SIGN IN).....	65
ILUSTRACIÓN 36 PROTOTIPO DE LA WEB (YOURTREEUSER).....	66
ILUSTRACIÓN 37 DIAGRAMA DE SECUENCIA	66
ILUSTRACIÓN 38 DIAGRAMA DE CLASES	67

ILUSTRACIÓN 39 DIAGRAMA DE USO	67
ILUSTRACIÓN 40 MAPA DE NAVEGACIÓN	68
ILUSTRACIÓN 41 ARQUITECTURA DEL BACKEND	71
ILUSTRACIÓN 42 ARQUITECTURA DEL FRONTEND	72
ILUSTRACIÓN 43 ESTRUCTURA DEL DIRECTORIO RAÍZ (/)	104
ILUSTRACIÓN 44 DEPENDENCIAS UTILIZADAS EN EL FRONTEND	108
ILUSTRACIÓN 45 ESTRUCTURA DE DIRECTORIOS FRONTEND	111
ILUSTRACIÓN 46 ESTRUCTURA DE DIRECTORIOS BACKEND	116
ILUSTRACIÓN 47 RESULTADO DE LA AUDITORIA DE ZAP	122
ILUSTRACIÓN 48 DIAGRAMA DEL FLUJO DE PETICIONES	125
ILUSTRACIÓN 49 MANUAL DE USUARIO (HOME).....	138
ILUSTRACIÓN 50 MANUAL DE USUARIO (SIGN IN)	138
ILUSTRACIÓN 51 MANUAL DE USUARIO (SIGN UP).....	139
ILUSTRACIÓN 52 MANUAL DE USUARIO (YOURTREE)	141
ILUSTRACIÓN 53 MANUAL DE USUARIO (RECENT PAGE SIN PERFIL PUBLICO).....	141
ILUSTRACIÓN 54 MANUAL DE USUARIO (RECENT PAGE CON PERFIL PUBLICO)	142
ILUSTRACIÓN 55 MANUAL DE USUARIO (FORUM)	143
ILUSTRACIÓN 56 MANUAL DE USUARIO (CREACIÓN DE UN FORO)	144
ILUSTRACIÓN 57 MANUAL DE USUARIO (PROFILE O @USERNAME)	145
ILUSTRACIÓN 58 INICIO DEL PROYECTO	146
ILUSTRACIÓN 59 ESTRUCTURA BÁSICA DEL HOME	146
ILUSTRACIÓN 60 ESTILOS BÁSICOS IMPLEMENTADOS.....	147
ILUSTRACIÓN 61 LOADER DE INICIO	147
ILUSTRACIÓN 62 VISTA DEL HOME CON ESTILOS MAS ACORDE	148
ILUSTRACIÓN 63 VISTA CON IMÁGENES DEL HOME.....	149
ILUSTRACIÓN 64 TEMA OSCURO DEL HOME	150
ILUSTRACIÓN 65 ESTRUCTURA DEL FRONTEND	151
ILUSTRACIÓN 66 HOME REALIZADO CON REACT	151
ILUSTRACIÓN 67 CREACION DEL BACKEND CON EXPRESS	152
ILUSTRACIÓN 68 PRIMERA VERSIÓN DE LA BASE DE DATOS	152
ILUSTRACIÓN 69 SEGUNDA VERSIÓN DE LA BASE DE DATOS.....	153
ILUSTRACIÓN 70 TERCERA VERSIÓN DE LA BASE DE DATOS	153
ILUSTRACIÓN 71 RUTA /.....	154
ILUSTRACIÓN 72 RUTA TEMPLATES	154
ILUSTRACIÓN 73 RUTA PLANS	155
ILUSTRACIÓN 74 RUTA TOP PAGES	155
ILUSTRACIÓN 75 RUTA FÓRUMS	155
ILUSTRACIÓN 76 RUTA SIGN UP.....	156
ILUSTRACIÓN 77 RUTA SIGN IN	157
ILUSTRACIÓN 78 RUTA HOME	157

1 INTRODUCCIÓN

Actualmente curso el segundo año del Grado Superior en Desarrollo de Aplicaciones Web (DAW) en el IES Virgen de la Paloma, y el presente documento constituye la memoria del Proyecto Intermodular del ciclo, necesario para superar el curso satisfactoriamente.

En la actualidad, existe una gran presencia de creadores de contenido, profesionales y pequeños proyectos que buscan darse a conocer y centralizar todos sus enlaces en un único lugar para facilitar el acceso a sus seguidores o usuarios. En muchos casos mantienen cuentas en diferentes plataformas con distintos propósitos, lo que puede resultar poco práctico tanto para ellos como para quienes desean consultar sus recursos, y cambiar constantemente entre sitios web para localizar un contenido ya publicado anteriormente supone una experiencia incómoda y poco eficiente.

Por este motivo, YourTree fue creado con el objetivo de permitir a los usuarios compartir sus logros, recursos y contenidos de forma rápida, segura y centralizada. Además, incorpora una sección de foros en la que los usuarios pueden plantear dudas y recibir respuestas de otros miembros de la comunidad, reuniendo todo en una única plataforma accesible y estructurada y evitando la necesidad de acudir a diferentes sitios web para buscar un recurso concreto o resolver una cuestión específica.

2 RESUMEN Y ABSTRACT

2.1 RESUMEN (CASTELLANO)

YourTree es una plataforma web tipo link-in-bio orientada a creadores de contenido, profesionales, pequeñas empresas y cualquier usuario que necesite tener sus enlaces más importantes en un único lugar. El objetivo de la aplicación es facilitar que cualquier persona pueda compartir sus perfiles, proyectos, redes sociales y recursos digitales desde una página personalizable y fácil de distribuir.

Permite organizar enlaces de forma visual y accesible desde una única URL, mejorando la presencia online del usuario sin necesidad de redirigir a múltiples plataformas. Su arquitectura está diseñada para ser escalable, segura y desplegable con tecnologías modernas de desarrollo web.

Incorpora también una sección de foros donde los usuarios pueden publicar dudas sobre temas concretos y recibir ayuda mediante comentarios de otros miembros, fomentando la colaboración entre usuarios y creadores.

El proyecto abarca desarrollo frontend, backend, base de datos y despliegue real, integrando autenticación JWT, búsquedas dinámicas y medidas de seguridad activas para construir un producto web funcional a nivel profesional.

Palabras claves:

- Link-in-bio
- Centralizar
- Difusión digital
- Organización de enlaces
- Presencia online
- Foros comunitarios
- Autenticación JWT.

2.2 ABSTRACT (INGLÉS)

YourTree is a link-in-bio web platform aimed at content creators, professionals, small businesses and any user who needs to keep their most important links in one place. The goal of the application is to make it easy to share profiles, projects, social media accounts and digital resources through a single customizable page.

Users can organize their links visually and share them through a unique URL, improving their online presence without having to redirect visitors across multiple platforms. The architecture is designed to be scalable, secure and deployable using modern web technologies.

The platform also includes a forum section where users can post questions on specific topics and receive help through community comments, encouraging collaboration between users and creators.

The project covers frontend and backend development, database design and real deployment, integrating JWT authentication, dynamic queries and active security practices to build a fully functional web product.

Key Words:

- Link-in-bio
- Centralize
- Digital sharing
- Link organization
- Online presence
- Community forums
- JWT authentication.

3 GLOSARIO

ACID (Atomicity, Consistency, Integrity, Durability)

Conjunto de propiedades en bases de datos que garantizan transacciones fiables: atomicidad (todo o nada), consistencia (respeto de reglas), integridad (datos válidos) y durabilidad (persistencia tras fallos).

Asincronia

Modelo de ejecución donde las operaciones no bloquean el hilo principal, usando mecanismos como promesas o `async/await`.

Asíncrono

Ejecución no secuencial que permite continuar el flujo sin esperar a que una tarea finalice.

Backend

Capa del sistema encargada de la lógica de negocio, acceso a datos y procesamiento en servidor.

Background

Ejecución de procesos en segundo plano, desacoplados del flujo principal.

Boilerplate

Código base inicial que define la estructura mínima de un proyecto.

Boolean

Tipo de dato lógico con dos valores posibles: `true` o `false`.

Branding

Proceso de construcción y gestión de identidad de marca.

Bundler

Herramienta que empaqueta y optimiza recursos (JS, CSS, HTML) resolviendo dependencias.

Children

Propiedad en componentes (React) que permite renderizar elementos hijos.

Contexto

Mecanismo para compartir estado entre componentes sin pasar props manualmente.

Cookies

Almacenamiento en navegador en formato clave-valor para persistencia de sesión o datos.

CORS (Cross-Origin Resource Sharing)

Política de seguridad que regula solicitudes entre distintos dominios.

CSR (Control-Service-Repository)

Patrón arquitectónico que separa controladores, lógica de negocio y acceso a datos.

CTR (Click-Through Rate)

Métrica que mide clics sobre impresiones.

Debounce

Técnica que retrasa la ejecución de una función hasta que cesan llamadas repetidas.

Dependencias

Paquetes externos necesarios para el funcionamiento del proyecto.

DevDependencies

Dependencias usadas solo en desarrollo.

DI (Dependency Injection)

Patrón donde las dependencias se proporcionan externamente para desacoplar componentes.

Directivas de rastreo

Reglas para motores de búsqueda (robots.txt, meta tags) sobre indexación.

DNS (Domain Name System)

Sistema que traduce dominios a direcciones IP.

DOM (Document Object Model)

Representación en árbol del HTML que permite manipulación dinámica mediante scripts.

Endpoint

Punto específico de acceso a una API mediante HTTP.

Etiqueta (HTML tag)

Elemento sintáctico que define nodos en el DOM (ej. <div>).

FN (Forma Normal)

Reglas de normalización en bases de datos para evitar redundancias.

Frontend

Capa de presentación ejecutada en el cliente (navegador).

Granular

Alto nivel de detalle o segmentación en componentes o procesos.

Hash

Función que transforma datos en una cadena fija, usada en seguridad e integridad.

Hooks

Funciones en React que permiten gestionar estado y ciclo de vida en componentes funcionales.

Infraestructura

Conjunto de recursos técnicos (servidores, redes, cloud) que soportan aplicaciones.

Input

Dato o elemento de entrada proporcionado por el usuario.

JWT (JSON Web Token)

Token firmado que permite autenticación y autorización sin estado.

Librerías

Colecciones reutilizables de código que abstraen funcionalidades comunes.

Multi-stage build

Técnica en contenedores que reduce tamaño final separando fases de build.

MVP (Minimum Viable Product)

Versión mínima funcional para validar una idea.

Nginx

Servidor web y proxy inverso de alto rendimiento.

Nucleares

Elementos esenciales o críticos del sistema.

Open Graph

Protocolo de metadatos para definir cómo se comparte contenido en redes sociales.

Optimistic UI

Técnica donde la interfaz se actualiza antes de confirmar respuesta del servidor.

Orquestación

Coordinación automatizada de múltiples servicios o contenedores.

Overhead

Coste adicional en recursos debido a abstracciones o procesos extra.

Overlay

Capa visual superpuesta sobre la interfaz principal.

Pageview

Registro de visualización de una página en analítica web.

Pnpm (Performant npm)

Gestor de paquetes optimizado en rendimiento y almacenamiento.

Pool

Conjunto de recursos reutilizables (ej. conexiones a base de datos).

PostgreSQL

Sistema de gestión de bases de datos relacional avanzado y open source.

Props

YourTree – Proyecto FP

Nicolas Pezo Ccama

Mecanismo de paso de datos entre componentes en React.

Proxy

Servidor intermediario entre cliente y servidor.

Proxy inverso

Servidor que redirige peticiones externas a servicios internos.

React.memo

Función que optimiza renderizado evitando ejecuciones innecesarias.

Refresh Token

Token que permite renovar credenciales sin reautenticación.

Renderizar

Proceso de generar interfaz visual a partir de datos o componentes.

Responsive

Diseño adaptable a distintos dispositivos y resoluciones.

SameSite

Atributo de cookies que controla su envío en contextos cross-site.

Script

Conjunto de instrucciones ejecutables en un entorno.

SEO (Search Engine Optimization)

Técnicas para mejorar posicionamiento en buscadores.

SGBD (Sistema de Gestión de Bases de Datos)

Software para administrar bases de datos.

Síncrono

Ejecución secuencial donde cada operación bloquea hasta finalizar.

SQL (Structured Query Language)

Lenguaje para gestionar bases de datos relacionales.

Sprint

YourTree – Proyecto FP

Nicolas Pezo Ccama

Iteración de trabajo en metodologías ágiles con duración fija.

Sparse-checkout

Función de Git que permite descargar únicamente un subconjunto de archivos de un repositorio, sin clonar su contenido completo.

SSL (Secure Sockets Layer)

Protocolo de seguridad (obsoleto, reemplazado por TLS).

TLS (Transport Layer Security)

Protocolo criptográfico moderno para comunicaciones seguras.

Token

Credencial digital que representa identidad o permisos.

Tracker

Script que recopila datos de comportamiento del usuario.

UI (User Interface)

Interfaz visual e interactiva del sistema.

Unopinionated

Herramienta que no impone estructura ni convenciones estrictas.

UseEffect

Hook de React para ejecutar efectos secundarios.

UseLocation

Hook para acceder a información de la URL en routing.

UseState

Hook para gestionar estado en componentes funcionales.

UX (User Experience)

Experiencia global del usuario al interactuar con un sistema.

Vanilla

Implementación sin frameworks (ej. JavaScript puro).

VPS (Virtual Private Server)

Servidor virtualizado que simula uno dedicado.

WSL (Windows Subsystem for Linux)

Capa que permite ejecutar Linux dentro de Windows.

ZAP (Zed Attack Proxy)

Herramienta de seguridad para detectar vulnerabilidades web.

4 TECNOLOGÍAS ACTUALES

4.1 COMPARATIVA DE TECNOLOGÍAS ACTUALES

4.1.1 Frontend – comparativa de frameworks

Framework / Biblioteca	Ventajas	Desventajas
React	Ofrece un ecosistema masivo basado en componentes reutilizables, lo que resulta óptimo para el desarrollo de aplicaciones interactivas y dinámicas. (Meta, 2024)	La gestión de estados suele incrementar su complejidad conforme la aplicación crece, lo que suele requerir la integración de herramientas externas adicionales. (MDN, 2026)
Vue.js	Destaca por una curva de aprendizaje suave y una sintaxis clara, permitiendo el uso de componentes reactivos de manera intuitiva, especialmente en equipos de trabajo reducidos. (Vue.js, s.f.)	Posee un ecosistema más limitado en comparación con React y presenta una menor tasa de adopción en proyectos de gran escala. (MDN, 2025)
Angular	Proporciona una solución integra que incluye de serie enrutador, cliente	Presenta una curva de aprendizaje elevada, además de generar un

	HTTP, validación de formularios e inyección de dependencias (DI), todo bajo un tipado estricto con TypeScript. (Angular, s.f.)	mayor código base (<i>boilerplate</i>) y un peso de archivo superior al de alternativas como React o Vue. (MDN, 2025)
Svelte	A diferencia de otros frameworks, compila a JavaScript puro sin utilizar un Virtual DOM, lo que garantiza un <i>bundle</i> muy ligero y un alto rendimiento en el navegador. (Svelte, s.f.)	Cuenta con un ecosistema reducido, disponibilidad limitada de librerías de componentes y una menor madurez para su implementación en proyectos complejos. (Kinsta, 2025)
Astro	Implementa la "Arquitectura Islands", la cual carga únicamente el código JavaScript estrictamente necesario, optimizando el SEO y el rendimiento en sitios de contenido estático. (Astro, s.f.)	No se considera la opción adecuada para Single Page Applications (SPAs) que requieren una interactividad constante en el cliente, siendo más utilizado para tableros de control o <i>dashboards</i> reactivos. (Astro, s.f.)

4.1.2 Backend – comparativa de frameworks

Framework + lenguaje	Ventajas	Desventajas
Node.js + Express	Ligereza extrema y libertad total de arquitectura. Ideal para MVPs de alto rendimiento y microservicios rápidos. (MDN, 2025)	Falta de estándares, al no imponer una estructura, el código puede volverse caótico si el equipo o programa crece. (MDN, 2025)
Node.js + NestJS	Arquitectura escalable basada en módulos que facilita el mantenimiento mediante la inyección de dependencias (DI). Ofrece	Overhead, introduce mucha complejidad y archivos innecesarios para aplicaciones simples o

	soporte nativo y tipado fuerte con TypeScript. (NestJS , s.f.)	prototipos pequeños. (mytaskpanel, s.f.)
Python + Django	Ayuda a crear proyectos de forma rápida y eficiente. Además, incluye funciones de seguridad y administración que se pueden usar al inicio. (JetBrains, s.f.)	Es más difícil de optimizar para alta concurrencia comparado con Node.js (JetBrains, s.f.)
PHP + Laravel	Máxima productividad usando un lenguaje claro junto con herramientas integradas que facilitan tareas de sesión, gestión de colas y procesar pagos. (InMotion Hosting, 2025)	Al ser un lenguaje interpretado tradicionalmente síncrono, consume más recursos por petición que Node.js. (dongee, 2024)
Java + SpringBoot	Ofrece la máxima seguridad y puede manejar varias tareas al mismo tiempo. Es ideal para sistemas que requieren procesar muchas transacciones. (Microsoft Azure, s.f.)	Requiere servidores con mucha RAM y tiene boilerplates muy extensas. (campusMVP.es, 2022)

4.1.3 Base de datos - comparativa de SGBD

SGBD	Ventajas	Desventajas
PostgreSQL	SGBD relacional de código abierto y con soporte para datos complejos, robustez y total cumplimiento de	Configuración y administración más compleja que otros sistemas, puede llegar a ser lento en lectura de datos masivo.

	propiedades ACID. (TodoPostgreSQL, 2018)	
MySQL	SGBD con una gran comunidad, tiene un excelente rendimiento en operaciones de lectura y fácil desplegar en entornos estándar de hosting. (Inesem, s.f.)	Históricamente menos estricto con la integridad de datos que PostgreSQL. Algunas funciones avanzadas requieren licencias comerciales. (Inesem, s.f.)
MongoDB	Ofrece gran escalabilidad horizontal y un esquema flexible que permite iterar rápido en el desarrollo. (Inesem, s.f.)	No es nativa para relaciones complejas entre datos y tiene un mayor consumo de memoria. (Inesem, s.f.)
SQLite	Base de datos local. Ideal para desarrollo, aplicaciones móviles o proyectos pequeños. (Inesem, s.f.)	No es apta para aplicaciones con múltiples escrituras simultáneas. Carece de un sistema de gestión de usuarios y permisos avanzado. (Inesem, s.f.)
Oracle	Tiene un gran rendimiento y seguridad en entornos empresariales críticos. Soporta volúmenes de datos masivos y herramientas de administración profesionales. (Inesem, s.f.)	Coste de licenciamiento muy elevado. Requiere hardware potente y personal especializado (DBA) para su mantenimiento. (Inesem, s.f.)

Tras el análisis comparativo de las tecnologías disponibles para las capas de frontend, backend y base de datos, en la siguiente sección se presentan y justifican las soluciones técnicas seleccionadas. Esta elección se basa en criterios de escalabilidad, rendimiento y adecuación a los requisitos específicos del proyecto.

Elección final: React, Express + Node.js y PostgreSQL

4.2 COMPARATIVA CON LA COMPETENCIA

4.2.1 Aplicaciones Link-in-BIO

En este apartado se analizan algunas de las plataformas Link-in-Bio más conocidas, comparando sus puntos fuertes y sus limitaciones frente a YourTree, que busca ofrecer un equilibrio entre personalización, rendimiento y experiencia de usuario.

4.2.1.1 *Milkshake.app*

Milkshake apuesta por un modelo de diseño tipo "story" optimizado exclusivamente para dispositivos móviles, priorizando la narrativa visual y gráfica sobre la estructura de enlaces tradicional. Sin embargo, bajo un análisis técnico y de usuario, se observan los siguientes puntos:

- **Interfaz y Usabilidad:** Aunque su interfaz es intuitiva, la disposición de las tarjetas puede generar desorden visual en ciertas configuraciones, lo que perjudica la claridad del perfil del usuario. Según revisiones especializadas en la plataforma, esta estructura puede resultar caótica para el usuario final si no se gestiona con rigor. (The Blessing Aniedi, n.d.)
- **Diseño y Accesibilidad:** Los contrastes de color predefinidos no siempre cumplen con los estándares óptimos de legibilidad y, debido a que no son fácilmente ajustables, pueden impactar negativamente en la experiencia visual.
- **Personalización y Coste:** Si bien ofrece una variedad de plantillas, fuentes e iconos, muchas de estas opciones avanzadas de personalización están vinculadas a su modelo de suscripción, limitando la capacidad del usuario en el plan gratuito para diferenciar su marca personal. (Milkshake, n.d.)

Ilustración 1 referencia de milkshake free

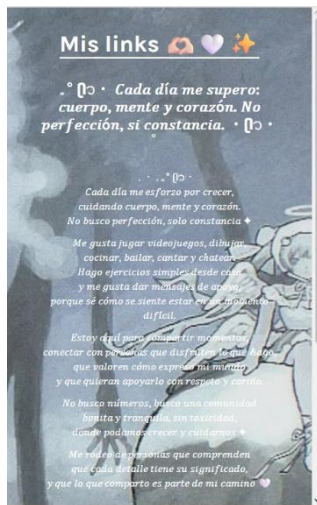


Ilustración 2 referencia de milkshake free

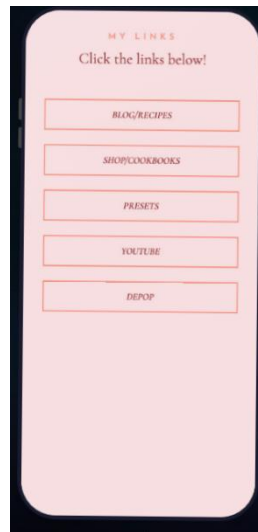


Ilustración 3 referencia de milkshake paid



4.2.1.2 LinkTr.ee

Es la plataforma más conocida dentro del ecosistema Link-in-Bio, con una curva de adopción muy baja y una estructura basada exclusivamente en una lista de enlaces. Su simplicidad es su mayor virtud, pero también su principal limitación para usuarios que buscan algo más que un listado básico.

El plan gratuito es muy restrictivo: únicamente permite cambiar el color y el contorno de los botones, sin opciones de branding propio, diseños avanzados ni estructuras de página complejas. Es una solución válida para montar rápidamente una lista de enlaces, pero resulta insuficiente para quienes quieren construir una identidad visual sólida o una marca personal reconocible. (Linktree, n.d.)

Ilustración 4 referencia de linktree free

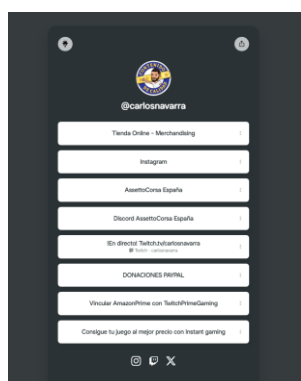


Ilustración 5 referencia de linktree free

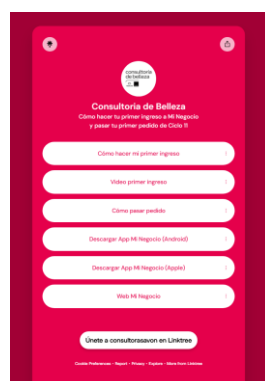
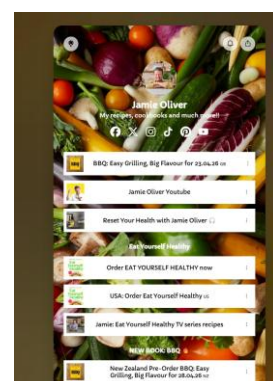


Ilustración 6 referencia de linktree paid

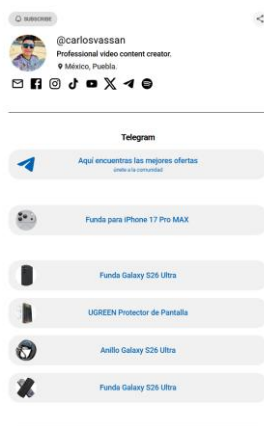


4.2.1.3 Beacons.ai

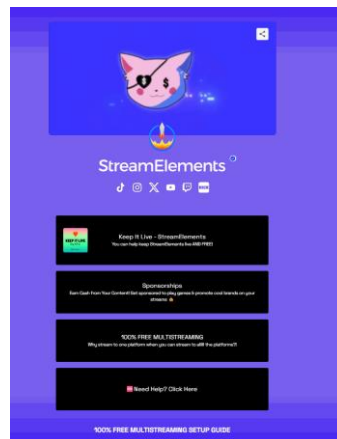
Beacons presenta un diseño flexible y moderno que va más allá del simple listado de enlaces, permitiendo añadir bloques de bio, productos, vídeos, formularios y otros módulos. Ofrece analíticas en tiempo real, integración de herramientas de monetización y automatizaciones de marketing, lo que lo convierte en una opción más completa para creadores con necesidades avanzadas. (Beacons, n.d.)

Su punto débil más claro es el rendimiento en pantallas grandes: el perfil puede volverse lento o poco intuitivo en escritorio o portátil, dificultando la localización del contenido.

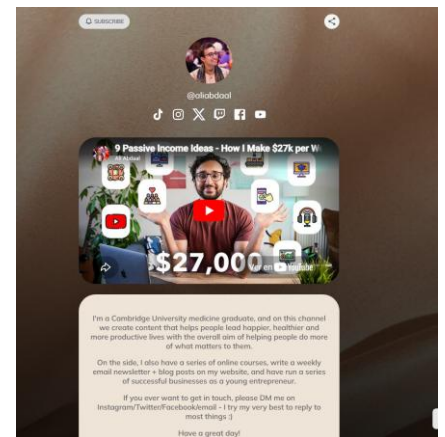
*Ilustración 7 referencia de
beacons free*



*Ilustración 8 referencia de
beacons paid*



*Ilustración 9 referencia de
beacons paid*



4.2.1.4 Campsite.bio

Campsite apuesta por un estilo profesional y limpio, más personalizable que Linktree pero menos expresivo visualmente que Milkshake. Permite ajustar colores y fondos dentro de ciertos límites, aunque en el plan gratuito el nivel de personalización es bastante restrictivo y depende casi por completo de plantillas predefinidas. Su rendimiento es rápido y optimizado, lo que lo hace especialmente adecuado para perfiles con muchos enlaces sin sobrecargar visualmente al visitante. (Campsite, n.d.)

Ilustración 10 referencia de
campsite free

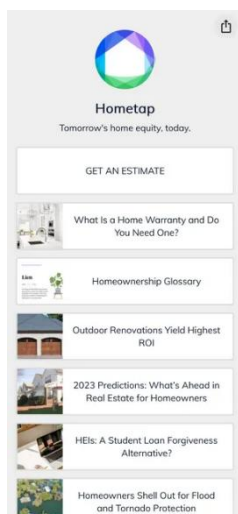
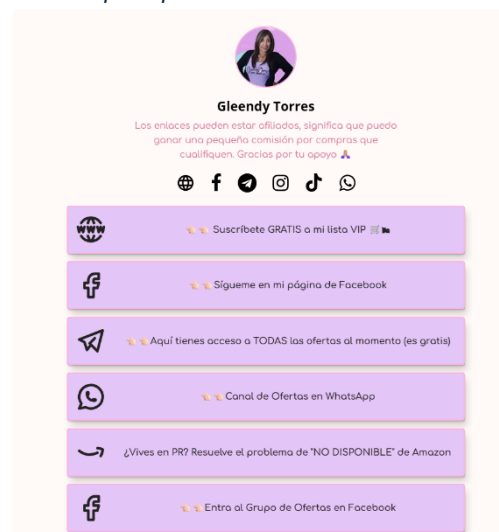


Ilustración 11 referencia de
campsite free



Ilustración 12 referencia de
campsite paid



4.2.2 Comparativa gráfica

Característica	Producto competencia	Producto propio
Personalización de Imágenes e Iconos	Restringida a planes de pago (Pro/Premium). La versión gratuita limita a iconos predeterminados.	Permite usar URLs de imágenes e iconos personalizados para cada enlace sin ningún coste.
Integración de Comunidad (Foros)	Inexistente. Son plataformas unidireccionales (el creador expone, el usuario clicca).	Bidireccionalidad mediante foros integrados que permiten crear hilos, responder y construir comunidad sin salir de la aplicación.
Marca de Agua de la Plataforma	En las versiones gratuitas es obligatorio mostrar el logo de la plataforma en la parte inferior siendo un uno de los elementos más visibles.	El usuario es dueño de su perfil, dándole un aspecto más profesional sin pagar por ocultar o minimizar logos.
Monetización	Se suele cobrar mediante pasarelas de pago para usar planes con mejoras.	Integración directa de links de apoyo externo sin que la plataforma retenga

		ningún porcentaje de las ganancias
Visibilidad y Ocultación de Enlaces	Programar enlaces para que aparezcan/desaparezcan temporalmente es una función Premium.	Posibilidad de activar, desactivar o gestionar el estado (CRUD) de los enlaces en tiempo real de forma nativa.
SEO y Metadatos del Perfil	Personalizar el título SEO y la descripción para Google está bloqueado en planes básicos.	Estructura pensada para que el perfil del usuario tenga buen alcance orgánico sin costes adicionales.
Código y Transparencia	Sistemas propietarios cerrados (Black box). El usuario no sabe cómo se procesan sus datos.	Open Source o de código abierto y auditable en GitHub, garantizando privacidad, escalabilidad y contribución comunitaria.
Buscador Global Inteligente	No disponen de buscadores entre perfiles. El tráfico debe llegar forzosamente desde redes sociales externas.	Buscador global integrado que permite encontrar tanto perfiles de otros creadores como hilos de foro interesantes.

Ilustración 13 referencia de demo.treedlink comunitario

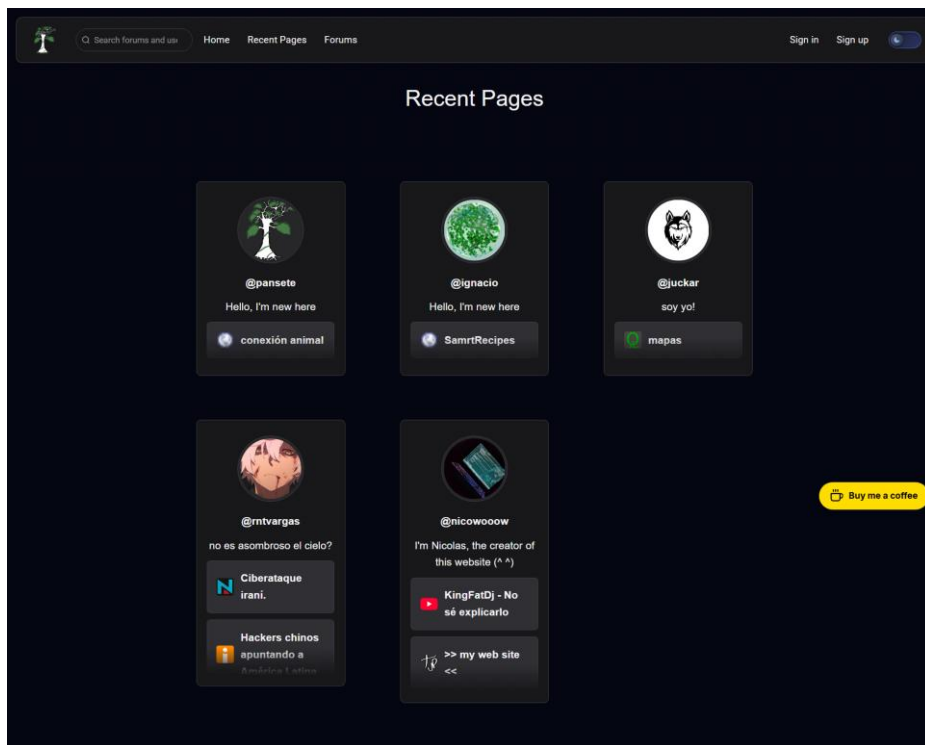


Ilustración 16 referencia de demo.treedlink 1

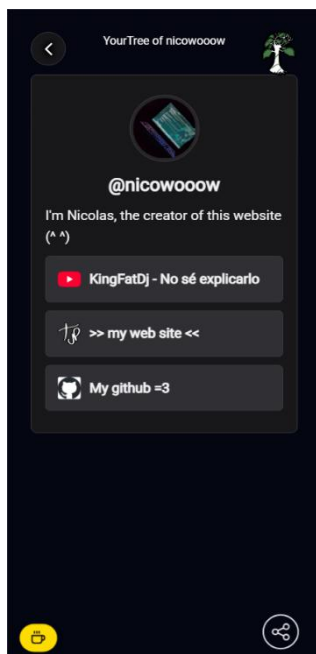


Ilustración 15 referencia de demo.treedlink 2

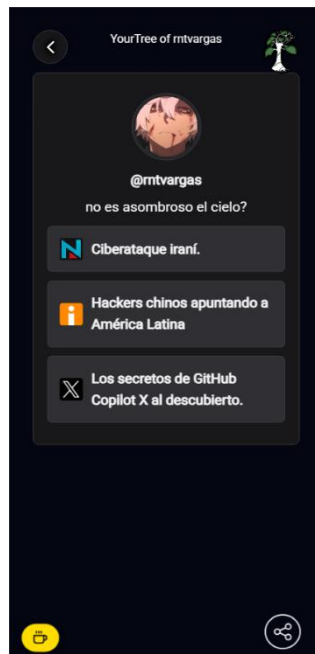
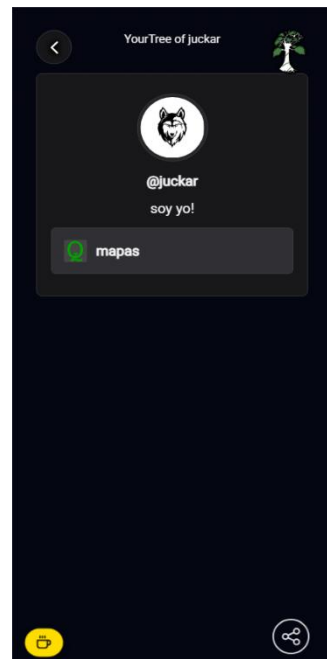


Ilustración 14 referencia de demo.treedlink 3



5 JUSTIFICACIÓN

Actualmente existen plataformas similares como Linktree que cubren la necesidad básica de centralizar enlaces, pero presentan limitaciones relevantes en sus planes gratuitos. Por ejemplo, la personalización visual de los enlaces mediante una URL de imagen propia queda restringida a planes de pago, lo que obliga al usuario a adaptarse a las opciones predefinidas de la plataforma o asumir un coste mensual. Además, si un usuario tiene una duda relacionada con su contenido o con el uso de la herramienta, no tiene otra opción que acudir a plataformas externas como Reddit o foros especializados para resolverla.

YourTree nace precisamente para cubrir esas carencias. Cualquier persona puede centralizar y compartir lo que considera importante “sus conocimientos, logros, proyectos o actividades” desde una única página personalizable y sin restricciones de pago. La personalización de los enlaces incluye la posibilidad de asignar una imagen propia mediante URL, sin necesidad de contratar ningún plan.

Además, la plataforma incorpora una sección de foros integrada donde los usuarios pueden plantear dudas sobre temas concretos y recibir respuestas de la comunidad sin salir de la aplicación. Los comentarios de cada hilo se muestran en orden descendente, con el más reciente primero, facilitando el seguimiento de las conversaciones activas.

De esta forma, YourTree combina en un mismo sitio dos necesidades reales: la difusión de contenido propio y la resolución de dudas de forma colaborativa, todo ello de forma gratuita y sin depender de herramientas externas.

5.1 INSPIRACIÓN DEL PROYECTO

En la actualidad las redes sociales tienen un papel fundamental en la difusión de conocimientos, noticias, acontecimientos y proyectos, entre muchas otras cosas. Partiendo de esa realidad, surge la pregunta de por qué, teniendo tantas aplicaciones que cumplen su propósito, resulta tan agobiante tener que gestionar cuentas en distintas plataformas. Esto genera una serie de problemas concretos:

No siempre se dispone del mismo nombre de usuario en todas las plataformas, por lo que si alguien intenta encontrarte no tiene la certeza de que la cuenta que localiza sea realmente la correcta.

La dispersión de recursos en distintos sitios tampoco favorece la simplicidad. Por ejemplo, una empresa puede tener varios enlaces que realizan acciones específicas,

pero si la página donde están alojados no tiene una buena UX/UI, eso puede traducirse en la pérdida de clientes actuales o potenciales.

Por último, en ciertas plataformas el alcance de las publicaciones está condicionado a planes de pago. Una vez que finaliza ese periodo de promoción, las publicaciones pierden visibilidad o actividad en comparación con crear contenido nuevo, lo que obliga al usuario a invertir de forma continua para mantener su presencia.

5.1.1 Nombre del proyecto y del dominio

Una vez definidos los puntos en los que se centra la plataforma, se optó por dos nombres distintos: uno para el proyecto y otro para el dominio.

- **Nombre del proyecto**

El nombre del proyecto es YourTree porque hace alusión a dos vocablos ingleses:

"Your", que es un adjetivo posesivo de segunda persona, y "Tree", que hace referencia a la palabra árbol en español. Al combinarlos se obtiene "Tu árbol", haciendo alusión a que los enlaces que aparecen en la página están ahí porque el usuario así lo decidió.

- **Nombre de DNS**

El nombre del dominio sigue la misma lógica de combinar vocablos ingleses, en este caso tres:

Tree, Feed y Link. Cada uno hace referencia a un concepto clave de la plataforma: Tree alude a que puedes crear una ramificación de enlaces como página pública del usuario, Feed al flujo de contenido que el usuario comparte tanto a través de los enlaces como de los foros, y Link a la forma en la que ese contenido se distribuye por internet, que es precisamente mediante enlaces.

6 COSTES DEL PROYECTO

6.1 ESTUDIO ECONÓMICO Y PRESUPUESTO

A continuación, se detalla el estudio económico del proyecto, considerando tanto los costes de desarrollo como los de infraestructura y herramientas utilizadas.

Concepto	Detalle	Coste estimado
Horas de desarrollo	Aproximadamente 120 horas a 15€/hora (tarifa junior)	1800€
Hosting / VPS	Servidores cloud (Hetzner) por 12 meses	57,96 €
Dominio	Registro de dominio “.com” por 1 año	20,77 €
Certificado SSL	CertBot (gratuito)	0 €
Herramientas de desarrollo	VS Code, GitHub, Bruno, Dbeaver, MobaXterm, ZAP (versiones gratuitas)	0 €
Base de datos	servidores cloud (Hetzner) por 12 meses	57,96 €
Coste total estimado		1.936,69 €

Cabe destacar que los costes recurrentes de infraestructura una vez finalizado el desarrollo se reducen a aproximadamente 14,99 €/mes (hosting + dominio prorrateado), lo que sitúa el umbral de sostenibilidad operativa en un nivel muy bajo.

6.2 PLAN DE EMPRESA

El presente apartado describe el modelo de negocio aplicable a la fase actual del proyecto, centrado en el nivel Freemium como base de crecimiento natural. El plan de empresa completo con visión a futuro, incluyendo los niveles de suscripción, arquitectura técnica de próxima generación y proyecciones de ingresos a 3 años, se adjunta como Anexo I del presente documento.

6.2.1 Propuesta de Valor

YourTree es una plataforma link-in-bio que permite centralizar enlaces, logros y proyectos en una única página personalizada, de forma totalmente gratuita. Su diferencial frente a competidores como Linktree o Beacons.ai es la integración nativa de un sistema de comunidad y foros, que facilita el descubrimiento orgánico de contenido entre usuarios sin necesidad de redes sociales externas.

6.2.2 Perfil de Usuario

El público objetivo abarca creadores de contenido, profesionales independientes (freelancers, diseñadores, fotógrafos) y pequeñas empresas que necesitan visibilidad

online sin los costes de suscripción que imponen las plataformas equivalentes. El mercado principal es hispanohablante, con un segmento prioritario de usuarios de entre 16 y 35 años, nativos digitales activos en redes sociales.

6.2.3 Modelo de Servicio y Funcionalidades

- El servicio se centra en el nivel Freemium, que ofrece de forma permanente y sin restricciones de pago:
- URL pública indexable con SEO básico de metadatos.
- Acceso total al ecosistema de foros y comunidad nativa de la plataforma.
- Control de visibilidad del perfil (público o privado).
- Visualización de anuncios Google Ads no invasivos como contraprestación al acceso gratuito.

6.2.4 Estrategia de Monetización

La rentabilidad del proyecto en su fase actual se apoya en dos pilares que no requieren pagos obligatorios por parte del usuario:

- **Publicidad (Google AdSense):** Los perfiles de usuarios gratuitos muestran anuncios integrados de forma no intrusiva. Con una base estimada de 300 usuarios en el primer año y un CPM medio de 1,5 € para el mercado hispanohablante, se proyecta una generación de entre 15 y 20 €/mes en ingresos pasivos, suficiente para cubrir los costes de infraestructura recurrentes.
- **Donaciones voluntarias ("Buy me a coffee"):** Sistema de apoyo directo vía PayPal ya implementado en el entorno demo (demo.treedlink.com), que permite medir la disposición de pago orgánica de los usuarios antes de activar cualquier modelo de suscripción.

6.2.5 Análisis de Costes y Sostenibilidad

El modelo está diseñado para ser viable desde etapas muy tempranas por tres razones:

- **Bajo riesgo financiero:** Los costes recurrentes de infraestructura (hosting + dominio) se mantienen por debajo de los 15 €/mes, lo que significa que los ingresos por publicidad cubren los gastos operativos con una audiencia mínima.
- **Eficiencia operativa:** El uso íntegro de herramientas open source y servidores de bajo coste (Hetzner) elimina licencias y reduce la dependencia de proveedores con modelos de precios variables.

- **Escalabilidad sin inversión previa:** El modelo Freemium permite hacer crecer la base de usuarios y validar el interés real del mercado antes de comprometer cualquier inversión en infraestructura adicional, siguiendo el principio de validate before you build.

7 OBJETIVOS

7.1 OBJETIVO GENERAL

El objetivo general de YourTree es ofrecer una plataforma que permita a los usuarios encontrar y compartir recursos de forma centralizada. Integrándolo con la funcionalidad de hacer preguntas mediante foros y permitir el descubrimiento de nuevos usuarios a través de los recursos que comparten desde cualquier parte del mundo.

7.2 OBJETIVOS ESPECÍFICOS

- Implementar foros globales donde los usuarios puedan plantear y resolver dudas sobre el tema de cada foro.
- Proporcionar acceso rápido y centralizado a recursos digitales, eliminando la necesidad de navegar entre múltiples plataformas externas.

8 DESARROLLO DEL PROYECTO

En este apartado se describe el proceso seguido para desarrollar la aplicación web.

8.1 TECNOLOGÍAS ESCOGIDAS

8.1.1 Frontend

1. React

Se ha seleccionado React por su eficiencia en el renderizado mediante el uso de un DOM Virtual, lo que permite gestionar interfaces dinámicas con alta frecuencia de actualización de forma óptima. Además, su arquitectura basada en componentes reutilizables facilita el mantenimiento del código y acelera el proceso de desarrollo.

2. Vite

Se emplea como herramienta de construcción (build tool) debido a su velocidad superior en el arranque del servidor de desarrollo y su optimización mediante ES Modules, mejorando significativamente la productividad frente a bundlers tradicionales como Webpack. (Vite, s.f.)

8.1.2 Backend

Node.js

Se ha seleccionado Node.js como entorno de ejecución en el servidor por su eficiencia y escalabilidad. Al estar basado en el motor V8 de Google Chrome, destaca por su arquitectura orientada a eventos y su modelo de I/O no bloqueante, lo que permite gestionar múltiples peticiones de forma asíncrona sin afectar al rendimiento. Además, su uso permite mantener JavaScript como lenguaje único tanto en el cliente como en el servidor, unificando la lógica de desarrollo.

Express.js

Para el desarrollo del servidor se seleccionó Express.js debido a su enfoque minimalista. De acuerdo con sus especificaciones técnicas (Express.js, s.f.), su sistema de middlewares permite interceptar y procesar peticiones HTTP de manera granular, abarcando desde la autenticación hasta la gestión de errores. Esta naturaleza *unopinionated* otorga libertad total para organizar la arquitectura de carpetas, facilitando la creación de APIs RESTful robustas.

8.1.3 Base de datos

Se ha seleccionado PostgreSQL como Sistema de Gestión de Bases de Datos por su robustez y su estricto cumplimiento de las propiedades ACID (atomicidad, consistencia, integridad y durabilidad) lo que garantiza la integridad de los datos en todas las transacciones.

A diferencia de otros sistemas relacionales, PostgreSQL ofrece un soporte avanzado para consultas complejas, indexación eficiente y tipos de datos extensibles (como el soporte nativo para JSONB). Ideal para proyectos que requieren una estructura de datos sólida, pero con la capacidad de escalar y adaptarse a futuros requisitos funcionales.

8.1.4 Despliegue e infraestructura

Para el despliegue de la aplicación, se ha optado por dos instancias CX23 del proveedor Hetzner, ubicada en centros de datos europeos (cumpliendo con la normativa RGPD). Dichas instancias cuentan con 2 vCPU y 4 GB de memoria RAM, considerándolos como recursos suficientes para garantizar un rendimiento fluido tanto del motor Node.js como del servidor PostgreSQL ejecutándose en su propio servidor. La elección de este modelo específico corresponde a una relación coste-rendimiento óptima, permitiendo un entorno de producción profesional con escalabilidad garantizada. (Hetzner, s.f.)

8.2 METODOLOGÍA DE TRABAJO

Para llevar a cabo la creación del proyecto se usó la metodología de Kanban, al ser un proyecto individual las otras metodologías no se adecuan a la forma de trabajo y se procederán a dar los puntos que se tomaron en cuenta para la elección.

8.2.1 METODOLOGÍAS COMPARADAS

SCRUM

- **Fuertes**
 - Permite trabajar en secciones de tiempo específicas (sprints) para cierto apartado del cual se quiera cumplir. (Atlassian, 2024)
 - Tener una planificación rigurosa y con seguimiento constante.
- **Contras**
 - No puedes agregar funcionalidades una vez empieza el sprint, puesto que al ya estar avanzado afecta a toda la carga de trabajo y no cumple con la definición de equipo multifuncional.
 - Mantener reuniones diarias (daily stand-ups, planning, retrospective) para ver el progreso que se hizo en el proyecto y como puede influir más adelante o a los demás miembros del equipo. (Indeed, 2022)

KANBAN

- **Fuertes**

- Permite visualizar el flujo de trabajo mediante, lo que mejora la organización y el control de tareas. (Atlassian, 2015)
- Favorece la entrega continua al no trabajar en iteraciones cerradas, lo que lo hace flexible ante cambios en cualquier momento del proyecto. (Atlassian, 2026)
- Limita el trabajo en curso (WIP limits), reduciendo la sobrecarga del equipo y mejorando la eficiencia del proceso. (Atlassian, 2015)
- **Contras**
 - No define roles ni estructuras estrictas, lo que puede generar falta de organización en equipos que necesitan mayor control o jerarquía. (Atlassian, 2026)
 - Al no trabajar con sprints, puede ser más difícil medir el progreso en intervalos fijos o planificar a largo plazo de forma estructurada. (Atlassian, 2015)

XP (Extreme Programming)

- **Fuertes**
 - Exige una gran disciplina entre desarrolladores para tener un buen producto de calidad e intercambio de conocimientos. (IEBSchool, 2021)
 - Tener un nivel de automatización de pruebas e integración continua (IEBSchool, 2021)
- **Contras**
 - Gran parte de su efectividad reside en reglas estrictas de programación por parejas, donde se deben alternar constantemente los roles entre quien escribe el código y quien lo revisa en tiempo real, lo cual puede ser difícil de sostener según la dinámica del equipo. (GoDaddy, 2025)

Waterfall (Cascada)

- **Fuertes**
 - El proceso de planificación inicial en el que los miembros del equipo reúnen toda la información posible para garantizar el éxito del proyecto manteniendo una estructura clara y definida para cada uno de sus apartados en concreto. (Asana, 2026)
 - Tiene mayor eficiencia en planificar y documentar el proyecto siempre que el alcance este cerrado.

- **Contras**

- Tiene una rigidez extrema, la cual no permite hacer cambios en la estructura del proyecto sin importar en cual etapa se encuentre.
(OBS Business School, 2024)

Y al tener una metodología clara, fue seccionada por cuatro partes:

- Por hacer
- En curso
- En revisión
- Finalizado

Con estos criterios definidos, se configuró el tablero de trabajo en Jira tal como se muestra a continuación.

Ilustración 17 Backend en Jira

Espacio

 Backend  ...

 Resumen  Cronograma  Tablero  Calendario  **Lista**  Formularios  Trámite

 Filtro 

☐	Tipo	Clave	Resumen
☐	☒	BAC-1	Inicializar el servidor (Express)
☐	☒	BAC-2	Middleware de autenticación mediante JWT
☐	☒	BAC-3	Middleware de cifrado de contraseñas mediante Bcrypt
☐	☒	BAC-4	Configuración de servidor de correos mediante Nodema...
☐	☒	BAC-5	Controladores y rutas de autenticación
☐	☒	BAC-6	Controladores y rutas de verificación mediante correo
☐	☒	BAC-7	Controladores y rutas de perfiles
☐	☒	BAC-8	Controladores y rutas de usuarios
☐	☒	BAC-9	Controladores y rutas de foros
☐	☒	BAC-10	Controladores y rutas de almacenamiento de imágenes ...
☐	☒	BAC-11	Crear los modelos de los objetos con respecto a la base ...
☐	☒	BAC-12	Crear los repositorios de los cuales se encargaran de ha...

Ilustración 18 Frontend en Jira

Espacio


Frontend



 Resumen
  **Lista**
 Tablero
  Código
 



Filtro ▾

☐	Tipo	Clave	Resumen	
☐	☒	KAN-8	Crear un repositorio en GitHub para llevar el control de v...	
☐	☒	KAN-1	Configuración base del proyecto (React + Vite)	
☐	☒	KAN-2	Sistema de enrutamiento (React router DOM)	
☐		KAN-3	Cabecera de navegación (Header)	
☐		KAN-4	Tema de la pagina web (Claro y Oscuro)	
☐		KAN-5	Buscador de usuarios y foros	
☐		KAN-6	Pantalla de inicio de sesión	
☐		KAN-7	Pantalla de registro	
☐		KAN-9	Pantalla de foros	
☐		KAN-10	Pantalla de páginas recientes	
☐		KAN-11	Pantalla de recuperar contraseña	
☐		KAN-12	Pantalla de cambiar contraseña	
☐		KAN-13	Pantalla de perfil	
☐		KAN-14	Pantalla de error (página no encontrada)	

Ilustración 19 Base de datos y Docker en Jira

Espacio

DB y Docker

...

Resumen

Cronograma

Tablero

Calendario

Lista

Formularios

T...

Q

Buscar en la lista

TN

Filtro

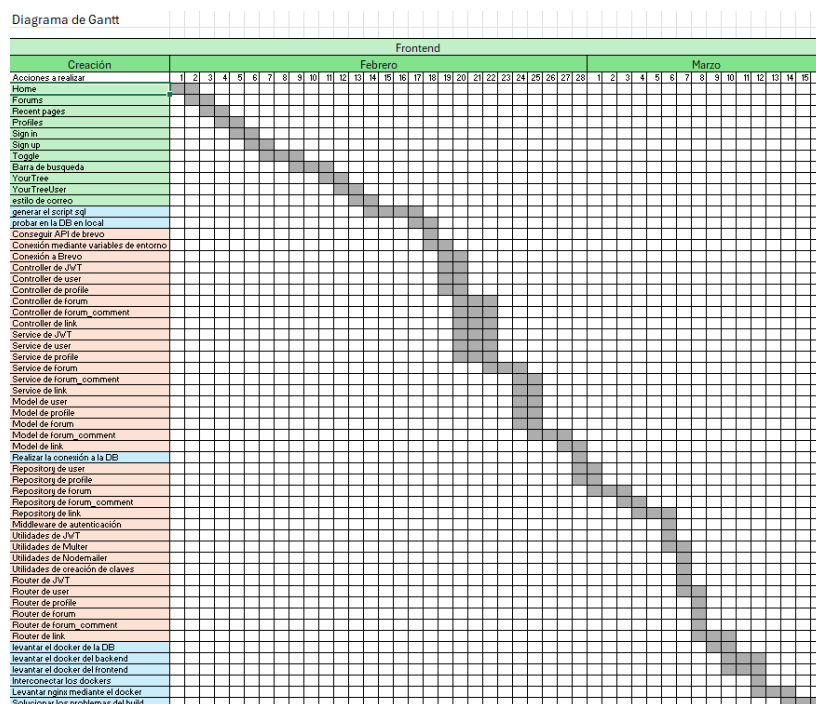
▼

☐	Tipo	Clave	Resumen
☐	☒	DYD-1	Construcción del diagrama ERE
☐	☒	DYD-2	Seguir las 3 formas de normalización para crear una bas...
☐	☒	DYD-4	Definir las restricciones que debe tener cada tabla
☐	☒	DYD-5	Crear un usuario que tenga los permisos necesarios par...
☐	☒	DYD-3	Construcción del modelo físico de la base de datos
☐	☒	DYD-6	Creacion de un docker compose para definir el comport...
☐	☒	DYD-7	Creación de distintos dockerfiles para que contienen los...

8.2.2 Diagrama de gantt

El diagrama de Gantt se utiliza para representar la planificación temporal del proyecto, mostrando cómo se fueron distribuyendo y desarrollando los distintos componentes a lo largo del tiempo.

Ilustración 20 diagrama de Gantt



8.2.3 Configuración del entorno

Para el despliegue de la aplicación se contrataron dos VPS independientes: una destinada exclusivamente a la base de datos y otra que aloja tanto el Frontend como el Backend mediante Docker.

- **Primera VPS**

Esta instancia aloja la capa de presentación y la capa de aplicación. La configuración se realizó ejecutando los siguientes comandos:

- **sudo su**

Acceso al modo superusuario para poder ejecutar los comandos de configuración sin restricciones de permisos.

- **apt update && apt upgrade**

Actualización del sistema operativo para asegurarse de que todos los paquetes están en su versión más reciente antes de instalar nada.

- **apt install docker.io docker-compose -y**

Instalación de Docker y Docker Compose en el servidor, necesarios para levantar los contenedores del Frontend y el Backend.

- **systemctl enable --now docker**

Activa el servicio de Docker y lo configura para que arranque automáticamente cada vez que el servidor se reinicie.

- **docker-compose -version**

Verificación de que Docker Compose se instaló correctamente y está disponible en el sistema.

- **ufw allow 80 && ufw allow 3000 && ufw allow 5173**

Apertura de los puertos necesarios en el firewall: el 80 para HTTP, el 3000 para el servidor Backend y el 5173 para el servidor de desarrollo del Frontend.

- **ufw enable**

Activación del firewall con las reglas configuradas.

- **ufw status**

Comprobación de que el firewall está activo y los puertos están correctamente abiertos.

- **apt install certbot -y**

Instalación de Certbot para gestionar el certificado SSL de forma gratuita.

- **certbot certonly --standalone -d demo.treedlink.com**

Generación del certificado SSL para el dominio de la aplicación, habilitando el protocolo HTTPS en el servidor.

- **Segunda VPS**

Esta instancia está dedicada exclusivamente a alojar la base de datos PostgreSQL, manteniéndola aislada del servidor de aplicaciones para mejorar la seguridad y el rendimiento. La configuración se realizó con los siguientes comandos:

- **apt update && apt upgrade**

Actualización del sistema operativo para asegurarse de que todos los paquetes están en su versión más reciente antes de instalar nada.

- **apt install docker.io docker-compose ufw -y**

Instalación de Docker, Docker Compose y el firewall UFW en un único comando.

- **systemctl enable docker --now**

Activa el servicio de Docker y lo configura para que arranque automáticamente con el servidor.

- **docker --version**

Verificación de que Docker se instaló correctamente y está disponible en el sistema.

- **ufw allow ssh**

Permite el acceso por SSH al servidor para poder administrarlo de forma remota sin quedarse bloqueado al activar el firewall.

- **ufw allow 5432/tcp**

Apertura del puerto 5432, que es el puerto por defecto de PostgreSQL, para que el servidor de aplicaciones pueda conectarse a la base de datos.

- **ufw enable**

Activación del firewall con las reglas definidas.

- **ufw status**

Comprobación de que el firewall está activo y los puertos están correctamente configurados.

- **apt install docker-compose-plugin -y**

Instalación del plugin oficial de Docker Compose como complemento al cliente de Docker, necesario para ejecutar los ficheros de orquestación en esta instancia

8.3 ESPECIFICACIÓN DE REQUISITOS

8.3.1 Análisis de requisitos

8.3.1.1 *Requisitos funcionales*

- El sistema debe permitir el registro de nuevos usuarios mediante formulario con nombre de usuario, correo electrónico y contraseña.
- El sistema debe permitir el inicio de sesión con correo o nombre de usuario y contraseña, devolviendo un token JWT.
- El usuario autenticado puede crear, editar, reordenar y eliminar enlaces en su página YourTree.
- El usuario puede personalizar su foto de perfil y su biografía.
- Cualquier visitante puede acceder a la página pública de un usuario mediante su URL (/yourtree/{username}).

- El sistema debe mostrar en la página 'Recent Pages' los perfiles de usuarios que hayan compartido al menos un enlace.
- Los usuarios registrados pueden crear nuevos foros con título y descripción.
- Los usuarios pueden ver todos los foros existentes en la sección de foros.
- Los usuarios autenticados pueden publicar respuestas (comentarios) dentro de un foro.
- El sistema debe enviar un correo de verificación al registrar una cuenta nueva.
- El sistema debe permitir recuperar la contraseña mediante correo electrónico.

8.3.1.2 Requisitos no funcionales

- La aplicación debe responder a las peticiones de la API.
- Las contraseñas deben almacenarse cifradas usando bcrypt con factor de coste mínimo 10.
- La comunicación entre cliente y servidor debe realizarse únicamente mediante HTTPS.
- La interfaz debe ser responsive y visualizarse correctamente en dispositivos móviles, tablets y escritorio.
- El sistema debe ser escalable horizontalmente, permitiendo añadir instancias de backend sin cambios en la arquitectura.
- El código debe estar estructurado siguiendo las convenciones del framework utilizado.
- La aplicación debe cumplir con las directrices básicas de accesibilidad WCAG 2.1 nivel AA.

8.3.2 Definición de funcionalidades

En este apartado se detallan los procesos operativos de la aplicación YourTree, describiendo la interacción entre la interfaz de usuario, la lógica de negocio en el servidor y la persistencia de datos.

8.3.2.1 Gestión de Identidad y Seguridad

Este apartado se centra en la protección de la plataforma, asegurando que solo los usuarios autorizados puedan gestionar sus enlaces y participar en la comunidad.

- Flujo de Registro y Acceso

El proceso comienza en el Frontend, donde el usuario interactúa con los componentes/páginas de acceso. Antes de enviar la información, se activa la lógica de UsePassword.jsx, que valida en el cliente que las contraseñas coincidan y cumplan con los requisitos mínimos de seguridad o que estén bien escritas por el usuario. Una vez validados, los datos se envían mediante una petición POST hacia las rutas del Backend: /yourtree/api/sign-up para el registro y /yourtree/api/sign-in para el acceso respectivo.

- Procesamiento en el Backend y Persistencia

Cuando la petición llega al servidor, el procesamiento se divide en dos flujos: inicio de sesión y registro.

En el registro de usuario: los datos pasan por un middleware de preparación de registro. Este se encarga de convertir los datos entrantes para que las características sean las adecuadas, verificando que la contraseña y su verificación sean correctas, que no falte ningún campo del formulario y convirtiendo la contraseña de texto plano en su hash respectivo. Para el registro, se almacenarán los datos y su código de verificación enviado a su correo electrónico.

En el inicio de sesión: se hará uso de Bcrypt para hacer la verificación de la contraseña introducida con la contraseña almacenada en la base de datos. Para el primer inicio de sesión, se le pedirá al usuario introducir el código que se envió al correo con el que se registró.

En ambos casos de autenticación: el sistema se encarga de comparar las credenciales con los registros en la tabla Users de la base de datos PostgreSQL para saber si el nombre de usuario o el correo electrónico ya existen. Según qué acción realice el usuario, tendrá un resultado distinto: si los datos son correctos, el backend responde enviando un par de tokens (Access y Refresh Token).

- Manejo de Sesión y Tokens

En el cliente, el componente token.jsx captura estos tokens y los almacena de forma segura en cookies. A partir de este momento, el componente auth.jsx

(AuthProvider) mantiene el estado global de la aplicación. Esto permite que el sistema sepa en todo momento si el usuario está logueado, habilitando rutas protegidas y cambiando los elementos del header.jsx (mostrando el perfil del usuario en lugar del botón de login).

- Seguridad Activa (Refresh Token)

Para mejorar la seguridad y la experiencia de usuario, se implementa una función de verificación constante. Si el Access Token expira, el sistema detecta automáticamente el fallo y utiliza el Refresh Token para solicitar uno nuevo a la ruta `/yourtree/api/refresh`, evitando que el usuario sea expulsado de la sesión mientras navega.

- Lógica de recuperación de contraseña

En el caso de la recuperación de acceso (`/yourtree/api/reset-password`), se utiliza el middleware `hash_password` para garantizar que la contraseña nunca se almacene en texto plano, aplicando algoritmos de cifrado asíncrono antes de impactar en la base de datos

- Verificación de Cuentas y Servicio de Mailing (Brevo)

A diferencia de los sistemas de autenticación básicos, se implementó un proceso de verificación de identidad para evitar registros fraudulentos.

Una vez ejecutado el controlador `sign_up_user`, el sistema genera un código de verificación único.

Se establece una conexión con la API de Brevo (anteriormente Sendinblue), que actúa como el servicio de mensajería transaccional encargado de enviar un correo electrónico al usuario con su código o enlace de activación.

El usuario debe consumir el endpoint `/yourtree/api/verify-code` para validar su cuenta. En caso de no recibirlo, se dispone de la ruta `yourtree/api/resent-code` que reinicia el proceso de envío.

- Recuperación de Acceso (Forgot/Reset Password)

El sistema cuenta con un flujo completo de recuperación gestionado por los controladores `/yourtree/api/forgot-password` y `/yourtree/api/reset-password`. El proceso utiliza nuevamente el servicio de mailing para enviar un token temporal. Al reiniciar la contraseña, el backend intercepta la petición, cifra la nueva clave y actualiza el registro en la tabla `Users`, invalidando los tokens anteriores para asegurar que la cuenta permanezca protegida.

8.3.2.2 Gestión de perfil público "Link in Bio" (YourTree)

Este apartado es el núcleo de la aplicación, permitiendo la creación, personalización y exposición de una tarjeta de presentación digital dinámica en donde aparecen los enlaces que el usuario quiere compartir.

- Panel de Configuración y Edición

El acceso a la modificación del perfil público se realiza a través de la ruta protegida gestionada por `YourTree.jsx`. En esta sección, el usuario dispone de herramientas específicas para alterar la estructura de su perfil público.

- Gestión de Enlaces (CRUD)

Mediante el componente `YourTree.jsx`, el usuario dispone de un control total sobre sus enlaces, permitiendo añadir, editar o eliminar entradas en tiempo real. Este proceso no se limita a la gestión de URLs, sino que incluye una capa de personalización estética optimizada:

- Identidad Visual

El formulario de creación permite asignar iconos de dos formas: mediante la extracción automática del `.ico` (favicon) de la web de destino para mantener la imagen corporativa de la red social, o mediante la inserción de una URL de imagen personalizada elegida por el usuario.

- Persistencia y Rendimiento

Cada acción interactúa con las rutas de la API, impactando directamente en la tabla de la base de datos vinculada a la

identidad del usuario. Se ha priorizado un sistema de formularios ligero frente a selectores complejos para evitar latencia (lag), garantizando que el diseño sea persistente, único y cargue de forma fluida tanto en el panel de edición como en la vista pública.

- **Página de Perfiles Generales**

La plataforma implementa un sistema de descubrimiento mediante el componente `Recent_Pages.jsx`, que actúa como un índice global de la comunidad. Esta funcionalidad realiza consultas indexadas a la base de datos para recuperar y renderizar una lista de perfiles recién unidos siempre y cuando tenga un enlace creado. El objetivo es fomentar el networking entre usuarios, permitiendo que cualquier visitante pueda explorar nuevas cuentas sin necesidad de conocer la URL exacta, dinamizando así el tráfico interno de la aplicación.

- **Visualización de Perfiles Específicos**

Cada usuario cuenta con una página pública única gestionada por el componente `YourTreeUser.jsx`. Al acceder a la ruta personalizada, el sistema ejecuta un proceso de carga dinámica.

- **Recuperación de datos**

El Backend identifica el perfil mediante el identificador único en la URL y extrae exclusivamente los enlaces y la configuración visual asociada en la base de datos.

- **Inyección de Metadatos (SEO)**

Se activa el componente `seo.jsx`, que personaliza dinámicamente las etiquetas del encabezado (título y descripción). Esto garantiza que, al compartir el perfil en redes sociales o ser indexado por buscadores, la tarjeta visual sea profesional y represente fielmente la identidad del usuario.

- Interacción y Redirección

Los perfiles no son solo vistas estáticas, sino centros de redirección inteligente. Cada botón de enlace está programado para abrirse en una nueva pestaña, asegurando que el usuario no abandone la plataforma principal. Además, esta interacción está vinculada al componente GA_4.jsx, lo que permite monitorizar el flujo de salida y los clics en tiempo real, cada clic queda registrado en GA4, lo que permite analizar qué enlaces tienen más actividad.

8.3.2.3 Sistema de Comunidad y Foro

- Creación y Gestión de foros

A través de una interfaz dedicada en el componente de Foros, los usuarios autenticados pueden generar nuevos hilos de discusión. El sistema captura el cuerpo del mensaje y el título, enviándolos al endpoint `/yourtree/api/forum/`. En el Backend, el controlador vincula automáticamente el `profileId` de la sesión activa con la nueva entrada en la tabla `Forums`, garantizando la integridad referencial y permitiendo que cada publicación muestre la autoría del creador en la base de datos.

- Sistema de respuestas

La interacción se gestiona mediante un esquema de comentarios anidados que son procesados por el componente `ForumDetailDialog`. Al abrirse, el sistema utiliza la etiqueta `<dialog>` y realiza una petición dinámica al endpoint `/yourtree/api/forum/:id/comments` para recuperar los mensajes de la tabla `forum_comments`. Esta tabla mantiene una relación de dependencia con el ID del post original, lo que permite que el Frontend renderice las conversaciones de forma jerárquica y con un flujo dinámico ("Optimistic UI"), actualizando la interfaz instantáneamente mientras se procesa la persistencia en el servidor.

- Gestión de Autoría y Borrado Seguro

El sistema incluye una lógica de protección que compara el `profile_id` del autor del foro con el ID del usuario logueado. Solo si existe coincidencia de identidad, se habilita el botón de borrado definitivo (DELETE), el cual requiere el envío del

Bearer Token en las cabeceras de la petición para validar la operación en el backend y eliminar el registro de la base de datos de forma segura.

- Motor de búsqueda y Optimización de Consultas

Para facilitar la navegabilidad, se implementa el componente SearchComponent.jsx, el cual actúa como un buscador global inteligente integrado en el encabezado de la aplicación.

- Búsqueda Asíncrona Dual

El sistema no solo escanea discusiones, sino que realiza peticiones paralelas a dos endpoints distintos del Backend:

/yourtree/api/forums/search para localizar hilos de debate y
/yourtree/api/users/search para encontrar perfiles de usuarios. Esto permite centralizar el descubrimiento de contenido y personas en una sola interfaz.

- Implementación de Debounce

Para optimizar el rendimiento del servidor y evitar el exceso de tráfico de red, se ha programado un retraso controlado (Debounce de 300ms). El sistema espera a que el usuario termine de escribir antes de disparar las peticiones fetch, mejorando drásticamente la fluidez de la interfaz.

- Interfaz Reactiva y Control de Cierre

El motor utiliza un desplegable dinámico (dropdown) que muestra los resultados categorizados por secciones. Además, incluye una lógica de detección de clics externos mediante el uso de useRef y eventos de escucha en el DOM, asegurando que el buscador se cierre automáticamente si el usuario interactúa con otra parte de la pantalla.

8.3.2.4 Monitorización y Optimización SEO

Este apartado transforma la aplicación en una plataforma más profesional mediante la gestión de metadatos y el análisis del comportamiento del usuario, asegurando que el contenido sea indexable y atractivo en redes sociales.

- **Configuración global**

En el componente principal AppWithHeader se define un esquema base de metadatos que incluye las etiquetas Open Graph y Twitter Cards. Gracias a esto, cualquier enlace de YourTree compartido en WhatsApp, Twitter o LinkedIn genera automáticamente una tarjeta visual con el logo y la descripción de la plataforma, sin necesidad de configurarlo página por página.

Para las vistas que necesitan metadatos propios, como Recent Pages o Forums, el componente SEO.jsx sobrescribe esos valores globales con un título y una descripción específicos. Así cada sección puede ser indexada de forma independiente por los motores de búsqueda, lo que mejora el posicionamiento orgánico de cada página por separado.

- **SEO dinámico por perfil**

En la vista de perfiles individuales (YourTreeUser.jsx) el SEO se personaliza por completo para cada usuario. Antes de inyectar ningún metadato, el sistema ejecuta una función asíncrona (existsUser) que comprueba si el perfil existe en la base de datos. Si no existe, se redirige a una página 404 de forma limpia, sin dejar entradas erróneas en el historial del navegador ni problemas de indexación.

Una vez confirmado que el perfil existe, el componente SEO.jsx inyecta dinámicamente el nombre del usuario y su biografía en las etiquetas title y meta description. El resultado es que cada perfil tiene sus propios metadatos únicos, lo que permite que las marcas personales de los usuarios sean localizables en buscadores de forma independiente.

- **Rastreo de actividad (AnalyticsTracker)**

Junto a la capa de SEO funciona el componente AnalyticsTracker, integrado con Google Analytics 4. Su trabajo es detectar los cambios de ruta

(location.pathname) y registrar de forma anónima el flujo de navegación, sin intervenir en la interfaz. Esto permite identificar qué secciones retienen más tiempo al usuario y qué perfiles generan más tráfico, información útil para decidir qué partes del proyecto necesitan más atención a medida que crece.

8.4 DISEÑO DE LA APLICACIÓN

8.4.1 Diseño conceptual

Con las ideas iniciales definidas, se realizó un primer maquetado en papel para visualizar cómo podría estructurarse la aplicación.

Ilustración 21 diseño conceptual papel 1

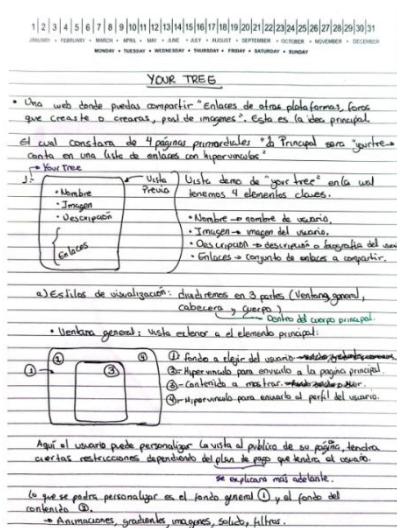


Ilustración 22 diseño conceptual papel 2

1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31

JANUARY • FEBRUARY • MARCH • APRIL • MAY • JUNE • JULY • AUGUST • SEPTEMBER • OCTOBER • NOVEMBER • DECEMBER
MONDAY • TUESDAY • WEDNESDAY • THURSDAY • FRIDAY • SATURDAY • SUNDAY

"Cabecera y cuerpo es lo que está en el interior de (3) de vista general."

- Cabecera: Podemos definir la vista en donde aparece la imagen, nombre y descripción.

a)

Nombre
Imagen
Descripción

b)

Nombre
Imagen
Descripción

Se separan por diagramas la cabecera y cuerpo

c)

Imagen
Nombre
Descripción

d)

Nombre
Imagen
Descripción

Tendrán distintos de presentar la cabecera, como se puede ver en la parte superior. Cada apartado dentro de la cabecera puede tener un estilo propio o uno conjunto.

"Quiero que más adelante se pueda poner marca en la imagen, por ejemplo."

Tendrán:

- Color de texto, Tipo de letra (Principal), otras personalización que serán secundarias.

Ilustración 23 diseño conceptual papel 3

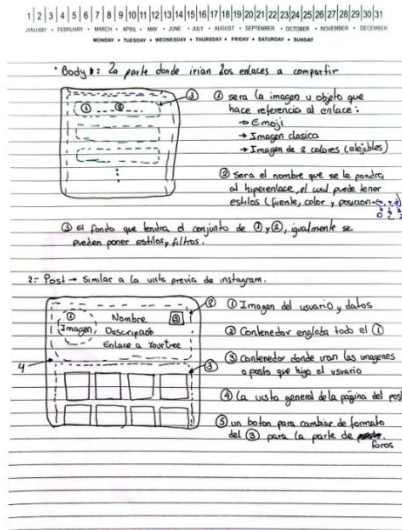


Ilustración 24 diseño conceptual papel 4

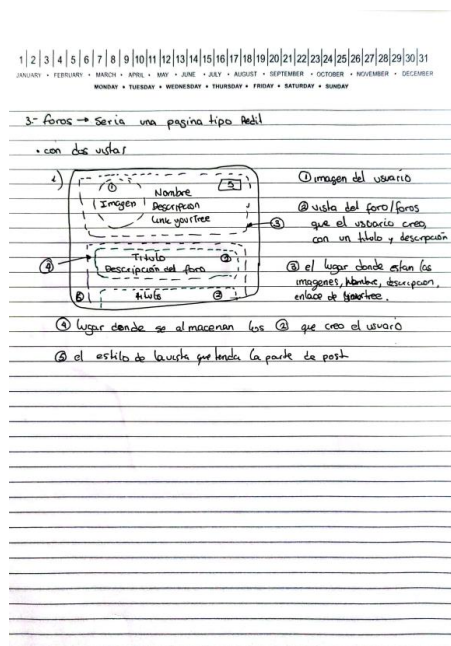
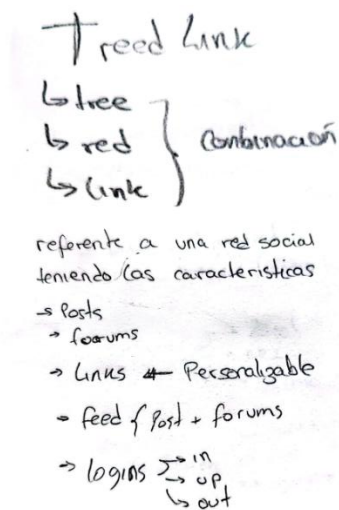


Ilustración 25 diseño conceptual papel 5



Una vez definidas las bases del diseño, se valoró la conveniencia de desarrollar una versión de demostración antes de abordar una aplicación completa, dado que no era posible prever con certeza su rendimiento ni la retención de usuarios. Por ello, se optó por implementar una demo centrada en los dos ejes clave: el árbol de enlaces y los foros comunitarios.

8.4.2 Modelado de la base de datos

Al tener la idea clara empezamos a visualizar que elementos puede tener la base de datos:

- **Usuario**

Datos relacionados al registro como la contraseña, nombre de usuario, correo electrónico, entre otras.

- **Roles**

Permite identificar qué privilegios tiene cada usuario dentro de la aplicación. Aunque en la versión actual la distinción es básica, esta tabla está pensada para escalar y permitir en el futuro la gestión de usuarios con permisos diferenciados como moderadores o administradores.

- **Perfil de usuario**

Separado de la tabla de usuario, recoge los datos más personales y configurables: biografía, correo de recuperación, tema visual seleccionado, fecha de creación de la cuenta, fecha de nacimiento, nombre visible, entre otros. Esta separación permite mantener la lógica de autenticación independiente de la información del perfil público.

- **Enlaces**

Contiene toda la información necesaria para cada enlace que el usuario decide compartir: título, URL de destino, imagen asociada y posición dentro de su página. El campo de posición es el que permite al usuario reordenar sus enlaces desde el panel de edición.

- **Foros**

Registra la información de cada hilo creado: el perfil del creador, el título, la descripción, la fecha de publicación y contadores de likes y compartidos. Estos últimos están incluidos de cara a funcionalidades que se podrían incorporar en versiones futuras.

- **Comentarios del foro**

Vincula cada comentario con el foro al que pertenece y con el usuario que lo creó, almacenando también su contenido y su fecha de creación. Esta relación es la que permite mostrar los comentarios en orden descendente, con el más reciente primero.

Todas las tablas incluyen campos adicionales reservados para implementaciones futuras, dejando margen para ampliar la funcionalidad de la plataforma sin necesidad de reestructurar el modelo de datos desde cero.

8.4.2.1 Diagrama Entidad Relación (ER)

Con las entidades ya definidas se procedió a construir el diagrama ER, estableciendo la cardinalidad entre tablas y describiendo mediante un verbo la acción que relaciona cada par de entidades. Las relaciones resultantes fueron las siguientes:

- **Roles $\leftarrow$ ser $\rightarrow$ Users**

Un rol puede estar asignado a varios usuarios, pero cada usuario tiene un único rol definido.

- **Users $\leftarrow$ identificar $\rightarrow$ Profiles**

Cada usuario tiene asociada una única identidad en la tabla de perfiles, estableciendo una relación de uno a uno entre ambas entidades.

- **Profiles $\leftarrow$ crear $\rightarrow$ Forums**

Un perfil puede crear uno o varios foros, pero cada foro pertenece a un único perfil creador.

- **Forums $\leftarrow$ contiene $\rightarrow$ Forums_comments**

Un foro puede contener uno o muchos comentarios, y cada comentario pertenece a un único foro.

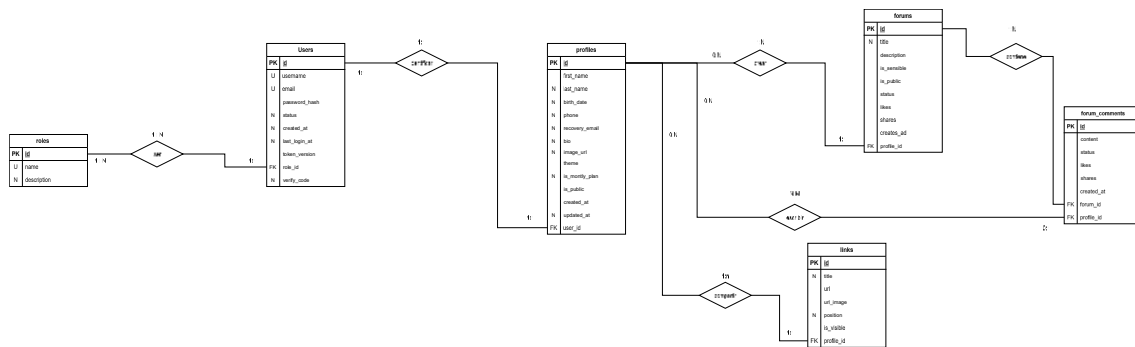
- **Profiles $\leftarrow$ escribir $\rightarrow$ Forums_comments**

Un perfil puede escribir uno o muchos comentarios a lo largo de distintos foros, y cada comentario tiene un único perfil como autor.

- **Profiles $\leftarrow$ compartir $\rightarrow$ Links**

Un perfil puede compartir uno o muchos enlaces en su página pública, y cada enlace pertenece a un único perfil.

Ilustración 26 Diagrama entidad relación (ER)



8.4.2.2 Diagrama lógico

Con el diagrama ER ya definido, el siguiente paso fue desarrollar el diagrama lógico aplicando las tres formas normales para garantizar que la estructura de la base de datos sea consistente y no contenga redundancias.

- FN1 (primera forma normal)**

Se revisó cada entidad para asegurarse de que todos sus atributos contienen valores atómicos, es decir, que no existan datos repetidos ni campos que puedan derivarse a partir de otros. Cada celda de cada tabla almacena un único valor indivisible.

- FN2 (segunda forma normal)**

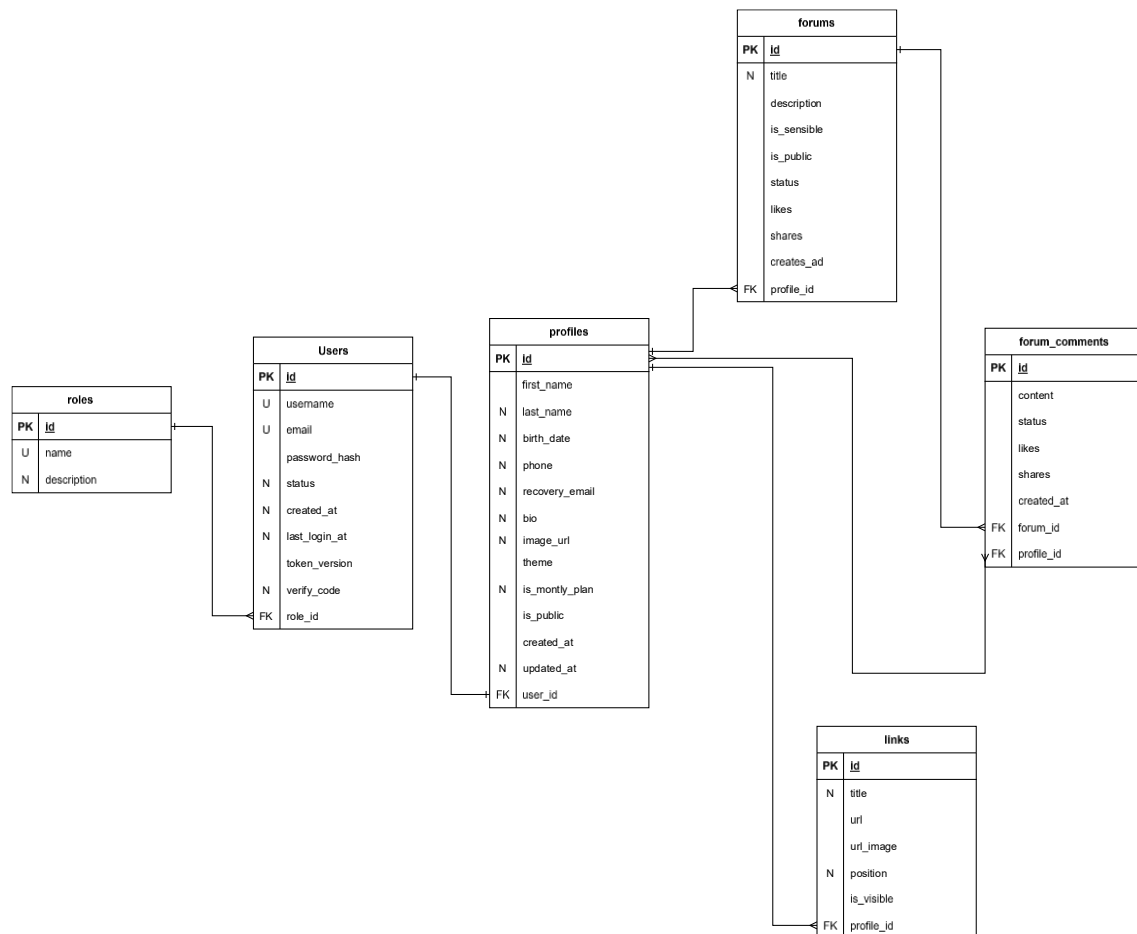
Una vez cumplida la primera forma normal, se verificó que todos los atributos que no forman parte de la clave primaria dependan de ella de forma completa y no parcial. Esto implica que ningún campo puede depender solo de una parte de una clave compuesta, sino de la clave en su totalidad.

- FN3 (tercera forma normal)**

Por último, se eliminaron las dependencias transitivas. Esto significa que ningún atributo puede depender de otro atributo que no sea la clave primaria. Si se detectaba que un campo dependía de otro campo no clave, ese dato se trasladaba a una tabla propia con su correspondiente relación.

Aplicando estas tres formas normales se consigue un modelo de datos limpio, sin redundancias y preparado para escalar sin comprometer la integridad de la información

Ilustración 27 diagrama lógico



8.4.2.3 Diagrama físico

Con el diagrama lógico ya definido, el siguiente paso fue trasladarlo a código SQL para que PostgreSQL pudiera ejecutarlo y crear la estructura real de la base de datos. Además de las tablas y sus relaciones, se implementaron índices sobre las columnas más consultadas para optimizar el rendimiento de las búsquedas más frecuentes.

8.4.2.3.1 Índices realizados

- **idx_profiles_user_id**

Optimiza la búsqueda de perfiles a partir del id del usuario con el que se registró en la plataforma. Es uno de los índices más consultados ya que esta relación se resuelve en prácticamente cada petición autenticada.

- **idx_links_profile_id**

Acelera la recuperación de todos los enlaces que pertenecen a un perfil concreto. Se usa cada vez que se carga la página pública de un usuario o su panel de edición.

- **idx_forums_profile_id**

Optimiza la búsqueda de los foros creados por un perfil específico a partir de su id, agilizando las consultas relacionadas con la gestión y visualización de foros propios.

- **idx_forum_comments_forum_id**

Acelera la carga de todos los comentarios asociados a un foro concreto, mejorando el rendimiento al abrir el detalle de cualquier hilo de discusión.

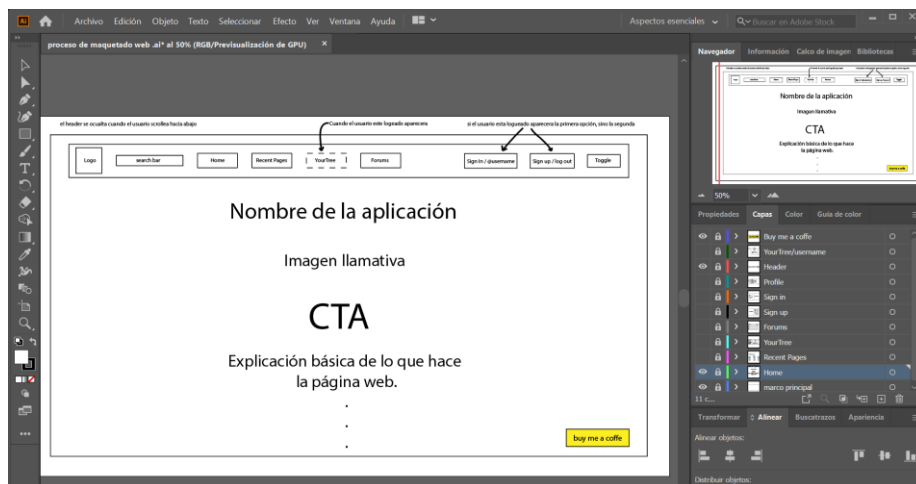
- **idx_forum_comments_profile_id**

Optimiza la búsqueda de todos los comentarios escritos por un perfil en particular, útil para consultas relacionadas con la actividad de un usuario dentro de los foros.

8.4.3 Prototipado digital

Con la base de datos definida y el concepto del proyecto claro, el siguiente paso fue trasladar las ideas a un prototipo visual mediante Adobe Illustrator.

Ilustración 28 vista general del entorno de Illustrator



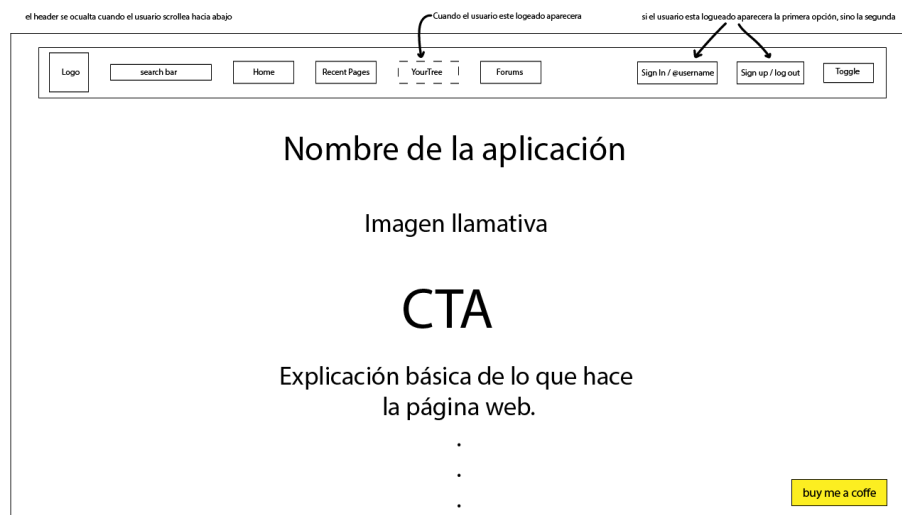
Como se puede observar en la previsualización del panel de capas, el prototipo está compuesto por siete páginas que corresponden a las rutas principales de la aplicación web, además de dos componentes visuales reutilizables: el header y un botón "Buy me a coffee". Ambos componentes acompañan a todas las páginas, con la excepción de YourTreeUser, donde únicamente aparece el botón "Buy me a coffee" ya que esa vista está pensada para ser lo más limpia y centrada posible en el perfil del usuario.

A continuación, se muestra el prototipo de cada página.

8.4.3.1 Home

La página de inicio es la primera impresión que recibe cualquier visitante de la plataforma. Arranca con una imagen y un texto pensados para captar la atención del usuario desde el primer momento, seguidos de una descripción clara de qué es YourTree y qué puede hacer con ella.

Ilustración 29 prototipo de la web (Home)

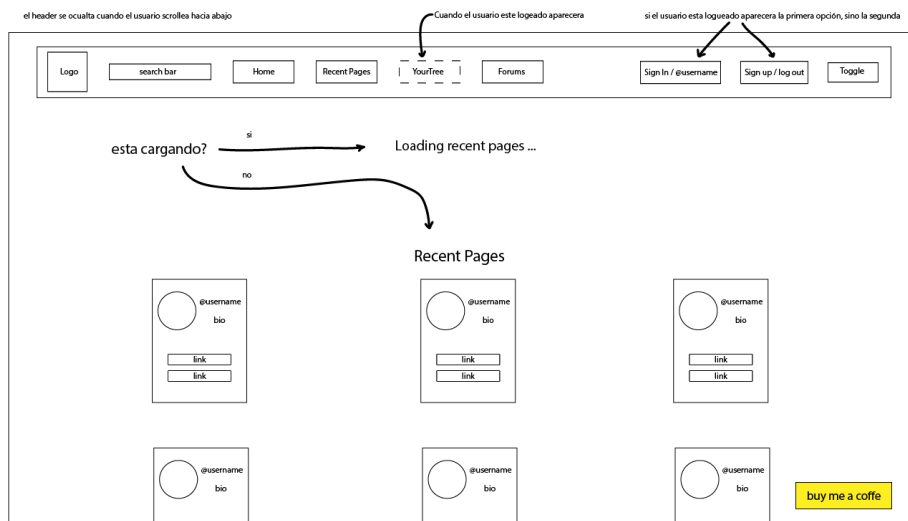


8.4.3.2 Recent pages

Recent Pages es la sección de descubrimiento de la plataforma. En ella se muestran tarjetas de previsualización de los perfiles de usuarios que han compartido al menos un enlace, mostrando su nombre de usuario y biografía para que cualquier visitante pueda hacerse una idea de quién es y qué tipo de contenido comparte antes de entrar a su página.

El objetivo de esta página es fomentar que los usuarios exploren perfiles nuevos de forma orgánica, sin necesidad de conocer de antemano la URL exacta de la persona que buscan.

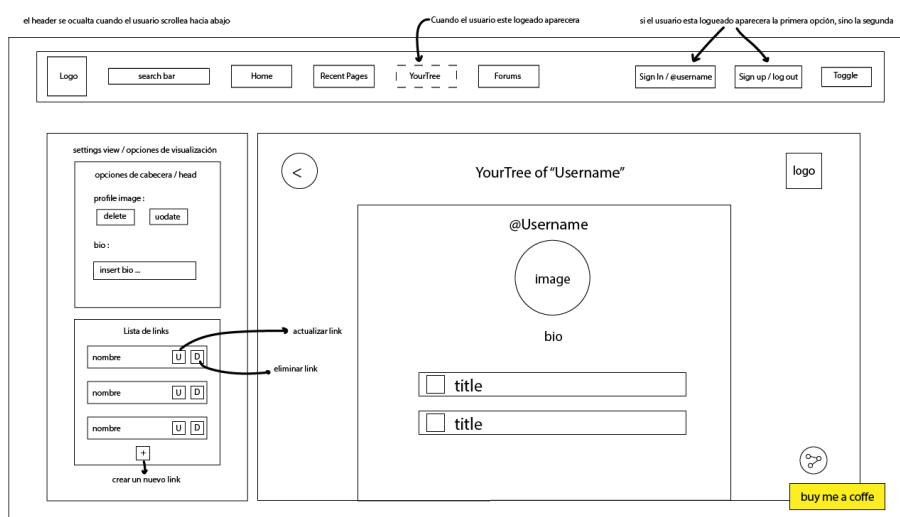
Ilustración 30 prototipo de la web (Recent pages)



8.4.3.3 YourTree

YourTree es la sección principal de la plataforma y el núcleo de la aplicación. Es el espacio donde el usuario construye y gestiona su página pública, añadiendo, editando, reordenando y eliminando sus enlaces mediante operaciones CRUD. Cada cambio que el usuario realiza desde este panel se refleja directamente en su página pública y en la vista que tiene en su lateral, de tal manera que cualquier visitante puede consultar a través de su URL personalizada.

Ilustración 31 prototipo de la web (YourTree)

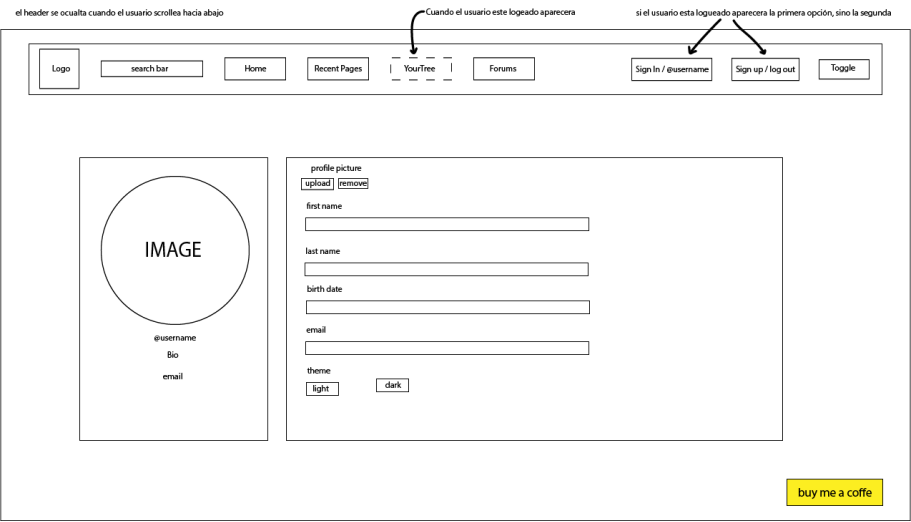


8.4.3.4 Profile

La página de perfil es accesible desde cualquier parte de la aplicación haciendo clic en el nombre de usuario. Desde aquí el usuario puede modificar su información personal: foto de perfil, nombre, apellidos y fecha de nacimiento. Además permite cambiar el tema

visual de la interfaz según sus preferencias y actualizar el correo de verificación, que es un dato privado visible únicamente para el propio usuario y no aparece en ninguna parte de su página pública.

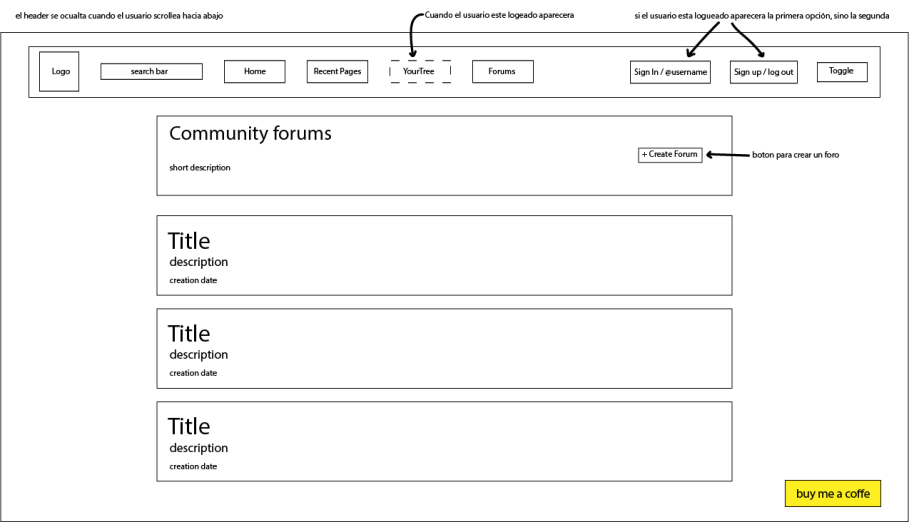
Ilustración 32 prototipo de la web (Profile)



8.4.3.5 Forums

La página de foros muestra un listado global de todos los hilos creados por los usuarios. Cada foro aparece en forma de tarjeta con su título y descripción a modo de vista previa. Al hacer clic en cualquiera de ellos, el hilo se expande ocupando toda la pantalla y muestra sus comentarios correspondientes en orden descendente. Desde esa vista ampliada el usuario también puede interactuar con el foro mediante los botones de likes y compartidos.

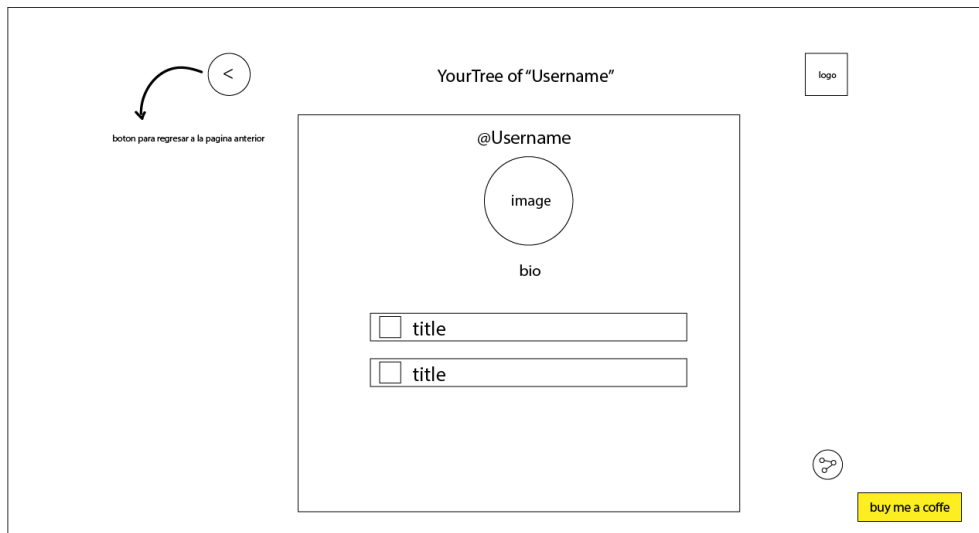
Ilustración 33 prototipo de la web (Forums)



8.4.3.8 YourTreeUser

Es la vista pública del perfil de cada usuario, generada a partir del contenido que ha configurado en su sección YourTree. Se accede a ella a través de la URL personalizada del usuario, que puede copiar desde su panel y compartir directamente desde el portapapeles.

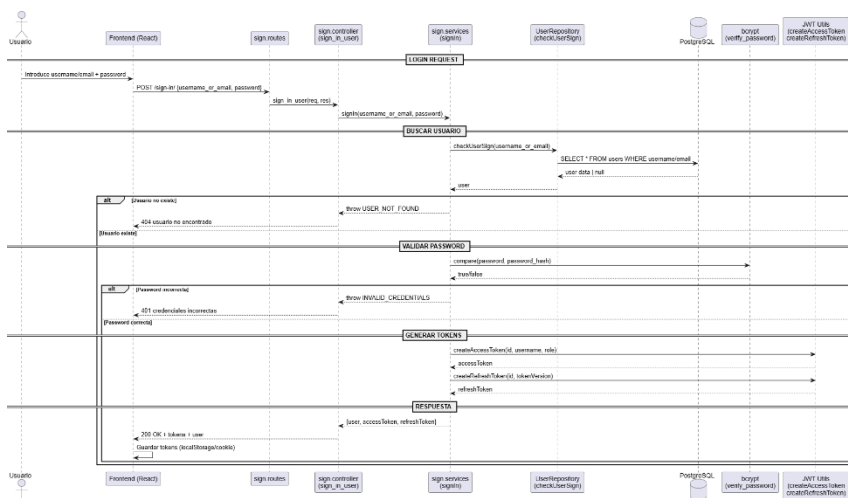
Ilustración 36 prototipo de la web (YourTreeUser)



8.4.4 Diagrama de secuencia

Representa el flujo completo de autenticación del sistema, incluyendo la interacción entre frontend, backend y base de datos, así como la generación de tokens JWT para la gestión de sesiones sin estado.

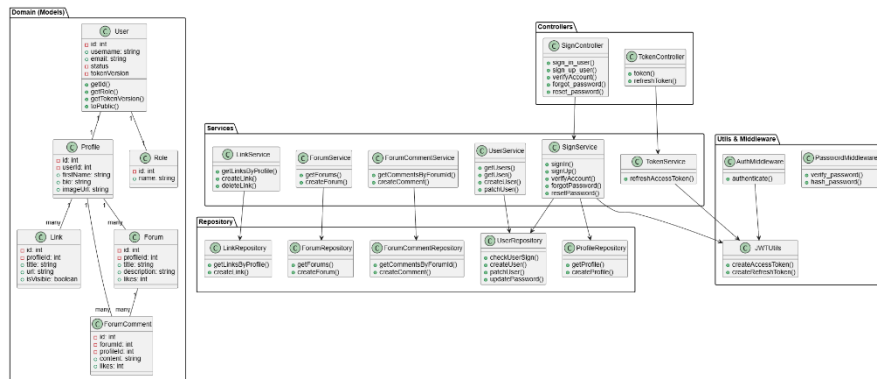
Ilustración 37 diagrama de secuencia



8.4.5 Diagrama de clases

Modela la estructura del backend siguiendo una arquitectura por capas (Controller–Service–Repository), mostrando tanto las entidades principales del dominio como sus relaciones y responsabilidades.

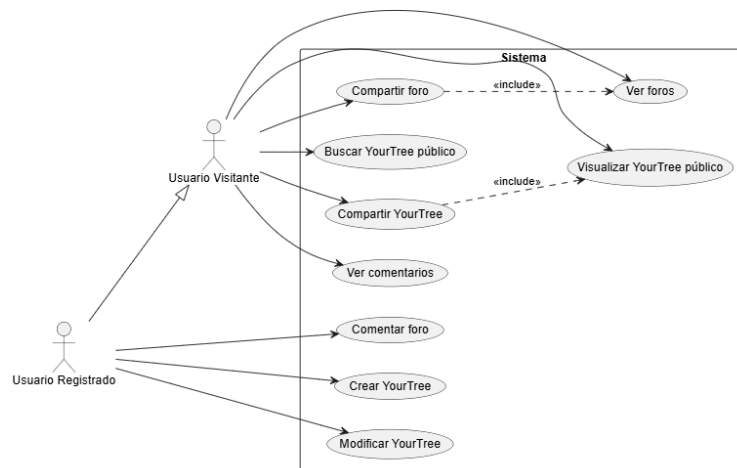
Ilustración 38 diagrama de clases



8.4.6 Diagrama de casos de uso

El diagrama de casos de uso muestra las acciones que un usuario puede realizar dentro de la aplicación web, diferenciando entre usuario visitante y usuario registrado.

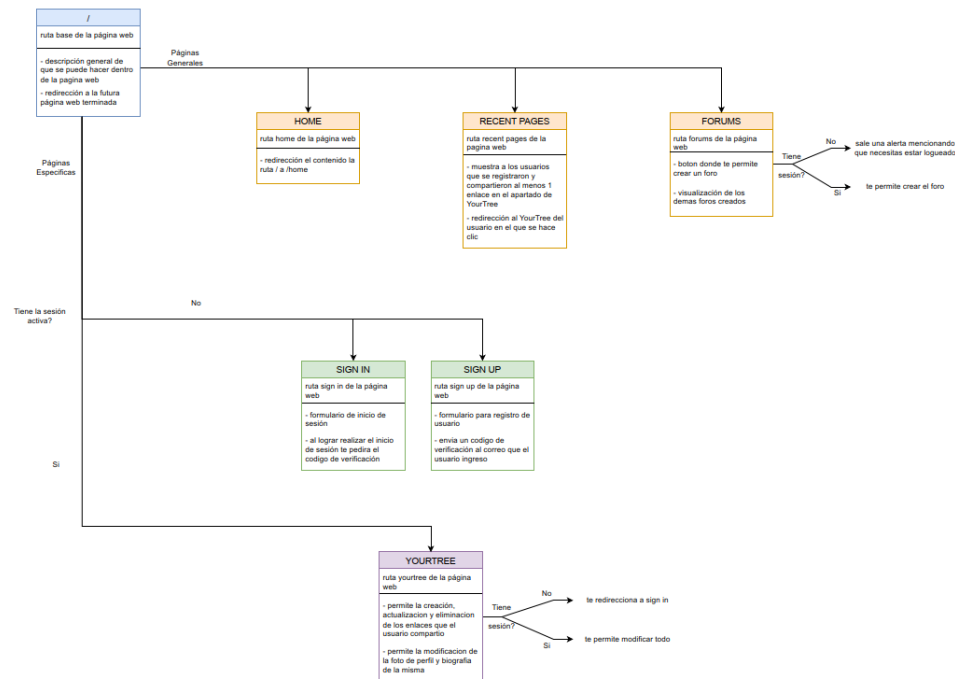
Ilustración 39 diagrama de uso



8.4.7 Mapa de navegación

El mapa de navegación representa la estructura de la aplicación y las relaciones entre las distintas páginas, permitiendo visualizar de forma clara el recorrido que puede seguir el usuario dentro del sitio.

Ilustración 40 mapa de navegación



8.5 IMPLEMENTACIÓN Y ARQUITECTURA

8.5.1 Arquitecturas utilizadas

8.5.1.1 Backend

Al inicio del desarrollo se optó por la arquitectura Modelo-Vista-Controlador (MVC) por ser la más conocida y directa de implementar. Sin embargo, a medida que el proyecto fue creciendo y añadiendo funcionalidades, la separación que ofrece MVC empezó a quedarse corta: distintas responsabilidades acababan mezcladas en los mismos ficheros, derivando en lo que se conoce como código espagueti, donde el mantenimiento y la incorporación de mejoras se vuelve cada vez más complicado.

Por ese motivo se tomó la decisión de migrar a una arquitectura más robusta, combinando la arquitectura por capas con el patrón Control-Service-Repository (CSR), consiguiendo una separación de responsabilidades mucho más clara y escalable.

8.5.1.1.1 Arquitectura Control-Service-Repository

El objetivo de esta arquitectura es separar las responsabilidades según el tipo de acción que realiza cada parte de la API, dividiendo el código en tres capas bien diferenciadas. (HACK A BOSS, n.d.)

- **Controller**

Capa encargada de gestionar la entrada y salida de datos a través de HTTP. Recibe las peticiones, valida los datos de entrada y devuelve la respuesta al cliente.

- **Service**

Capa donde reside la lógica de negocio de la aplicación. Es el intermediario entre el controlador y el repositorio, encargándose de procesar y coordinar las operaciones necesarias.

- **Repository**

Capa dedicada exclusivamente a la ejecución de consultas SQL directas contra la base de datos, sin mezclar ningún otro tipo de lógica.

8.5.1.1.2 Arquitectura por capas

Además de las tres capas principales de CSR, la arquitectura del backend se complementa con los siguientes directorios que dan soporte al funcionamiento global de la API. (ctrl-y, 2021)

- **Config**

Centraliza las variables y parámetros clave para el funcionamiento de la API, como las credenciales de conexión a la base de datos o los secretos para la firma de tokens. Tenerlos en un único lugar permite modificar el comportamiento del servidor sin tener que buscar valores dispersos por el código.

- **DB**

Contiene la lógica de conexión a PostgreSQL mediante la dependencia pg, incluyendo la configuración del pool de conexiones y los scripts SQL de inicialización de la base de datos.

- **Middlewares**

Agrupar las funciones intermedias que actúan como puente entre la petición entrante y el controlador final. Se utilizan para validar acciones transversales como la autenticación del usuario o la conversión y preparación de datos antes de que lleguen a la lógica de negocio.

- **Models**

Contiene las clases que representan las entidades de la base de datos. Convertir los datos crudos en objetos con una estructura definida garantiza que la información que circula por la API sea siempre coherente y tenga el formato esperado.

- **Routes**

Define los endpoints de la API y los asocia a su método HTTP correspondiente (GET, POST, PUT, DELETE). Actúa como el mapa de acceso al servidor, conectando cada URL con su controlador.

- **Utils**

Reúne funciones de utilidad reutilizables a lo largo de toda la API, envío de correos, generadores de códigos o formateadores, evitando duplicar código en distintos puntos del proyecto.

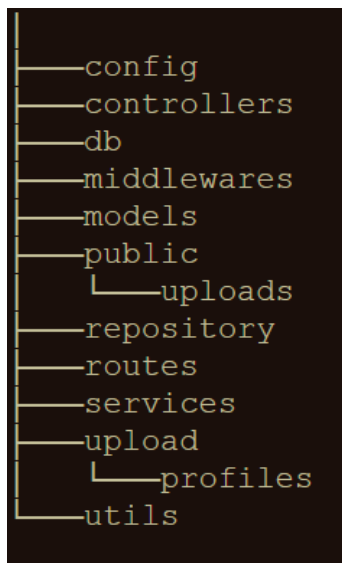
- **Public**

Directorio de acceso público directo. Actualmente no contiene ningún elemento, pero está reservado para recursos que en el futuro necesiten ser servidos de forma estática.

- **Uploads**

Gestiona los archivos subidos por los usuarios. Actualmente contiene un subdirectorio profiles donde se almacenan los avatares y fotos de perfil de cada usuario.

Ilustración 41 arquitectura del backend



8.5.1.2 Frontend

A diferencia del backend, el frontend se organizó desde el principio siguiendo una estructura basada en componentes, ya que este enfoque permite reutilizar código de forma más limpia y mantener una arquitectura más escalable dentro de una aplicación SPA. Además, facilita separar la lógica visual por bloques funcionales sin llegar a mezclar responsabilidades en un único fichero. (Angel, 2024)

8.5.1.2.1 Public

Directorio donde se almacenan los ficheros estáticos que no pasan por el proceso de compilación, pero que son importantes para el funcionamiento del sitio y para servicios externos como Google. Aquí se incluyen recursos como iconos, sitemap, robots, manifest y archivos relacionados con la indexación o la publicidad.

8.5.1.2.2 Src

Directorio principal que contiene los recursos utilizados para la creación de la página web. En él se agrupan tanto los componentes de React como los estilos, páginas, utilidades y demás elementos que forman parte de la lógica visual de la aplicación.

8.5.1.2.2.1 Assets

Directorio que contiene elementos estáticos como iconos, imágenes, hojas de estilo y pequeños scripts auxiliares. Su función es aportar tanto el diseño visual como ciertas funcionalidades reutilizables en la interfaz.

8.5.1.2.2.2 Components

Directorio que contiene los ficheros con funciones reutilizables que renderizan etiquetas HTML a través del virtual DOM de React. En este apartado se concentran los bloques visuales y lógicos que se repiten en distintas partes de la aplicación.

8.5.1.2.2.3 Pages

Directorio que contiene las páginas renderizadas por las rutas definidas con React Router. Cada página reutiliza componentes comunes para construir vistas concretas y mantener una estructura coherente en toda la aplicación.

8.5.1.2.2.4 Utils

Directorio que contiene utilidades en JavaScript puro que se reutilizan en distintas partes del frontend. Actualmente solo incluye el fichero relacionado con el recorte de imágenes, usado en el proceso de subida y edición de la foto de perfil.

Ilustración 42 arquitectura del frontend



8.5.2 Patrones utilizados

Mientras que la arquitectura define la estructura global del proyecto, los patrones de diseño son soluciones probadas para resolver problemas concretos a nivel de código y flujo de ejecución. En el proyecto se aplicaron los siguientes.

8.5.2.1 Patrones en el Backend

8.5.2.1.1 Patrón Middleware

Define el flujo por el que pasan las peticiones en Express antes de llegar al controlador. Cada solicitud HTTP atraviesa de forma secuencial una serie de funciones intermedias que pueden validar, transformar o rechazar la petición. En este proyecto se usa para tareas como verificar que el usuario está autenticado o preparar los datos de entrada antes de que lleguen a la lógica de negocio. (Expressjs, n.d.)

8.5.2.1.2 Patrón de repositorio

Aunque forma parte de la arquitectura por capas, a nivel de código funciona como un patrón estructural. Su objetivo es encapsular toda la lógica de acceso a datos en ficheros dedicados como `user.repository.js` o `forum.repository.js`, de forma que los servicios no tengan contacto directo con la base de datos. Esto facilita posibles migraciones a otro motor de base de datos en el futuro sin necesidad de tocar la lógica de negocio. (Fowler, 2002)

8.5.2.1.3 Patrón Singleton

Garantiza que ciertas configuraciones tengan una única instancia activa durante todo el ciclo de vida de la aplicación. En este proyecto se aplica en la conexión a PostgreSQL a través de `connection_db.model.js`, donde en lugar de abrir una conexión nueva por cada petición se reutiliza un pool compartido, evitando la saturación del servidor y mejorando el rendimiento general de la API. (Wikipedia, 2026)

8.5.2.2 Patrones en el Frontend

8.5.2.2.1 Patrón de Composición de Componentes

Es el patrón base sobre el que se construye cualquier aplicación en React. Consiste en dividir la interfaz en piezas pequeñas, independientes y reutilizables que se ensamblan para formar vistas más complejas. En este proyecto elementos como el `header.jsx`, los botones sociales o el `loader.jsx` se diseñan una sola vez y se importan en las distintas páginas, respetando el principio DRY (Don't Repeat Yourself) y evitando duplicar código a lo largo del proyecto. (Meta, n.d.)

8.5.2.2.2 Patrón Proveedor

Permite compartir estado global a lo largo de todo el árbol de componentes sin tener que pasar propiedades manualmente de nivel en nivel, lo que se conoce como prop drilling. En este proyecto se usa para dos casos concretos: el estado de la sesión del usuario gestionado desde `auth.jsx`, que cualquier componente de la aplicación puede consultar sin necesidad de recibirlo como prop, y las preferencias visuales del tema claro u oscuro gestionadas desde `ThemeToggle.jsx`. (Meta, n.d.)

8.5.2.2.3 Patrón de Enrutamiento del lado del Cliente

Implementado mediante React Router, este patrón intercepta los cambios en la URL del navegador y carga el componente correspondiente `Home.jsx`, `Profile.jsx`, `Forums.jsx`, etc. sin recargar la página completa. Es lo que permite que la aplicación funcione como

una Single Page Application (SPA) real, donde las transiciones entre páginas son instantáneas y la experiencia de navegación es fluida.

8.5.3 Desarrollo del backend

El primer paso antes de arrancar el desarrollo fue instalar de forma global Node.js y pnpm. Node.js actúa como el entorno de ejecución que permite correr archivos JavaScript fuera del navegador, mientras que pnpm fue el gestor de paquetes elegido en lugar de npm. Su sistema de almacenamiento enlazado evita duplicidades entre paquetes y reduce el espacio en disco, ofreciendo una instalación de dependencias más eficiente y segura.

8.5.3.1 Elección del backend

A lo largo del año académico se fueron presentando distintos lenguajes y tecnologías como Java, JavaScript, PHP y Python. Habiendo trabajado con todos ellos tanto en clase como por cuenta propia, la decisión final fue usar Express.js como framework del backend. La razón principal es su naturaleza minimalista: no impone una estructura rígida ni incluye funcionalidades que no se vayan a usar, lo que permite construir la arquitectura a medida e ir añadiendo dependencias según las necesidades reales del proyecto.

8.5.3.1.1 Comandos para iniciar

- **pnpm init**

Inicializa el proyecto y genera el fichero package.json, donde quedarán registradas todas las dependencias y scripts del servidor.

- **pnpm install express**

Instala Express.js en el proyecto a través de pnpm, añadiéndolo automáticamente como dependencia en el package.json. (Expressjs, n.d.)

- **pnpm install -D nodemon**

Instala nodemon como dependencia de desarrollo. Su función es monitorizar activamente los ficheros del servidor y reiniciarlo automáticamente cada vez que se detecta un cambio, evitando tener que parar y arrancar el proceso manualmente durante el desarrollo. (Remy, n.d.)

- **pnpm install bcrypt**

Instala bcrypt, una librería utilizada para cifrar contraseñas de forma segura antes de almacenarlas en la base de datos. (Kelektiv, n.d.)

- **pnpm install cors**

Instala cors, necesario para permitir o restringir el acceso al backend desde otros dominios, como el frontend. (Expressjs, n.d.)

- **pnpm install dotenv**

Instala dotenv, que permite cargar variables de entorno desde un fichero .env para gestionar datos sensibles y configuraciones del proyecto. (Motdotla, n.d.)

- **pnpm install jsonwebtoken**

Instala jsonwebtoken, la librería usada para generar y validar tokens JWT en el sistema de autenticación. (Auth0, n.d.)

- **pnpm install multer**

Instala multer, utilizada para gestionar la subida de archivos al servidor. (Expressjs, n.d.)

- **pnpm install nodemailer**

Instala nodemailer, que permite enviar correos electrónicos desde el backend mediante credenciales. (Nodemailer, n.d.)

- **pnpm install pg**

Instala pg, el paquete oficial para conectar el backend con una base de datos PostgreSQL. (Brianc, n.d.)

8.5.3.1.2 Ficheros necesarios

- **package.json**

Fichero generado al inicializar el proyecto con pnpm init. Almacena toda la información necesaria para que la aplicación Node.js funcione correctamente: el nombre del proyecto, la versión, los scripts de ejecución y el listado de dependencias instaladas tanto en producción como en desarrollo.

- **pnpm-lock.yaml**

Fichero generado automáticamente por pnpm que registra las versiones exactas de cada dependencia instalada. Garantiza que cualquier instalación futura del

proyecto, ya sea en otro entorno o por otro desarrollador, utilice exactamente las mismas versiones, evitando incompatibilidades.

- **app.js**

Fichero donde se concentra la configuración inicial de Express: el registro de middlewares, la definición de las rutas de la API y cualquier ajuste global que el servidor necesite antes de ponerse a escuchar peticiones.

- **index.js**

Punto de entrada de la aplicación. Su único cometido es importar la configuración definida en app.js y arrancar el servidor en el puerto especificado, confirmando por consola que el proceso se ha iniciado correctamente.

- **.env**

Directorio donde se encontrarán las variables más importantes de la aplicación, las cuales no se deben de compartir ni ser visibles por los usuarios.

8.5.3.2 Directorios y ficheros utilizados

8.5.3.2.1 Config

Directorio en el que tendrán las variables para que la aplicación funcione de forma correcta.

- **config.js**

Fichero js en donde se llamarán las variables que están registradas en el fichero .env mediante dotenv para poder exportarlas a cualquier parte de la aplicación.

8.5.3.2.2 Controllers

Directorio donde se ubican todos los controladores de la aplicación, cuyo único propósito es recibir las peticiones, llamar al servicio correspondiente que contiene la lógica de negocio y devolver la respuesta.

- **email.controller.js**

Fichero donde el controller envía el id del usuario traído desde el body al servicio que actualiza el código de verificación de este (codeAccount).

- **forum.controller.js**

Fichero donde se encuentran los controladores para Get, Post, Delete, Put y Patch y Search que hacen llamado a sus servicios respectivos devolviendo un

objeto en formato JSON (get_forums, get_forum, post_forum, delete_forum, put_forum, patch_forum y search_forums).

- **forum_comment.controller.js**

Fichero donde se definen los controladores encargados de gestionar las operaciones sobre los comentarios del foro, conectándose a sus servicios y devolviendo la respuesta en JSON (get_forum_comment, get_forum_comments_by_forum_id, post_forum_comment, delete_forum_comment, put_forum_comment y patch_forum_comment).

- **links.controller.js**

Fichero donde se encuentran los controladores que manejan las operaciones relacionadas con los enlaces, que llaman a sus servicios y devuelven la respuesta en formato JSON (get_links, get_link, post_link, delete_link, put_link y patch_link).

- **profiles.controller.js**

Fichero donde se definen los controladores responsables de gestionar los perfiles de usuario, conectándose a sus servicios y devolviendo el resultado en JSON (get_profiles, get_recent_profiles, get_profile, get_private_profile, post_profile, delete_profile, put_profile y patch_profile).

- **sign.controller.js**

Fichero donde se agrupan los controladores de registro e inicio de sesión, que reciben las peticiones, llaman a sus servicios y devuelven la respuesta en JSON (sign_in_user, sign_up_user, log_out_user, verifyAccount, resendCode, forgot_password y reset_password).

- **token.controller.js**

Fichero donde se encuentran los controladores que gestionan la validación y renovación de tokens, conectándose a sus servicios y devolviendo la respuesta en formato JSON(token y refreshToken).

- **users.controller.js**

Fichero donde se definen los controladores encargados de manejar las operaciones sobre usuarios, que llaman a sus servicios y devuelven la respuesta en formato JSON(get_users, get_user_public, get_user, post_user, delete_user, put_user, patch_user y search_users).

8.5.3.2.3 Db

Directorio que contiene los ficheros clave relacionados con la base de datos.

- **connection_db.model.js**

Fichero que se encarga de establecer la conexión con la base de datos PostgreSQL usando el objeto “pool” de la librería pg, cargando los datos de la conexión desde las variables exportadas en config.js.

- **database_final.sql**

Fichero que contiene el script usado para crear la base de datos con sintaxis de PostgreSQL, incluyendo la creación de la base de datos, las tablas y los índices necesarios.

- **inserts_and_selectors.sql**

Fichero que contiene las consultas SELECT generales de la base de datos y las inserciones de datos iniciales para poblar las tablas al inicio del proyecto.

8.5.3.2.4 Middlewares

Directorio que contiene ficheros que actúan como capas intermedias entre la petición entrante y el controlador, para validar y preparar los datos antes de llegar al controller.

- **auth.middleware.js**

Fichero que se encarga de validar el token del usuario y, si es necesario, gestionar el envío y verificación de la clave de verificación (authenticate y verifyAccount).

- **password.middleware.js**

Fichero que se encarga del tratamiento de las contraseñas enviadas desde el backend, aplicando cifrado y validación tanto para el registro como para el inicio de sesión (prepare_sign_up y hash_password).

8.5.3.2.5 Models

Directorio que contiene los modelos de la base de datos para mantener consistencia entre los datos recibidos desde la base de datos y los objetos que se envían al cliente.

- **forum.model.js**

Fichero que recrea la estructura de la entidad Forums de la base de datos, definiendo los campos y relaciones que se usan en el backend.

- **forum_comment.model.js**

Fichero que recrea la estructura de la entidad Forum_Comments de la base de datos, mapeando sus campos y relaciones dentro del modelo.

- **link.model.js**

Fichero que recrea la estructura de la entidad Links de la base de datos, normalizando los datos que se envían y reciben respecto a los enlaces.

- **profile.model.js**

Fichero que recrea la estructura de la entidad Profiles de la base de datos, definiendo los atributos y relaciones asociadas al perfil de usuario

- **role.model.js**

Fichero que recrea la estructura de la entidad Roles de la base de datos, representando los distintos roles y permisos dentro del sistema.

- **user.model.js**

Fichero que recrea la estructura de la entidad Users de la base de datos, definiendo los campos y relaciones principales del usuario en la aplicación.

8.5.3.2.6 Public

Directorio en el que almacenaran ficheros de acceso público, si es que se necesitan a futuro.

8.5.3.2.7 Repository

Directorio que contiene las sentencias SQL que se ejecutan directamente contra la base de datos, sin incluir lógica de negocio extra.

- **forum.repository.js**

Fichero que contiene los métodos getForum, getForumTitle, getForums, postForum, deleteForum, putForum, patchForum y searchForums, que interactúan directamente con la base de datos usando parámetros o comodines para evitar inyecciones SQL.

- **forum_comment.repository.js**

Fichero que contiene los métodos getForumComment, getForumCommentsByForum, postForumComments, deleteForumComment, putForumComment y patchForumComment, encargados de ejecutar las consultas sobre los comentarios del foro directamente en la base de datos.

- **link.repository.js**

Fichero que contiene los métodos `createLink`, `deleteLink`, `putLink` y `patchLink`, que realizan las operaciones CRUD sobre la tabla de enlaces usando consultas SQL directas.

- **profile.repository.js**

Fichero que contiene los métodos `getProfileById`, `getProfile`, `createProfile`, `deleteProfile` y `patchProfile`, que gestionan las operaciones sobre el perfil de usuario mediante consultas SQL directas.

- **user.repository.js**

Fichero que contiene los métodos `getUser`, `checkUserSign`, `checkUser`, `createUser`, `deleteUser`, `putUser`, `patchUser`, `updatePassword`, `updateVerifyCode` y `searchUsers`, que interactúan directamente con la tabla de usuarios para todas las operaciones necesarias en la aplicación.

8.5.3.2.8 Routes

Directorio que contiene los ficheros que definen las rutas o endpoints de la API. Cada fichero redirige las peticiones a los controllers correspondientes, teniendo en cuenta que las rutas de lectura simple no usan el middleware de autenticación.

- **forum.routes.js**

Fichero que define las rutas del foro, importando los controladores de `forum.controller.js` y `forum_comment.controller.js`, y asignando cada endpoint al método adecuado. Las rutas de lectura (`/forums`, `/forums/:id`, `/forum/:id/comments`, `/forum/comments/:id`, `/forums/search`) no llevan middleware de autenticación, mientras que las de escritura (`post_forum`, `delete_forum`, `put_forum`, `patch_forum`, así como las de comentarios) van acompañadas del middleware `authenticate` para asegurar que solo usuarios autenticados puedan modificar el foro.

- **links.routes.js**

Fichero que define las rutas relacionadas con los enlaces y sus estadísticas, usando los controladores de `links.controller.js` y `link_stats.controller.js`. Las rutas de lectura (`/links/:profileId` y `/link/:id`) no requieren token, mientras que las de

escritura (post_link, delete_link, put_link, patch_link, y sus equivalentes de estadísticas) están protegidas con el middleware authenticate.

- **profiles.routes.js**

Fichero que define las rutas para la gestión de perfiles, importando los controladores de profiles.controller.js junto con el middleware uploadProfileImage de multer para la subida de la imagen de perfil. Las rutas públicas (/profiles, /profiles/recent, /profile/:username) no están protegidas, mientras que /profile/private/:username y la modificación del perfil (/profile/:username) usan authenticate y el middleware de subida de imagen, respectivamente.

- **sign.routes.js**

Fichero que define las rutas de registro e inicio de sesión, importando los controladores de sign.controller.js y los middlewares prepare_sign_up y hash_password. Las rutas de autenticación (/sign-in, /sign-up, /verify-code, /resend-code, /forgot-password, /reset-password) se encargan de gestionar el flujo completo de autenticación, incluyendo el registro, login, verificación por código y recuperación de contraseña.

- **token.routes.js**

Fichero que define las rutas para la validación y renovación de tokens, usando el controlador de token.controller.js y el middleware authenticate. El endpoint /authenticate valida el token actual, mientras que /refresh se usa para renovar el token sin obligar al usuario a volver a iniciar sesión.

- **users.routes.js**

Fichero que define las rutas para la gestión de usuarios, importando los controladores de users.controller.js y los middlewares authenticate, verifyAccount y hash_password. Las rutas de lectura pública (/users/search, /users, /user/verify/:username) no están protegidas, mientras que las operaciones de escritura y las lecturas privadas (/user/:username, /user, /user/:id con put y patch) pasan por el middleware de autenticación y el de cifrado de contraseñas para garantizar seguridad y consistencia.

8.5.3.2.9 Services

Directorio que contiene la lógica de negocio implementada para cada entidad de la base de datos, sirviendo como puente entre los controllers y los repositories.

- **forum.services.js**

Fichero que implementa la lógica relacionada con la entidad Forum, usando ForumRepository y métodos como getForumById, createForum, deleteForum, updateForum, patchForum y searchForums para gestionar los hilos del foro.

- **forum_comment.services.js**

Fichero que implementa la lógica para los comentarios del foro, con métodos como getCommentById, getCommentsByForumId, createComment, deleteComment, updateComment y patchComment que se comunican con el repository correspondiente.

- **link.services.js**

Fichero que implementa la lógica para los enlaces, usando métodos como getLinksByProfile, getLinkById, createLink, deleteLink, updateLink y patchLink para gestionar la relación entre perfiles y enlaces.

- **profile.services.js**

Fichero que implementa la lógica para los perfiles de usuario, con métodos como get_profiles, get_recent_profiles, get_profile y patch_profile que preparan los datos antes de devolverlos al controller.

- **sign.services.js**

Fichero que implementa la lógica de registro e inicio de sesión, con métodos como signIn, signUp, verifyAccount, resendCode, forgotPassword y resetPassword que coordinan validaciones, envío de códigos y gestión de cuentas.

- **token.services.js**

Fichero que implementa la lógica para la renovación de tokens, con el método refreshAccessToken que genera un nuevo token de acceso a partir de un token de refresco válido.

- **user.services.js**

Fichero que implementa la lógica asociada a la entidad User, con métodos como getUsers, getUser, userExists, createUser, deleteUser, updateUser, patchUser y searchUsers para gestionar todo lo relacionado con los usuarios.

- **user_verification.services.js**

Fichero que implementa la lógica de verificación de cuentas de usuario, con el método scheduleCodeExpiration que gestiona el plazo de expiración del código de verificación enviado por correo.

8.5.3.2.10 Upload

Directorio donde se almacenan los archivos subidos por el usuario, correspondiéndose en este caso con las imágenes de perfil que se guardan dentro de la carpeta Profile.

8.5.3.2.11 Utils

Utilidades que serán reutilizables en gran parte del código o áreas específicas.

- **crearclave.js**

Fichero que exporta funciones para generar claves aleatorias con distintos formatos.

- **crearClave64dig()**

Genera una clave de 64 caracteres usando letras, números y símbolos.

- **crearClave32digNL()**

Genera una clave de 32 caracteres usando solo letras y números.

- **crearClaveAuth()**

Genera un código de verificación de 6 dígitos separado por un guion en la mitad, pensado para enviar por correo.

- **jwt.utils.js**

Fichero que implementa la lógica de tokens JWT usando las claves definidas en config.js.

- **createAccessToken(id, username, role)**

Genera un token de acceso con duración de 1 día, usando JWT_ACCESS_SECRET.

- **createRefreshToken(id, tokenVersion)**

Genera un token de refresco con duración de 7 días, usando `JWT_REFRESH_SECRET`.

Se basa en el patrón típico donde el access token es de corta vida y el refresh token permite renovarlo sin pedir la contraseña al usuario.

- **multer.utils.js**

Fichero que configura Multer para la subida de imágenes de perfil, asegurando que el directorio `upload/profiles` exista.

Define `uploadProfileImage` con almacenamiento en disco, nombres únicos basados en timestamp y límite de 3 MB, solo permitiendo extensiones de imagen comunes (jpeg, jpg, png, webp).

- **nodemailer.utils.js**

Fichero que configura el cliente de envío de correos usando Nodemailer con los datos de SMTP definidos en `config.js`.

- **sendEmail(email, code)**

Envía un correo HTML de verificación con un código de activación, usando una plantilla con diseño responsive y un mensaje claro para el usuario.

La función devuelve `true` si el envío es correcto y `false` en caso de error.

8.5.4 Desarrollo del frontend

8.5.4.1 Elección del frontend

Para el frontend se ha elegido React, debido a su arquitectura basada en componentes reutilizables, que facilita la organización del código, el mantenimiento de la aplicación y la escalabilidad del proyecto. Asimismo, se ha empleado Vite como entorno de desarrollo y herramienta de construcción, por su rapidez de arranque, su ligereza y su optimización para el trabajo con aplicaciones web modernas. Esta combinación permite desarrollar una interfaz dinámica, eficiente y con una experiencia de usuario más fluida.

8.5.4.1.1 Comandos para iniciar

- **`pnpm create vite@latest`**

Crea el proyecto front con Vite, usando la plantilla por defecto de React o la que se seleccione en el flujo interactivo. (Vite, n.d.)

- **`pnpm i @mui/material @emotion/react @emotion/styled`**

Instala Material-UI y sus dependencias de Emotion para usar componentes estilizados y el sistema de diseño de MUI. (Material UI, n.d.)

- **`pnpm i react-router-dom`**

Instala React Router para gestionar las rutas y la navegación entre pantallas de la aplicación. (Remix-run, n.d.)

- **`pnpm i js-cookie`**

Instala el paquete js-cookie para leer, escribir y borrar cookies de forma sencilla desde el navegador. (Duian, n.d.)

- **`pnpm i react-ga4`**

Instala el paquete react-ga4 para integrar Google Analytics 4 y el seguimiento de eventos en el frontend. (Codler, n.d.)

- **`pnpm i react-helmet-async`**

Instala react-helmet-async para gestionar dinámicamente el <head> de la página (títulos, metas, tags, etc.) en una aplicación React asincrónica. (Staylor, n.d.)

- **`pnpm i react-easy-crop`**

Instala react-easy-crop para añadir un componente de recorte de imágenes sencillo y personalizable. (ValentinH, n.d.)

- **`pnpm i react-verification-input`**

Instala react-verification-input para gestionar de forma visual y cómoda la entrada del código de verificación (por ejemplo, para el código de 6 dígitos). (andreaswilli, n.d.)

8.5.4.1.2 Ficheros necesarios

- **`package.json`**

Fichero generado al inicializar el proyecto con `pnpm create vite@latest`. Almacena toda la información necesaria para que la aplicación React funcione correctamente: el nombre del proyecto, la versión, los scripts de ejecución y las dependencias instaladas tanto en producción como en desarrollo.

- **`pnpm-lock.yaml`**

Fichero generado automáticamente por pnpm que registra las versiones exactas de cada dependencia instalada. Garantiza que cualquier instalación futura del frontend, ya sea en otro entorno o por otro desarrollador, use exactamente las mismas versiones, evitando incompatibilidades.

- **eslint.config.js**

Fichero de configuración de ESLint en el nuevo formato experimental, usado para definir reglas de linting, plugins (como React, hooks o accesibilidad) y prevenir errores de estilo o código en el frontend.

- **main.jsx**

Fichero de entrada principal de la aplicación React. Es donde se monta el árbol de componentes usando createRoot y se renderiza el componente raíz dentro del nodo del DOM definido por index.html.

- **.env**

Fichero donde se almacenan las variables de entorno del frontend, como las credenciales de la API, que no se deben compartir ni exponer en el código fuente público

- **vite.config.js**

Fichero de configuración de Vite, donde se definen ajustes como el puerto del servidor de desarrollo, alias de rutas, proxy a la API del backend, plugins de React y opciones de build para producción.

- **index.html**

Fichero HTML base del proyecto. Es el punto de entrada físico del navegador, donde se define el contenedor (`<div id="root">`) en el que React inyecta la aplicación y se incluyen metaetiquetas básicas de la página.

8.5.4.2 Directorios y ficheros utilizados

8.5.4.2.1 Public

Directorio raíz para los archivos estáticos que se copian tal cual al build de producción y que no pasan por el bundler de Vite.

- **ads.txt**

Fichero utilizado por plataformas de publicidad para declarar los anuncios permitidos en el dominio.

- **apple-touch-icon.png**

Icono que se usa en dispositivos Apple cuando el usuario añade la página al inicio de iOS.

- **favicon-96x96.png**

Favicon en tamaño 96×96 para navegadores y motores de búsqueda.

- **favicon.ico**

Favicon clásico que se usa en navegadores tradicionales y en pestañas del navegador.

- **favicon.svg**

Favicon en formato vectorial para pantallas con alta densidad de píxeles y soporte moderno.

- **robots.txt**

Fichero que indica a los robots de los buscadores qué rutas pueden o no indexar.

- **robots2.txt**

Fichero alternativo o auxiliar de configuración para robots de indexado, usado para pruebas o entornos separados.

- **site.webmanifest**

Fichero de manifest de la PWA que define nombre, iconos, tema y comportamiento de la aplicación al instalarse.

- **sitemap.xml**

Mapa del sitio en formato XML para ayudar a los motores de búsqueda a descubrir las páginas de la web.

- **web-app-manifest-192x192.png**

Icono de la PWA con tamaño 192×192 para homescreen y OS modernos.

- **web-app-manifest-512x512.png**

Icono de la PWA con tamaño 512×512 para soporte de alta resolución y pantallas grandes.

8.5.4.2.2 Src

Directorio principal del frontend donde se almacena todo el código fuente de la aplicación React, incluyendo el punto de entrada, assets, componentes, páginas y utilidades.

- **Main.jsx**

Fichero de entrada principal del frontend React, donde se configuran el router, el contexto de autenticación, el manejo de SEO y el punto de montaje en el DOM.

Usa BrowserRouter para gestionar las rutas y React.lazy con Suspense para cargar de forma perezosa (lazy) páginas grandes como YourTree, YourTreeUser, Forums, Recent_Pages, Profile, Forgot_Password y Reset_Password, dejando solo las páginas ligeras (Home, Sign_in, Sign_up, Log_out, NotFound) importadas de forma estática.

Incluye Helmet y HelmetProvider para gestionar títulos, meta tags y SEO por ruta, usando un objeto titles y descriptions que mapean cada ruta a su contenido correspondiente.

Condiciona la visualización del Header según la ruta (oculta el header en la ruta de/YourTree/:username), mientras que mantiene visibles el tracker de AnalyticsTracker y el botón BuyMeACoffee en todo el sitio.

Monta todo dentro de StrictMode y AuthProvider, para activar el modo estricto de React, proveer el estado de autenticación global y renderizar la aplicación sobre el nodo #root del DOM mediante createRoot.Assets.

- **YourTree.ico y YourTree.svg**

Iconos del proyecto usados como favicon y logotipo dentro de componentes y páginas.

- **Css**

Directorio con hojas de estilo globales y por sección.

- **ads.css**

estilos para elementos de publicidad.

- **background_dark.css y background_light.css**

temas de fondo oscuro y claro.

- **base.css**

Fichero de estilos base que define el sistema de diseño global de la aplicación, usando variables CSS en :root y distinguiendo temas claro y oscuro mediante el atributo data-theme en el html o en el :root. Importa Tailwind CSS y agrupa tipografía, espaciado, bordes, transiciones y efectos de “glassmorphism” en un único punto de configuración.

- **Colores y temas**

Define una paleta de colores organizada por superficies, texto, bordes, estados (éxito, alerta, información, warning) y degradados, para que el resto de CSS pueda usarlas mediante var(--color-...).

El tema claro (light) usa un fondo muy suave cercano al blanco y textos oscuros, mientras que el tema oscuro (dark) usa un fondo grafitho suave y textos claros, manteniendo el mismo esquema de variables pero con valores distintos para cada :root[data-theme="..."].

- **Tipografía, espaciado y bordes**

Declara familias tipográficas (--font-heading, --font-body) y pesos (--fw-light, --fw-regular, --fw-medium, etc.) para que el resto de la aplicación tenga texto coherente.

Define un sistema de espaciado (--space-xs, --space-sm, hasta --space-3xl) y bordes redondeados (--radius-...) usados como unidades base en el resto de las hojas de estilo.

- **Transiciones y efectos visuales**

Incluye variables de transición rápidas, base y lentas (--transition-fast, --transition-base, --transition-slow) para animaciones suaves al cambiar colores o fondos.

Configura estilos de “glassmorphism” y opacidad de overlays, que se usan en tarjetas o paneles modales mientras otros ficheros de

CSS (theme-toggle.css, base_card.css) consumen estas variables.

- **base_card.css, links_base.css**

Estilos base para tarjetas, foro y enlaces.

- **buy-me-a-coffee.css, footer.css, forms.css, Sign.css, Profile.css, forums.css, header.css, home.css, icons.css, loader.css, LogoSVG.css, recent_pages.css, theme-toggle.css y YourTree.css**

Estilos para componentes o páginas concretas.

- **Images**

Directorio que contiene las imágenes que muestran en la pagina web.

- **Happy_Users.svg y people_thinking.svg**

Ilustraciones para páginas de marketing o landing.

- **profile_default.ai y profile_default.svg**

Imagen de perfil por defecto para usuarios sin avatar subido

- **YourTree Title.ai y YourTree Title.svg**

Título estilizado del proyecto para cabeceras o secciones.

- **Js**

Directorio que contiene ficheros que tienen acceso al DOM o contenido de JavaScript vanilla.

- **Demo.js**

scripts de pruebas rápidas que se utilizaron para comprobaciones.

- **Header-js.jsx**

Fichero con funciones de utilidad para el header React, usado dentro de header.jsx.

- **HeaderScroll()**

Hook que detecta la dirección de scroll y añade/elimina la clase header-hidden al header.

Mantiene el header visible cuando se está cerca del top (por debajo de SHOW_AT_TOP) y lo oculta cuando se baja suficiente ($\text{diff} > \text{DELTA}$), mostrándolo de nuevo cuando se sube ($\text{diff} < -\text{DELTA}$).

- **HeaderPageEffect()**

Hook que gestiona el estado activo de los enlaces del header.

Al hacer clic en un enlace se añade la clase active a ese enlace y se la quita al resto, manteniendo la navegación por SPA sincronizada con el estilo visual.

- **HeaderSmallScreen()**

Placeholder para lógica específica de header en móvil, pensado para extender con menú desplegable, márgenes, etc.

- **handleLinkClick()**

Función que se llama desde los enlaces del menú para cerrar el menú móvil (`setMobileMenuOpen(false)`) al hacer clic en cualquier opción.

- **useClickOutside(mobileMenuOpen, mobileMenuClose, selector)**

Hook genérico que cierra el menú cuando se hace clic fuera del selector indicado (header por defecto).

Añade el listener mousedown al document solo mientras mobileMenuOpen es true y lo limpia en el useEffect, para evitar fugas de memoria.

- **Loader.js**

Fichero que define una animación de carga visual y controla el comportamiento del loader hasta que la página está lista.

- **simulateLoading()**

Captura el tiempo de carga de la página con `performance.getEntriesByType` y anima una barra de progreso (loading-bar) que va del 0 al 100%. Mientras la animación está en curso, el loader ocupa toda la

pantalla y el body tiene overflow: hidden; cuando la barra llega al 100% oculta el loader y vuelve a overflow: auto. Además, genera un texto animado “Loading ...” en el spinner usando span con la clase jump para dar sensación de actividad.

- **ThemeToggle.jsx**

componente de alternancia entre tema claro y oscuro.

8.5.4.2.2.1 Components

Directorio donde se concentran los componentes reutilizables de la interfaz, desde autenticación hasta formularios y gestión de enlaces.

- **Ads.jsx**

Componente que integra anuncios de Google AdSense usando ins.adsbygoogle y carga el script de forma dinámica.

Si el script falla o no carga en 5 segundos, muestra un bloque de “anuncio” estático estilo tarjeta (Anuncio, Contenido patrocinado) con un botón Ver oferta.

- **Auth.jsx**

Componente que define un contexto global de autenticación (AuthProvider) y un hook useAuth para consumirlo.

Almacena en cookies el token, el refreshToken y los datos del usuario, expone login, logout, isLoggedIn, isVerify y loading.

Valida el token al montar y en cada cambio de ruta, eliminando cookies y disparando logout si el token no es válido.

- **BuyMeACoffee.jsx**

Componente simple que muestra un botón “Buy me a coffee” con icono SVG, que enlaza a una página de donaciones en PayPal, pensado para monetización ligera o apoyo al proyecto.

- **Collapsible.jsx**

Componente Collapsible que renderiza un acordeón controlable (título + botón ▲/▼) y un cuerpo que aparece/oculta según el estado open,

usando `useMemo` para el texto del botón y `React.memo` para evitar re-renders innecesarios.

- **forms.jsx**

Conjunto de formularios específicos de la aplicación, todos envueltos en `React.memo` para optimizar rendimiento.

- **FormUpdateLink**

Abre un `<dialog>` para editar un enlace específico, hace PUT a la API y muestra alertas de éxito/error con MUI Alert.

- **FormCreateLink**

Crea un nuevo enlace asignado a un perfil, enviando POST con `profile.id`.

- **FormDeleteLink**

Borra un enlace mediante DELETE y confirmación en modal.

- **FormUploadImage**

Gestiona subida de imagen de perfil usando `react-easy-crop` y `cropUtils.js` para recortar a círculo antes de subirla.

- **FormCodeVerification**

Interfaz modal para introducir el código de 6 dígitos, con contador de 30s, botón “Resend Code” y lógica de `verify-code` contra la API.

- **GA_4.jsx**

Componente `AnalyticsTracker` que inicializa Google Analytics 4 y envía un evento `pageview` cada vez que cambia la ruta, usando `ReactGA.send` con `useLocation`.

- **header.jsx**

Componente principal de cabecera, que usa contexto de autenticación, tema oscuro/claro y lógica de responsive.

Condiciona la vista según el ancho de pantalla, muestra un overlay de menú y sincroniza el estado con el efecto de scroll ocultando el header al bajar lo suficiente.

En móvil carga el componente HeaderPageEffect y HeaderScroll y consume useClickOutside para cerrar el menú al hacer clic fuera.

- **icons.jsx**

Módulo que agrupa íconos SVG reutilizables (por ejemplo, ShareIcon) usados en distintos componentes, como linksBase, botones de acción o formularios.

- **linksBase.jsx**

Componente que monta y gestiona el bloque de enlaces de un perfil (PutLinks, Links_base, General_tree).

- **PutLinks**

Recupera los enlaces de la API de un perfil por username, los guarda en estado y genera una lista de > que inyectan acciones (FormUpdateLink, FormDeleteLink) mediante children.

- **Links_base**

Consume PutLinks para mostrar el árbol de enlaces estructurado según el layout visual de la app.

- **LogoSVG.jsx**

Componente de logo SVG (Logo_yourtrees) que encapsula la imagen principal de marca y se usa en el header y en distintas páginas para mantener consistencia visual.

- **SearchComponent.jsx**

Componente de búsqueda que se integra dentro del header, usando react-router-dom y manejando la lógica de filtrado/consulta según el texto introducido.

- **seo.jsx**

Componente que controla títulos, metadescripción y tags SEO por ruta, usando Helmet para gestionar dinámicamente el <head> en una SPA React.

- **token.jsx**

Componente de utilidad que agrupa funciones de manejo de JWT.

- **getToken, getRefreshToken**

Leen tokens de cookies.

- **refreshToken**

Actualiza el token de acceso mediante el refresco.

- **verifyToken**

Valida el token actual contra el backend y devuelve un booleano de autenticación.

- **UsePassword.jsx**

Componente que agrupa controles de contraseña: PasswordField y VerifyPassword, usados en formularios de registro, cambio de contraseña y verificación, con validación visual y lógica de contraseña segura.

8.5.4.2.2.2 Pages

Directorio donde se concentra la lógica de cada página de la aplicación. Cada fichero corresponde a una ruta principal gestionada por React Router DOM.

- **Forgot_Password.jsx**

Página de recuperación de acceso. El usuario introduce su nombre de usuario o correo electrónico y el sistema envía un código de recuperación a su dirección de email. Una vez enviado correctamente, redirige automáticamente a la página de Reset_Password tras unos segundos.

- **Forums.jsx**

Página principal de la comunidad. Carga los foros disponibles mediante scroll infinito usando IntersectionObserver, mostrando cada hilo en forma de tarjeta con título, descripción y fecha. Al hacer clic en una tarjeta, se abre un diálogo nativo (<dialog>) con el detalle completo del foro y sus comentarios. Desde ahí el usuario puede comentar, compartir el enlace o borrar el foro si es el autor. También incluye el formulario de creación de nuevos hilos, accesible solo para usuarios autenticados.

- **Home.jsx**

Página de inicio de la plataforma. Es la primera vista que recibe cualquier visitante y sirve como punto de entrada general a la aplicación.

- **Log_out.jsx**

Página de cierre de sesión. Al montarse ejecuta automáticamente la función logout del contexto de autenticación y redirige al usuario a la página de inicio, sin mostrar ningún contenido visible.

- **NotFound.jsx**

Página de error 404. Se muestra cuando el usuario intenta acceder a una ruta que no existe dentro de la aplicación.

- **Profile.jsx**

Página de perfil del usuario. Si el usuario accede a su propio perfil, muestra un formulario editable con campos como nombre, apellidos, fecha de nacimiento, correo de recuperación, biografía, tema visual e imagen de perfil. Para evitar que el navegador muestre una versión antigua de la foto tras actualizarla, se usa un cacheBuster que fuerza la descarga de la imagen actualizada. Si el perfil visitado pertenece a otro usuario, la vista se adapta mostrando solo la información pública.

- **Recent_Pages.jsx**

Página de descubrimiento de perfiles. Recupera de la API los perfiles que tienen al menos un enlace publicado y los muestra en una cuadrícula de tarjetas. Al hacer clic en cualquier tarjeta, navega al perfil público de ese usuario.

- **Reset_Password.jsx**

Página de restablecimiento de contraseña. El usuario introduce el código recibido por email junto con su nueva contraseña y su confirmación. Si viene redirigido desde Forgot_Password, el campo de usuario o email se autocompleta automáticamente usando el estado de navegación de React Router.

- **Sign_in.jsx**

Página de inicio de sesión. Recoge el nombre de usuario o email y la contraseña, los envía al backend y, si la respuesta es correcta, almacena los tokens mediante el AuthProvider y redirige al usuario a la página de inicio. Incluye un enlace directo a Forgot_Password para los usuarios que no recuerden su contraseña.

- **Sign_up.jsx**

Página de registro. Recoge nombre de usuario, email, contraseña y confirmación de contraseña. Usa el componente `PasswordField` para los campos de contraseña y la función `VerifyPassword` para comparar en tiempo real ambos valores, bloqueando el envío del formulario mientras no coincidan.

- **YourTree.jsx**

Página principal del editor de perfil. Permite al usuario gestionar su página pública: subir o eliminar su foto de perfil, editar su biografía y administrar sus enlaces mediante operaciones CRUD. Incluye una vista previa en tiempo real de cómo quedará su página pública. Si el usuario no está autenticado, redirige automáticamente a `Sign_in`.

- **YourTreeUser.jsx**

Vista pública del perfil de cada usuario. Antes de renderizar nada, verifica mediante una llamada asíncrona a la API que el usuario de la URL existe. Si no existe, redirige a la página 404 usando `replace: true` para no dejar la búsqueda fallida en el historial del navegador. Si existe, carga su biografía e imagen de perfil y renderiza su árbol de enlaces público.

8.5.4.2.2.3 *Utils*

Directorio que contiene funciones auxiliares reutilizables independientes de la interfaz visual.

- **cropUtils.js**

Fichero que centraliza toda la lógica de recorte y transformación de imágenes antes de subirlas al servidor. Trabaja directamente con la API Canvas del navegador para procesar la imagen en el cliente, evitando enviar ficheros sin recortar al backend.

El fichero contiene las siguientes funciones:

- **createImage**

Crea un objeto `Image` a partir de una URL y lo devuelve como una promesa. El atributo `crossOrigin` se establece como "anonymous" para evitar bloqueos CORS al cargar imágenes desde dominios externos.

- **getRadianAngle**

Convierte un valor de rotación en grados a radianes, necesario para los cálculos trigonométricos del canvas.

- **rotateSize**

Calcula el nuevo ancho y alto del área que ocupa un rectángulo tras aplicarle una rotación, usando las funciones `Math.cos` y `Math.sin`. Esto es necesario para que el canvas tenga el tamaño correcto y la imagen no quede cortada al rotar.

- **getCroppedImg**

Es la función principal del fichero. Recibe la imagen original, las coordenadas del área de recorte en píxeles, el ángulo de rotación y si se debe voltear horizontal o verticalmente. Con esos datos crea un canvas, aplica las transformaciones mediante `translate`, `rotate` y `scale`, dibuja la imagen y extrae únicamente la zona seleccionada con `getImageData`. El resultado se devuelve como un archivo Blob en formato PNG, listo para enviarse al servidor como parte de un FormData.

8.5.5 Descripción de los componentes (Frontend)

En este apartado se detallan los componentes clave que forman parte de la lógica y la interfaz de la aplicación. Seleccionado aquellos que desempeñan funciones críticas en la seguridad, el posicionamiento y la analítica del sistema.

8.5.5.1 Componentes de lógica y seguridad

8.5.5.1.1 Auth.jsx

Auth.jsx es el componente que da acceso al estado de autenticación a toda la aplicación. Se implementa como un proveedor de contexto (AuthProvider) que envuelve la web entera, de forma que cualquier componente hijo pueda saber en todo momento si hay un usuario conectado o no. En este fichero realizamos varias tareas críticas:

Para gestionar ese estado se usan los hooks `useState` y `useEffect`, que controlan el objeto del usuario (user) y el booleano de sesión activa (isLoggedIn). Al arrancar la web se lanza automáticamente una función asíncrona que comprueba si los tokens guardados en las cookies siguen siendo válidos en el servidor, mostrando un estado de carga (loading) mientras se resuelve esa verificación.

Las funciones de login y logout manejan las cookies a través de la librería js-cookie con políticas estrictas (secure, sameSite: "strict"). Al hacer logout, ambos tokens (acceso y el de refresco) se eliminan completamente, sin dejar ningún dato residual en el navegador.

8.5.5.1.2 Token.jsx

Token.jsx centraliza toda la lógica de tokens en el cliente. El objetivo es mantener la sesión activa sin pedirle las credenciales al usuario cada poco tiempo.

El fichero expone tres funciones de lectura (getUser, getToken y getRefreshToken) que extraen directamente de las cookies la información del usuario y sus claves de acceso.

La parte más importante es la función refreshToken: cuando el token principal caduca, hace una petición POST automática al servidor con el token de refresco para obtener uno nuevo y sobrescribir la cookie, todo sin que el usuario note ninguna interrupción. Si esa renovación también falla, entra en juego verifyToken, que envía el token actual al backend mediante el encabezado Bearer. Un error 401 en la respuesta activa el intento de renovación; si ese intento también falla, la función devuelve false y tanto el header como el AuthProvider cierran la sesión automáticamente.

8.5.5.1.3 UsePassword.jsx

UsePassword.jsx resuelve dos problemas concretos del formulario de contraseñas: que el usuario pueda ver lo que está escribiendo, y que no pueda enviar el formulario si las contraseñas no coinciden.

El componente PasswordField usa useState para alternar el tipo del input entre password y text, controlado por un botón interactivo. Recibe el valor y el handler desde el componente padre, por lo que no gestiona estado propio y puede reutilizarse en cualquier formulario del proyecto.

La función VerifyPassword hace la validación en el cliente antes de que llegue ninguna petición al servidor. Compara en tiempo real la contraseña con su confirmación y devuelve tres cosas: un booleano con el resultado, una clase CSS dinámica (declined) para colorear el campo en rojo si no coinciden, y un mensaje de error concreto. Así el usuario recibe feedback inmediato y el formulario bloquea el envío hasta que los datos sean correctos.

8.5.5.2 Componentes de Optimización y Servicios

8.5.5.2.1 GA4.jsx

GA4.jsx es un componente que no renderiza nada visible, su único trabajo es conectar la aplicación con Google Analytics 4 y registrar los movimientos del usuario en segundo plano.

El ID de seguimiento no está escrito directamente en el código, sino que se lee desde una variable de entorno (`import.meta.env.VITE_GA_ID`) a través de la librería `react-ga4`. Esto evita exponer el identificador en el repositorio y permite usar IDs distintos entre el entorno de desarrollo y el de producción sin tocar el código.

El problema concreto que resuelve este componente es que React, al ser una SPA, no recarga la página al cambiar de ruta, lo que significa que Google Analytics no detecta esos cambios por sí solo. Para solucionarlo, se combinan `useLocation` de `react-router-dom` y `useEffect`: cada vez que cambia el `pathname` o el parámetro de búsqueda, el componente dispara manualmente un evento `pageview` hacia Google Analytics, asegurando que cada visita quede registrada correctamente.

8.5.5.2.2 SEO.jsx

Las aplicaciones de una sola página tienen un problema conocido con el SEO: como todo el contenido lo inyecta JavaScript, los buscadores y las redes sociales a veces no leen bien los metadatos de cada página. `SEO.jsx` existe para resolver eso.

Usando la librería `react-helmet-async`, el componente inyecta dinámicamente las etiquetas del `<head>` según la ruta activa: título, descripción meta y palabras clave. Así cada perfil de usuario o hilo del foro tiene sus propios metadatos en lugar de compartir los mismos para toda la web, lo que mejora cómo lo indexan Google o Bing.

Además del SEO básico, el componente configura las etiquetas Open Graph y Twitter Cards. Esto es lo que controla cómo se ve un enlace al compartirlo en redes sociales, sin estas etiquetas LinkedIn o WhatsApp mostrarían una previsualización vacía o genérica en lugar del nombre y la imagen del perfil.

Por último, gestiona la URL canónica a través de `useLocation`, indicando a los buscadores cuál es la dirección oficial de cada página para evitar problemas de contenido duplicado, y controla las directivas de rastreo (`index`, `follow`) para decidir qué secciones deben ser visibles para los bots y cuáles no.

8.5.5.3 Componentes de estructura e interfaz

8.5.5.3.1 Header.jsx

Header.jsx es el componente de navegación principal y uno de los más dinámicos de la aplicación, ya que su contenido cambia según dos factores: si el usuario está autenticado y el tamaño de la pantalla.

Para gestionar la autenticación, usa el hook `useAuth` para leer el estado `isLogged`. Si el usuario no ha iniciado sesión muestra los botones de "Sign in" y "Sign up", y si ya está autenticado los sustituye por el acceso al perfil personalizado (`@username`) y la opción de cerrar sesión.

La parte responsive se resuelve leyendo `window.innerWidth` dentro de un `useEffect`, lo que permite cambiar automáticamente entre el menú de escritorio y el menú hamburguesa para móvil según el ancho de pantalla disponible.

Además del menú, el componente integra el buscador (`SearchComponent`), el selector de tema (`ThemeToggle`) y efectos visuales al hacer scroll. Para que todo esto no afecte al rendimiento, el componente está envuelto en `React.memo`, evitando renderizados innecesarios cuando las props no cambian.

8.5.5.3.2 Collapsible.jsx

`Collapsible.jsx` es un componente reutilizable que muestra u oculta bloques de contenido según la interacción del usuario. Se usa en distintas partes de la web para agrupar información extensa sin ocupar todo el espacio visual desde el inicio.

En cuanto al rendimiento, está envuelto en `React.memo` para que solo se vuelva a renderizar cuando cambian sus props. El icono del botón que indica si está abierto o cerrado (`▲` o `▼`) se calcula con `useMemo`, evitando que ese cálculo se repita en cada renderizado sin necesidad.

La flexibilidad del componente viene de la propiedad `children`, que permite meter dentro cualquier tipo de contenido: formularios, listas, texto o imágenes. Las clases CSS que aplica cambian dinámicamente según la sección donde esté integrado.

8.5.6 Estructura del proyecto

En este apartado se detalla la estructura que rige, describiendo cada directorio y su jerarquía tanto del frontend y backend respectivamente.

8.5.6.1 Capas de responsabilidad

La arquitectura del proyecto se fundamenta en una separación clara de responsabilidades para garantizar la escalabilidad y el mantenimiento. El sistema se divide en tres capas principales.

8.5.6.2 Capa de Interfaz (Frontend)

Gestionada por React y Vite, enfocada en la experiencia de usuario y la optimización de recursos estáticos.

8.5.6.3 Capa de Lógica de Negocio (Backend)

Implementada con Express.js, donde se procesan las peticiones, se gestionan los servicios y se aplican filtros de seguridad mediante middlewares.

8.5.6.4 Capa de Persistencia (Base de Datos)

Basada en PostgreSQL, aislada mediante el patrón Repository para separar las consultas SQL de la lógica del servidor.

8.5.6.5 Estructura del repositorio

8.5.6.5.1 Directorio raíz (/)

- **.gitignore:**

Fichero de configuración que instruye a git que ficheros se deben almacenar en su servidor.

En este fichero pondremos el que no se consideren los ficheros y directorios (.env, node_modules y .vscode). Dichas configuraciones serán aplicadas para el proyecto general dado a que se encuentra en la raíz en donde se ramificaran en los directorios de Frontend y Backend, los cuales se explicaran en secciones separadas más adelante.

- **Docker-compose.yml:**

El fichero nombrado docker-compose.yml indica los pasos a seguir para la creación de un contenedor en específico, siguiendo las directrices indicadas en su interior.

Dentro del archivo definimos que se tendrán dos servicios los cuales se armaran en su propio contenedor (Frontend y Backend), la construcción seguirá los pasos definidos en el Dockerfile que se encuentra en el directorio mencionado, con un nombre de contenedor descriptivo y volúmenes en los cuales se almacenaran los datos de maneta persistente sin que le afecte el reinicio del contenedor o su eliminación y ambos constaran con una cláusula muy importante que les dicta reiniciar el servicio siempre y cuando se pare.

Por otro lado, tenemos instrucciones específicas para cada volumen en el backend tendrá que importar las variables que de encuentran en el .env del directorio en el que se encuentra actualmente y en el frontend tendrá los puertos 80 y 443 expuestos para el funcionamiento del protocolo HTTP y HTTPS y una condición denominada depends_on la cual dicta el orden de construcción de los contenedores, si el contenedor del backend no está construido no se construirá el del frontend.

- **LICENSE:**

Fichero que contiene la Licencia MIT. Es una licencia de software permisiva que permite la reutilización del código dentro de software propietario o de código abierto, con la única condición de que se incluya el aviso de copyright y la autorización original en todas las copias o partes sustanciales del software.

- **Readme.md:**

Fichero que contienen una descripción del proyecto realizado con capturas de pantallas mostrando cada apartado de la web ya desplegada,

características importantes y tecnologías utilizadas en el proyecto general.

- **Auditoria de prueba:**

Directorio en el cual se encuentra la auditoría realizada con la aplicación ZAP la cual ataca la página web para detectar vulnerabilidades que puede tener llegar a tener, distinguiéndolas en cinco categorías rojo (severidad alta o critica), amarilla/anaranjado (severidad media o alertas), verde/azul (severidad baja o alertas informacionales).

- **Documentos:**

Directorio en el que se encuentran distintos ficheros que tienen importancia al realizar el proyecto, de inicio a fin, contando con la documentación, criterios a tomar en cuenta, diagramas e imágenes ilustrativas.

Ilustración 43 estructura del directorio raíz (/)


```
.gitignore
docker-compose.yml
LICENSE
README.md
YourTree.docx

├── auditoria de prueba
│   ├── 2026-03-25-ZAP-Report-.html
│   ├── Sesión sin Nombre.session
│   ├── Sesión sin Nombre.session.data
│   ├── Sesión sin Nombre.session.properties
│   └── Sesión sin Nombre.session.script
├── 2026-03-25-ZAP-Report-
│   ├── zap32x32.png
│   ├── normalize
│   │   ├── LICENSE.md
│   │   └── normalize.css
│   └── themes
│       └── original
│           ├── colors.css
│           └── main.css
└── documentos
    ├── Casos de Uso UML.drawio
    ├── Documentación a presentar y criterios 25_26.pdf
    ├── mapa de navegación.drawio
    ├── mapa de navegación.pdf
    ├── metodologia Kanban.png
    ├── proceso de maquetado web .ai
    ├── Programación Proyecto Intermodular DAW.pdf
    ├── Web de reserva de cotxes de lloguer.pdf
    ├── Xarxa social per amos de mascotes.pdf
    └── YourTree.pdf
```

8.5.6.5.2 Frontend

- **.env**

En este fichero se almacenan las variables de entorno utilizadas en el servidor de Node.js, conteniendo claves sensibles y que no pueden darse a conocer al público.

Aquí almacenamos el identificador que nos proporciona Google Analytics, para poder sacar las estadísticas que tendrá la página.

- **Docker-compose.yml**

Este archivo es una configuración provisional que sirve como respaldo. Su función principal es permitir el levantamiento del Frontend de forma aislada e independiente. Lo tenemos aquí por si el docker-compose.yml que se encuentra en la raíz del proyecto tiene algún problema o conflicto de red. Con este fichero, nos aseguramos de que el desarrollo del Frontend no se detenga, permitiendo construir el contenedor, mapear los puertos y configurar los volúmenes de manera rápida sin tener que ejecutar todo el sistema completo si no es necesario.

- **Dockerfile**

El fichero Dockerfile es el que contiene las instrucciones para empaquetar nuestra aplicación. En este caso, usamos una técnica profesional llamada Multi-stage build que se divide en dos pasos.

En la primera etapa, usamos una imagen de Node 20 muy ligera (alpine) para instalar pnpm y construir el proyecto; aquí se instalan las librerías y se corre el comando build para que Vite genere la carpeta de producción.

En la segunda etapa, pasamos a un servidor Nginx. Lo que hacemos es copiar únicamente la carpeta con el código ya compilado de la etapa anterior hacia el servidor Nginx. Esto siendo la mejor opción porque la imagen final es muy pequeña y segura, ya que no lleva el código fuente, solo lo necesario para que la web funcione. Además, exponemos los puertos 80 y 443 y configuramos el servidor para que se mantenga siempre activo.

- **Eslint.config.js**

Es el fichero de configuración encargado de la auditoría del código. Nos sirve para que el editor nos avise si cometemos errores de sintaxis o si no seguimos las buenas prácticas de programación, manteniendo el código limpio y ordenado en todo el equipo.

- **Index.html**

Fichero principal que sirve como punto de entrada de la aplicación web. Contiene el elemento raíz (`<div id="root">`) donde React montará e inyectará toda la interfaz de usuario para que el navegador la renderice según qué acciones realice y simultáneamente teniendo el SEO principal de la página web.

- **Package.json:**

Fichero en donde se almacenan los datos que se usaron para el creación, ejecución y correcto funcionamiento del Frontend, separándolas en dos secciones configuración, Scripts, dependencies y devDependencies.

- **Configuración (propiedades que objeto general)**

Son aquellas que indican el comportamiento que debe tomar el proyecto de Node.js, como su nombre, versión, tipo de empaquetado ("type": "module"), privacidad, entre muchas otras.

- **Scripts**

Se consideran los alias que ejecutan comandos complejos de la terminal de forma simplificada. Gracias a estos, podemos usar comandos cortos como dev para iniciar el servidor local, build para compilar el código a producción, lint para auditar errores, y test para correr pruebas, sin tener que escribir las instrucciones completas cada vez.

- **Dependencies y devDependencies**

Las dependencies (como React, Material UI, React Router) son las librerías esenciales que se empaquetan en el código final porque el

navegador del cliente las necesita de forma constante para que la web funcione. Por otro lado, las devDependencies (como Vite, ESLint o TailwindCSS) son exclusivas del entorno de trabajo, son herramientas de apoyo que nos ayudan a programar y compilar, pero son descartadas del empaquetado final de la build para que la página sea lo más ligera y rápida posible.

Ilustración 44 dependencias utilizadas en el frontend

```
"name": "frontend",
"private": true,
"version": "1.0.0",
"type": "module",
  > Debug
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "test": "vitest"
},
"dependencies": {
  "@emotion/react": "^11.14.0",
  "@emotion/styled": "^11.14.1",
  "@mui/material": "^9.0.0",
  "js-cookie": "^3.0.5",
  "react": "^19.1.1",
  "react-colorful": "^5.6.1",
  "react-dom": "^19.1.1",
  "react-easy-crop": "^5.5.7",
  "react-ga4": "^2.1.0",
  "react-helmet-async": "^3.0.0",
  "react-router-dom": "^7.9.6",
  "react-verification-input": "^4.2.2"
},
"devDependencies": {
  "@eslint/js": "^9.36.0",
  "@tailwindcss/vite": "^4.2.2",
  "@types/react": "^19.1.16",
  "@types/react-dom": "^19.1.9",
  "@vitejs/plugin-react-swc": "^4.1.0",
  "eslint": "^9.36.0",
  "eslint-plugin-react-hooks": "^5.2.0",
  "eslint-plugin-react-refresh": "^0.4.22",
  "globals": "^16.4.0",
  "tailwindcss": "^4.2.2",
  "vite": "^7.1.7"
}
```

- **Readme.md**

Fichero de documentación específico para la sección del Frontend. Detalla cómo levantar el entorno de desarrollo local, los comandos disponibles y características específicas de la interfaz.

- **Vite.config.js**

Este fichero configura el empaquetador y el servidor de desarrollo del frontend. Tiene tres configuraciones relevantes en el proyecto.

La primera es el plugin React-SWC, que sustituye al compilador habitual de Babel por uno escrito en Rust. El resultado es una diferencia notable en los tiempos de arranque y refresco durante el desarrollo.

La segunda es la integración de Tailwind CSS como motor de estilos, que se procesa directamente desde aquí sin configuración adicional.

La tercera, y la más importante para el flujo de desarrollo, es el proxy inverso. Sin él, cualquier petición del frontend a la API del backend es bloqueada por el navegador debido a las políticas CORS, ya que en local corren en puertos distintos. El proxy intercepta las llamadas a rutas como /yourtree/api y las redirige internamente al servidor backend, usando changeOrigin para ajustar las cabeceras automáticamente. Esto simula el comportamiento de producción donde frontend y backend comparten el mismo dominio sin tener que tocar la configuración del servidor cada vez que se trabaja en local.

- **Public**

Directorio que contiene los archivos estáticos que no pasan por el proceso de compilación, como el favicon de la web, imágenes fijas y el archivo robots.txt para el SEO.

- **Src (source)**

Directorio principal donde reside todo el código fuente de la aplicación React. Todo lo que esté aquí será procesado y optimizado por Vite siendo su punto de acceso por convención y buenas prácticas.

- **Main.jsx**

Archivo principal para la creación de la página web mediante React, puesto que en el fichero se encuentra el script para iniciar la construcción dentro del componente root.

- **Assets**

Directorio en donde se encuentran los archivos estáticos que Vite procesa, teniendo los subdirectorios Css, Images y Js.

- **Css**

Directorio en donde se almacenan todos los ficheros con extensión .css los cuales dotan el apartado visual de la página web.

- **Images**

Directorio en donde se encontrarán las imágenes estáticas a utilizar en la construcción de la página web.

- **Js**

Directorio en donde se encuentran scripts realizados con JavaScript Vanilla y React. Estos se utilizan para el manejo directo en el DOM o realizar cálculos pesados fuera del ciclo de vida de React dentro de los componentes grandes, logrando así una mejor interactividad y rendimiento en apartados específicos donde el Virtual DOM no es la opción más eficiente.

- **Components:**

Directorio en donde se encuentran los componentes que se utilizan en la página web, cumpliendo el principio de reutilización de código.

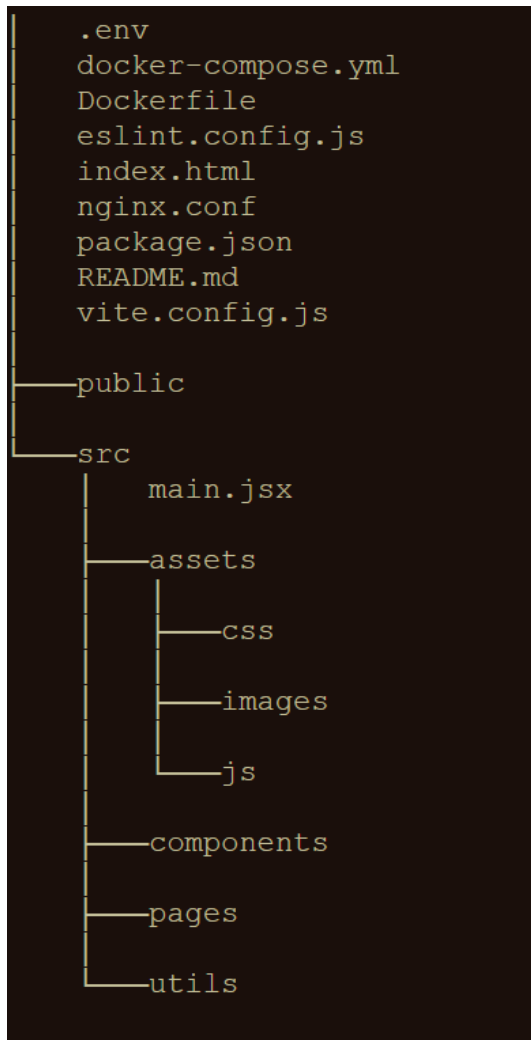
- **Pages:**

Directorio en donde se encuentran las paginas que se mostraran mediante las rutas que se implementan en main.jsx.

- **Utils:**

Directorio en donde se encuentran las funciones de utilidad o herramientas auxiliares que son independientes de la interfaz visual.

Ilustración 45 estructura de directorios Frontend



8.5.6.5.3 Backend

- **.env**

En este fichero se almacenan las variables de entorno que el servidor de Express utiliza para su configuración interna. Aquí guardamos datos críticos como las credenciales de acceso a la base de datos PostgreSQL, el puerto del servidor y las claves secretas para la firma de tokens, evitando que información sensible quede expuesta en el código fuente.

- **App.js**

App.js es donde se configura todo el comportamiento del servidor Express antes de que empiece a escuchar peticiones. No arranca el servidor, eso lo hace index.js, sino que prepara la lógica que este va a usar.

Lo primero que se configura es el CORS, leyendo el origen permitido desde una variable de entorno para que solo el frontend pueda conectarse al servidor. A continuación se registran los middlewares básicos: `express.json()` para que Express entienda el cuerpo de las peticiones en formato JSON, y el acceso estático a la carpeta de uploads para que las imágenes de perfil sean accesibles directamente mediante una URL.

Por último, todas las rutas de la API (usuarios, perfiles, estilos, foros y links) se montan bajo el prefijo `/yourtree/api`, manteniendo una estructura limpia y coherente. Al final de la cadena hay un manejador de errores 404 que responde con un mensaje claro cuando alguien intenta acceder a un endpoint que no existe.

- **Docker-compose.yml**

Fichero provisional de orquestación local. Permite levantar exclusivamente el entorno del backend y la base de datos de manera aislada. Fundamental para realizar pruebas de integración rápidas sin necesidad de ejecutar el frontend, asegurando que la API responda correctamente de forma independiente.

- **Dockerfile**

Contiene las instrucciones para generar la imagen del servidor. A diferencia del frontend, aquí usamos una sola etapa basada en Node 20 Alpine, donde instalamos las dependencias con `pnpm` y preparamos el entorno para ejecutar el proceso de Node de forma persistente. También se encarga de exponer el puerto de comunicación que usará la API.

- **Index.js**

Es el punto de arranque de nuestra aplicación Backend. Su función es muy específica: importa la configuración que preparamos en `app.js` y la constante del puerto (`ND_PORT`) desde el fichero de configuración. Al ejecutarse, utiliza el método `app.listen` para activar el servidor y, mediante un operador ternario, nos

muestra por consola un mensaje confirmando si el puerto está funcionando correctamente o si ha ocurrido algún fallo en el inicio del sistema.

- **Package.json**

Fichero de gestión del proyecto donde se definen los scripts de ejecución (como el uso de nodemon para desarrollo) y se listan las librerías necesarias para el servidor, como Express, controladores de base de datos y herramientas de seguridad como JWT.

- **Readme.md**

Manual técnico del backend. Incluye los pasos para configurar la base de datos, la estructura de la API y los pasos para configurar la aplicación y levantar el servidor desde cero.

- **Config**

Directorio que centraliza los parámetros globales del sistema, como la configuración de los secretos para el JWT o los parámetros de conexión externos, permitiendo modificar el comportamiento del servidor desde un solo lugar.

- **Controllers**

Directorio que contiene la lógica de control. Se encarga de recibir las peticiones de las rutas, validar la entrada del usuario a la acción que quiere realizar y coordinar la respuesta final, actuando como el cerebro que decide qué hacer con cada solicitud HTTP.

- **DB**

Directorio dedicado a la persistencia de datos. Contiene la lógica de conexión, el "pool" de clientes para PostgreSQL y los scripts SQL.

- **Database-final.sql**

Script de estructura DDL que contiene la creación de tablas, relaciones y procedimientos necesarios para inicializar la base de datos desde cero.

- **Middlewareares**

Directorio que contiene funciones intermedias que analizan las peticiones antes de que lleguen a los controladores finales. Se utilizan para tareas de protección, como verificar que el usuario esté logueado o validar que los datos enviados cumplen con el formato requerido.

- **Models**

Directorio donde se definen las entidades de la base de datos. Aquí se da transforman los datos crudos a objetos asegurando que la información que viaja por el sistema sea coherente y esté limpia.

- **Public**

Carpeta de acceso público del servidor. Se utiliza para almacenar recursos que el backend debe servir directamente.

- **Repository**

Directorio que funciona como capa de acceso a datos. Es el único lugar donde se escriben consultas SQL puras. Su objetivo es separar la base de datos del resto del código, para que, si en el futuro se cambia de base de datos, no haya que tocar la lógica de los controladores.

- **Routes**

Directorio que define los puntos de acceso (endpoints) de la API. Organiza las URLs del servidor (ej. /login, /upload) y las conecta con su respectivo controlador.

- **Services**

Directorio que contiene la lógica de negocio compleja y comunicaciones con servicios externos (como el envío de correos o integración con APIs de terceros). Sirve para mantener los controladores ligeros, delegando las tareas pesadas a estos servicios especializados.

- **Upload**

Espacio dedicado al almacenamiento de archivos subidos por los usuarios.

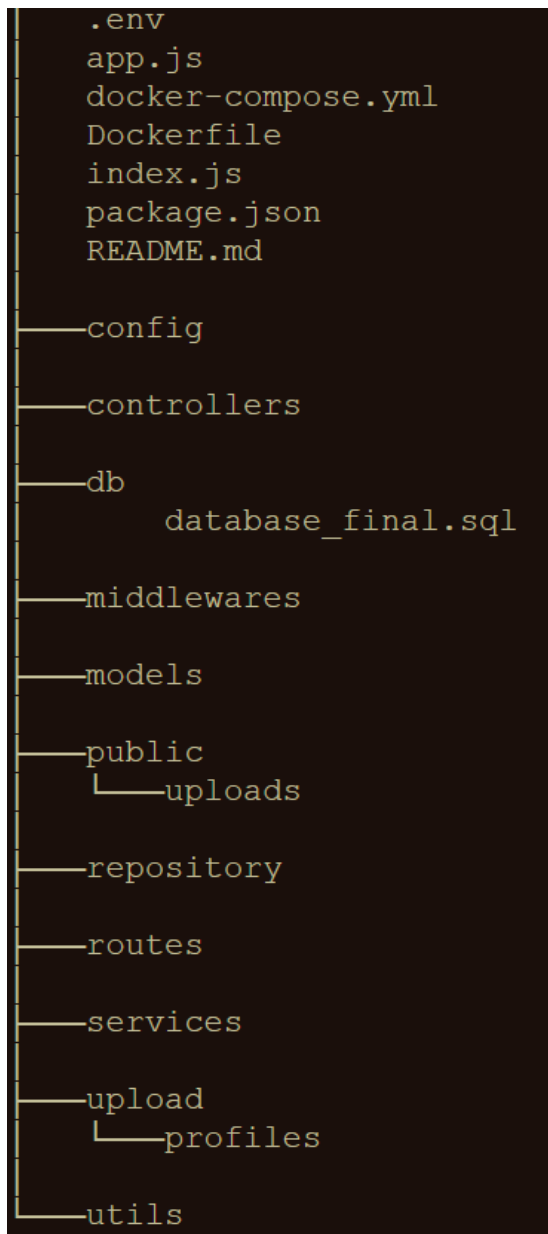
- **Profiles**

Subdirectorio específico donde el sistema organiza y guarda las imágenes de perfil o avatares de forma estructurada.

- **Utils**

Directorio de herramientas que contiene funciones que se repiten en todo el backend, como encriptadores de contraseñas, generadores de códigos aleatorios o formateadores de email para respuestas.

Ilustración 46 estructura de directorios Backend

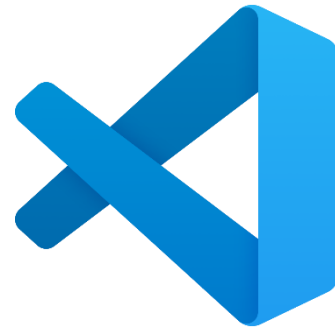


8.5.7 Herramientas utilizadas en el proyecto

8.5.7.1 Entorno de desarrollo y gestión de código

- **Visual Studio Code (VSC)**

Editor de código de código abierto desarrollado por Microsoft y uno de los más utilizados en la industria. Fue el editor principal a lo largo de todo el desarrollo gracias a su ecosistema de extensiones, su integración con Git y su soporte nativo para JavaScript y React.



- **GitHub**

Plataforma de alojamiento de repositorios Git propiedad de Microsoft. Se utilizó para el control de versiones del proyecto, permitiendo llevar un historial completo de cambios y mantener el código organizado en ramas de desarrollo.



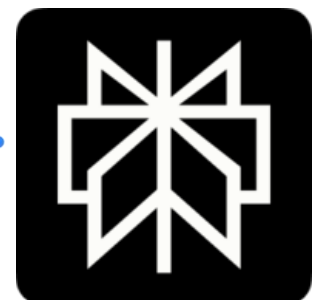
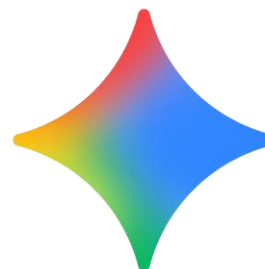
- **GitHub copilot**

Asistente de IA desarrollado por GitHub y OpenAI, integrado directamente en VSC. Se utilizó para agilizar tareas repetitivas como la creación de ficheros base o la generación de estructuras iniciales de código, reduciendo el tiempo dedicado a tareas repetitivas como la generación de boilerplate.



- **PerplexityAI y Gemini**

Herramientas de inteligencia artificial generativa utilizadas como apoyo durante el desarrollo para resolver dudas concretas, consultar documentación de forma rápida y buscar soluciones a problemas puntuales sin tener que navegar entre múltiples fuentes.



- **WSL (Windows Subsystem for Linux)**

Capa de compatibilidad desarrollada por Microsoft que permite ejecutar un entorno Linux directamente sobre Windows sin necesidad de una máquina virtual. Se utilizó para levantar la base de datos en local durante el desarrollo, replicando un entorno similar al del servidor de producción.



8.5.7.2 *Gestión de datos y bases de datos*

- **Dbeaver**

Cliente de base de datos universal y de código abierto compatible con la mayoría de motores SQL y NoSQL. Se utilizó durante la primera mitad del proyecto para gestionar, visualizar y consultar los datos de PostgreSQL gracias a su potente editor de consultas y su visualizador de esquemas.



- **TablePlus**

Cliente de base de datos minimalista con interfaz nativa disponible para macOS y Windows. Sustituyó a DBeaver a mitad del proyecto por ser una herramienta más ligera y ágil para las mismas tareas de gestión, ofreciendo una experiencia más fluida en operaciones del día a día.

8.5.7.3 *Pruebas, depuración y seguridad*

- **Bruno**

Cliente HTTP de código abierto y uso local, sin necesidad de cuenta ni sincronización en la nube. Se utilizó como herramienta principal para probar y depurar los endpoints de la API durante el desarrollo, almacenando las colecciones de peticiones directamente en el repositorio del proyecto.



- **Postman**

Una de las herramientas de prueba de APIs más extendidas en la industria. Se usó puntualmente en las fases iniciales del proyecto antes de migrar a Bruno como cliente principal.



- **ZAP**

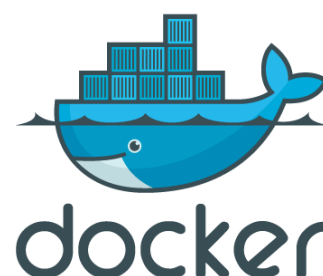
Herramienta de auditoría de seguridad de código abierto mantenida por la fundación OWASP, considerada un estándar en el análisis de vulnerabilidades web. Se utilizó para analizar la aplicación en busca de fallos de seguridad, categorizando los resultados por nivel de severidad: rojo para vulnerabilidades críticas o altas, amarillo o naranja para severidad media, y verde o azul para alertas informativas de baja severidad.



8.5.7.4 Infraestructura y despliegue (DevOps)

- **Docker**

Plataforma de código abierto para la creación y gestión de contenedores. Se utilizó para contenerizar el frontend y el backend de forma independiente, garantizando que el entorno de producción sea reproducible y que el despliegue en cualquier servidor sea consistente independientemente de su configuración base.



- **MobaXterm**

Ciente SSH y terminal remota para Windows que integra en una sola herramienta el acceso por SSH, la transferencia de ficheros y la gestión de sesiones. Se utilizó para conectarse a las dos VPS de Hetzner y ejecutar los comandos de configuración y despliegue del servidor.



- **Hostinger**

Proveedor de hosting y registro de dominios utilizado para gestionar el dominio DNS de la plataforma, configurando los registros necesarios para que el tráfico se dirija correctamente a los servidores de Hetzner a través de Cloudflare.



- **Cloudflare**

Plataforma de seguridad y rendimiento web que actúa como intermediario entre el dominio y el servidor. Se configuró como proxy inverso, ocultando la IP real del servidor, gestionando el tráfico entrante y añadiendo una capa adicional de protección frente a ataques externos.



- **Hetzner**

Proveedor de infraestructura cloud alemán conocido por ofrecer una buena relación entre coste y rendimiento cumpliendo con la normativa europea de protección de datos (RGPD). Aloja las dos VPS del proyecto: una para el servidor de aplicaciones y otra para la base de datos.



- **Google Analytics**

Servicio de analítica web de Google integrado en la aplicación mediante GA4.js. Permite monitorizar en tiempo real el tráfico, el comportamiento de los usuarios y las páginas más visitadas, proporcionando datos útiles para el mantenimiento y la evolución del proyecto.



- **Google Search Console**

Herramienta gratuita de Google que permite monitorizar la presencia de la web en los resultados de búsqueda. Se configuró para indexar la plataforma y hacer seguimiento de su posicionamiento orgánico. Actualmente se está trabajando en resolver un problema de indexación que dejó de rastrear la web.



8.5.7.5 Creación de maquetados

- **Adobe Photoshop**

Editor de imagen de Adobe utilizado junto a Illustrator para la creación del logotipo de la plataforma. La imagen se trabajó en Photoshop y posteriormente se convirtió a formato vectorial en Illustrator para garantizar su correcta visualización en cualquier resolución.



- **Adobe Illustrator**

Software de diseño vectorial de Adobe utilizado como herramienta principal para el prototipado digital de todas las vistas de la aplicación. Al trabajar con vectores, los diseños pueden escalarse a cualquier tamaño sin pérdida de calidad.



- **Concepts**

Aplicación de bocetos y diseño con soporte para vectores disponible en iPad y Windows. Se utilizó en las fases iniciales del proyecto para diseñar el mapa de navegación de la aplicación, definiendo las relaciones entre las distintas rutas antes de pasar al prototipado digital.



- **Draw.io (diagrams.net)**

Herramienta de diagramación online y de código abierto disponible tanto en navegador como en versión de escritorio. Se utilizó para diseñar los diagramas de la base de datos, desde el diagrama Entidad-Relación hasta el modelo lógico, gracias a su soporte nativo para notaciones de modelado de datos y su integración con GitHub para guardar los ficheros directamente en el repositorio.



- **Davinci Resolve**

Software profesional de edición de vídeo y color grading, utilizado para la postproducción de contenidos audiovisuales relacionados con el proyecto (video de funcionamiento y uso), gracias a su potente motor de edición y a sus herramientas de corrección de color avanzadas.



8.6 PLAN DE PRUEBAS

La seguridad se ha diseñado siguiendo el principio de "defensa en profundidad", combinando auditorías externas automatizadas con una implementación robusta en el ciclo de vida del desarrollo.

8.6.1 Análisis de Vulnerabilidades con OWASP ZAP

Se utilizó la herramienta de auditoría OWASP ZAP (Zed Attack Proxy) v2.17.0 para realizar un escaneo dinámico sobre el entorno de producción

(<https://demo.treedlink.com>). Este proceso incluyó tanto un rastreo pasivo como ataques activos para identificar debilidades en la configuración del servidor y las cabeceras HTTP. (ZAP, n.d.)

- **Resumen de Alertas**

El informe final arrojó 3 alertas de riesgo medio, 3 de riesgo bajo y 2 informativas.

Ilustración 47 resultado de la auditoria de ZAP

Excluded: Confirmed por Usuario, Alta, Media, Baja, Falso positivo

Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		Confirmado por Usuario	Alta	Media	Baja	Total
Risk	Alto	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)
	Medio	0 (0,0 %)	2 (25,0 %)	1 (12,5 %)	0 (0,0 %)	3 (37,5 %)
	Bajo	0 (0,0 %)	2 (25,0 %)	1 (12,5 %)	0 (0,0 %)	3 (37,5 %)
	Informativo	0 (0,0 %)	0 (0,0 %)	1 (12,5 %)	1 (12,5 %)	2 (25,0 %)
	Total	0 (0,0 %)	4 (50,0 %)	3 (37,5 %)	1 (12,5 %)	8 (100%)

Alert Counts by Site and Risk

- **Mitigación Destacada (CWE-525)**

Se identificaron oportunidades de mejora en la Directiva de Control de Caché

Implementando configuraciones en las cabeceras para evitar que información sensible del usuario sea almacenada en memorias caché compartidas o intermedias, cumpliendo con los estándares de gestión de sesiones de OWASP.

- **Seguridad en el Transporte**

El escaneo confirmó el uso correcto de HTTPS, garantizando que el tráfico entre el cliente y el servidor viaje cifrado mediante protocolos TLS.

8.6.2 Medidas de Seguridad Activa e Infraestructura

Más allá de las pruebas externas, el backend de la aplicación incorpora múltiples capas de protección:

- **Gestión de Identidad (JWT)**

Se implementó un sistema de autenticación basado en JSON Web Tokens. Para mitigar riesgos de robo de identidad, se separó la lógica en Access Tokens (de vida corta) y Refresh Tokens (almacenados de forma segura), permitiendo sesiones persistentes pero controladas.

- **Cifrado de Credenciales**

En cumplimiento con las mejores prácticas de seguridad, ninguna contraseña se almacena en texto plano. Se utiliza la librería Bcrypt para aplicar un hashing asíncrono con "sal" (salt), lo que protege los datos ante posibles filtraciones de la base de datos.

- **Arquitectura en el Backend (Middlewares)**

Se diseñaron middlewares de autorización en Express.js que interceptan cada petición a rutas críticas. Estos verifican la integridad del token y los permisos del usuario antes de permitir cualquier operación de lectura o escritura.

- **Integridad de Datos en PostgreSQL**

Para prevenir ataques de Inyección SQL, todas las interacciones con la base de datos se realizan mediante el uso de consultas parametrizadas dentro de la capa de repositorios, asegurando que los datos de entrada del usuario nunca se ejecuten como código.

- **Verificación de Cuentas**

Como medida para evitar el spam y registros malintencionados, se integró la API de Brevo para un flujo de confirmación de email, asegurando que cada perfil en la plataforma esté vinculado a una dirección de correo real.

8.6.3 Visualización de datos

En lugar de desarrollar un panel de estadísticas interno, se ha optado por integrar una solución profesional de Business Intelligence para la visualización y análisis de datos. Esto permite obtener métricas mucho más precisas y detalladas sin sobrecargar la base de datos del proyecto.

La visualización de datos se centraliza a través de GA4, permitiendo un seguimiento exacto del comportamiento de los usuarios en la plataforma:

- **Seguimiento de eventos**

Mediante el componente GA_4.jsx, la aplicación envía eventos personalizados a Google cada vez que un usuario realiza una acción clave (clics en enlaces, registros, navegación entre secciones).

- **Análisis de tráfico**

Permite visualizar en tiempo real el volumen de usuarios activos, su procedencia geográfica y los dispositivos utilizados.

- **Informes de comportamiento**

Se utilizan los paneles de exploración de Google para identificar qué partes de la aplicación (como el foro o los perfiles) generan mayor retención y cuáles presentan una mayor tasa de rebote.

8.7 DESPLIEGUE Y MANTENIMIENTO

El despliegue de la aplicación se realizó sobre las dos VPS contratadas en Hetzner. Mediante Docker se construyeron las imágenes de los servicios usando versiones fijas de Node, Nginx y PostgreSQL, evitando problemas de compatibilidad derivados de actualizaciones automáticas de paquetes o del sistema operativo base. Los datos generados por los usuarios, tanto los registros de la base de datos como las imágenes de perfil subidas, se mantienen persistentes mediante volúmenes de Docker, de forma que un reinicio o una recreación del contenedor no implica ninguna pérdida de información.

Las variables de entorno se crearon manualmente en cada servidor mediante ficheros .env, ya que al contener claves sensibles no pueden subirse al repositorio de GitHub. En el backend se gestionan mediante la dependencia dotenv, accediendo a ellas a través de process.env.NOMBRE_VARIABLE. En el frontend no es necesario instalar ninguna dependencia adicional ya que Vite lo incluye de forma nativa, con la única condición de que las variables deben comenzar con el prefijo VITE_ y se accede a ellas mediante import.meta.env.NOMBRE_VARIABLE.

8.7.1 Infraestructura de servidores

8.7.1.1 Separación de Servicio

La infraestructura se divide en dos servidores con responsabilidades bien diferenciadas. El primero aloja toda la lógica de la aplicación web, tanto el frontend como el backend, mientras que el segundo funciona exclusivamente como servidor de datos, dedicado a PostgreSQL sin ningún otro servicio corriendo sobre él.

Dentro del servidor de aplicaciones, Docker organiza el sistema en tres capas independientes. Esta separación tiene una ventaja operativa clara: si uno de los tres servicios falla, los otros dos siguen funcionando con normalidad, lo que facilita el diagnóstico de errores y evita que un problema puntual tumbe toda la aplicación.

8.7.1.2 Capa de Proxy y Seguridad

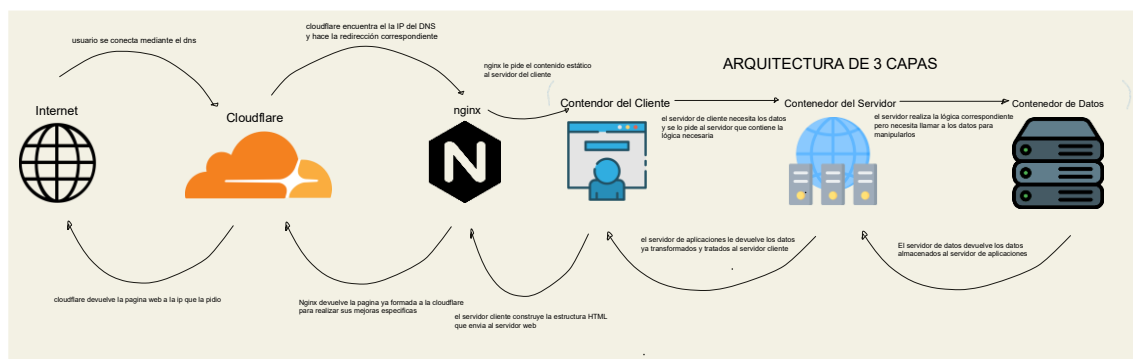
Por delante de ambos servidores se sitúa Cloudflare, que actúa como primera línea de defensa. Al enrutar todo el tráfico a través de su red, las IPs reales de las VPS quedan ocultas, lo que dificulta ataques directos a los servidores.

El cifrado se gestiona en dos tramos. Por un lado, Certbot genera y renueva automáticamente el certificado SSL del servidor, garantizando que la comunicación entre Cloudflare y la VPS esté cifrada de extremo a extremo. Por otro lado, en Cloudflare se configuró el modo SSL/TLS en Full (Strict), que es el nivel más alto disponible y exige que el certificado del servidor sea válido y esté firmado por una autoridad de confianza, rechazando cualquier conexión que no cumpla esa condición. La combinación de ambos garantiza que todo el tráfico, tanto entre el usuario y Cloudflare como entre Cloudflare y el servidor, viaje siempre cifrado.

8.7.2 Diagrama de flujo de peticiones

El diagrama muestra el recorrido completo de una petición desde que el usuario la genera hasta que recibe la respuesta. El tráfico entra primero a través de Cloudflare, que actúa como proxy inverso ocultando la IP real del servidor y aplicando sus filtros de seguridad, antes de redirigir la petición al servidor correspondiente para realizar sus mejoras específicas. Una vez procesada, Cloudflare devuelve la respuesta a la IP que realizó la solicitud.

Ilustración 48 diagrama del flujo de peticiones



8.7.3 Monitoreo y SEO

8.7.3.1 Monitorización de errores

El mantenimiento del sistema se apoya en la revisión periódica de los logs generados por Docker. Al tener los servicios separados en contenedores independientes, es posible consultar el estado de cada uno por separado e identificar errores concretos sin tener que revisar todo el sistema a la vez. Esta revisión permite detectar caídas, comportamientos inesperados o errores de conexión entre servicios antes de que afecten a los usuarios.

8.7.3.2 Optimización SEO y rastreo

El posicionamiento de la plataforma se monitoriza a través de Google Search Console, que permite ver qué páginas están indexadas, con qué frecuencia las rastrea Google y qué errores encuentra durante ese proceso. Actualmente se están analizando los errores de rastreo detectados para asegurar que todos los perfiles de usuario y las entradas del foro sean correctamente visibles en los resultados de búsqueda, ya que al tratarse de contenido generado dinámicamente requiere una configuración más específica para que los buscadores lo indexen de forma correcta.

9 CONCLUSIÓN

9.1 CONCLUSIÓN GENERAL DEL PROYECTO

El proyecto supuso un desafío real desde el principio, ya que implicó trabajar con una lógica de negocio mucho más compleja que la de los ejercicios habituales. Antes de empezar a desarrollar fue necesario investigar y aprender tecnologías nuevas para decidir cuáles se adaptaban mejor al proyecto, tanto en el backend como en el frontend, lo que añadió una carga de trabajo extra que no siempre es fácil de gestionar en paralelo al desarrollo.

La principal motivación fue ir más allá de lo estándar: no replicar una aplicación existente dentro de límites predefinidos, sino combinar funcionalidades que habitualmente se ofrecen por separado en plataformas distintas. El resultado final es satisfactorio, aunque queda claro que hay muchas mejoras posibles que se detallan en el apartado de visión a futuro, especialmente en lo que respecta a una evolución hacia un modelo más empresarial que permita monetizar sin sacrificar lo más importante, que es la experiencia del usuario.

9.2 CONCLUSIÓN DE OBJETIVOS

Al inicio del proyecto se plantearon más objetivos de los que finalmente se pudieron completar. Algunos se descartaron por falta de tiempo y otros por la decisión de migrar la arquitectura hacia una solución más robusta, lo que consumió tiempo que estaba previsto para otras funcionalidades.

Entre los objetivos que quedaron pendientes destacan la creación de un sistema de analíticas propio, que finalmente se delegó en Google Analytics 4 por ser una solución más rápida de integrar, y la implementación de un sistema de posts con un entorno de publicación editable. A pesar de eso, los objetivos que sí se completaron tienen una carga técnica interesante: autenticación con tokens, despliegue real en producción, auditoría de seguridad con ZAP, SEO dinámico por perfil y una sección de foros que ninguna de las plataformas analizadas en la comparativa incluye.

9.3 VALORACIÓN DE METODOLOGÍA Y PLANIFICACIÓN

La arquitectura y metodología utilizadas en el proyecto funcionan bien para la escala actual. La separación por capas y el patrón CSR permiten localizar cualquier parte del código de forma rápida y mantener una distinción clara de responsabilidades, lo que resulta muy eficaz para proyectos de tamaño pequeño o medio.

Sin embargo, de cara a un proyecto futuro de mayor escala la arquitectura cambiaría. La siguiente evolución natural sería pasar a microservicios combinados con Clean Architecture, que es la aproximación más adecuada para proyectos grandes donde la organización y la escalabilidad son críticas desde el principio.

9.4 VISIÓN A FUTURO

YourTree está actualmente desplegado en demo.treedlink.com como versión de demostración, y hay varias líneas de mejora planificadas para una versión más completa de la plataforma.

La primera es la migración del backend a NestJS, un framework más estructurado con soporte nativo para TypeScript que facilitaría escalar el proyecto y mantener una organización más estricta del código.

La segunda es la incorporación de funciones de pago que complementen la versión gratuita actual sin reemplazarla, manteniendo el acceso básico para todos los usuarios y añadiendo funcionalidades premium por encima de lo que ya existe.

La tercera es el desarrollo de una sección de posts integrada con la funcionalidad actual de la plataforma, ampliando las posibilidades de creación de contenido más allá de los enlaces y los foros.

La cuarta es implementar un sistema de analíticas nativo que permita a cada usuario consultar estadísticas de su propio perfil directamente desde la plataforma, sin depender de herramientas externas como Google Analytics.

Por último, en el frontend se trabajaría en mejorar la UX/UI y en migrar hacia la arquitectura MVVM combinada con la arquitectura basada en componentes que ya se usa actualmente, consiguiendo un proyecto más organizado, predecible y escalable a largo plazo.

10 REFERENCIAS

Microsoft Azure. (s.f.). *¿Qué es Java Spring Boot? Introducción a Spring Boot | Microsoft Azure*. Obtenido de Microsoft Azure: <https://azure.microsoft.com/es-mx/resources/cloud-computing-dictionary/what-is-java-spring-boot/>

AlexHost. (12 de junio de 2024). *Ventajas y Desventajas de MVC | AlexHost*. Obtenido de AlexHost: <https://alexhost.com/es/faq/what-is-mvc-advantages-and-disadvantages-of-mvc/>

andreaswilli. (s.f.). *react-verification-input - npm*. Obtenido de npm: <https://www.npmjs.com/package/react-verification-input>

Angel, V. M. (20 de febrero de 2024). *Arquitectura de Componentes. La arquitectura de componentes es un... | by Vanessa Marely Aristizabal Angel | Medium*. Obtenido de Medium: <https://vanessamarely.medium.com/arquitectura-de-componentes-e48816925d51>

Angular. (s.f.). *What is Angular? • Angular*. Obtenido de Angular: <https://angular.dev/overview>

AprendiendoDEV, Carlos Azaustre. (2023). *Cómo Consumir APIs en REACT como un PROFESIONAL*. Obtenido de Youtube: https://www.youtube.com/watch?v=6u1RHUoXIPi&list=PLwM_P5aKq8oy2aKslSPYXRpLDU_RcsPcE&index=5

Asana. (2026). *Qué es la metodología waterfall y cuándo utilizarla [2026]* • Asana. Obtenido de Asana: <https://asana.com/es/resources/waterfall-project-management-methodology>

Astro. (s.f.). *Islands architecture* | Docs. Obtenido de Astro: <https://docs.astro.build/en/concepts/islands/>

Astro. (s.f.). *Why Astro?* | Docs. Obtenido de Astro: <https://docs.astro.build/en/concepts/why-astro/>

Atlassian. (2015). *What is Kanban in Project Management?* Obtenido de Atlassian: <https://www.atlassian.com/agile/kanban>

Atlassian. (21 de mayo de 2024). *¿Qué es "scrum"? Guía para el marco de trabajo ágil.* Obtenido de Atlassian: <https://www.atlassian.com/es/agile/scrum>

Atlassian. (2026). *Diferencias entre kanban y scrum* | Atlassian. Obtenido de Atlassian: <https://www.atlassian.com/es/agile/kanban/kanban-vs-scrum>

Auth0. (s.f.). *jsonwebtoken* - npm. Obtenido de npm: <https://www.npmjs.com/package/jsonwebtoken>

Beacons. (s.f.). *Creator Management Tools for Talent Managers* | Beacons. Obtenido de Beacons: <https://beacons.ai/i/managers>

Brianc. (s.f.). *pg* - npm. Obtenido de npm: <https://www.npmjs.com/package/pg>

Campsite. (s.f.). *Campsite.bio Pricing and Features - Free or just \$7 a month.* Obtenido de Campsite: <https://campsite.bio/pricing>

CampusMVP. (2022). *Desventajas de Spring y Spring Boot - GraalVM y la API Java Reflection.* Obtenido de Youtube: https://www.youtube.com/watch?v=_GPx-RMg6QU&t=30s

campusMVP.es. (2022). *Desventajas de Spring y Spring Boot - GraalVM y la API Java Reflection.* Obtenido de campusMVP.es: https://www.youtube.com/watch?v=_GPx-RMg6QU&t=30s

Carlos, L. (s.f.). *Introducción a OWASP ZAP - Laprovittera Carlos.* Obtenido de laprovittera: <https://laprovittera.com/introduccion-a-owasp-zap/>

Code, Atomic. (2022). *Middleware explained - Node.JS.* Obtenido de Youtube: https://www.youtube.com/watch?v=YIOSYmDdZ5w&list=PLwM_P5aKq8oy2aKsISPYXRpLDU_RcsPcE&index=6

Codler. (s.f.). *react-ga4 - npm*. Obtenido de npm: <https://www.npmjs.com/package/react-ga4>

Creately. (s.f.). *Tutorial de diagramas de casos de uso (Guía con ejemplos)*. Obtenido de Creately: <https://creately.com/blog/es/diagramas/tutorial-diagrama-caso-de-uso/>

ctrl-y. (17 de marzo de 2021). *Arquitectura de capas para NodeJs | ctrl-y*. Obtenido de ctrl-y: <https://ctrly.blog/es/arquitectura-capas/>

dongee. (09 de octubre de 2024). *Ventajas y Desventajas de Laravel*. Obtenido de dongee: <https://www.dongee.com/tutoriales/ventajas-y-desventajas-de-laravel/>

Duian. (s.f.). *js-cookies - npm*. Obtenido de npm: <https://www.npmjs.com/package/js-cookies>

EDteam. (2023). *¿Qué es React y por qué domina el desarrollo frontend?* Obtenido de Youtube:
https://www.youtube.com/watch?v=kEMr0rFjB7E&list=PLwM_P5aKq8oy2aKslSPYXRpLDU_RcsPcE&index=3

eligeunaweb. (04 de agosto de 2025). *Cómo usar OWASP ZAP para auditar vulnerabilidades en un ecommerce | Eligeunaweb Agencia de marketing y programación web*. Obtenido de eligeunaweb: <https://eligeunaweb.es/como-usar-owasp-zap-para-auditar-vulnerabilidades-en-un-ecommerce/>

Express.js. (s.f.). *Express - Node.js web application framework*. Obtenido de ExpressJs: <https://expressjs.com>

Expressjs. (s.f.). *express - npm*. Obtenido de npm: <https://www.npmjs.com/package/express>

Expressjs. (s.f.). *multer - npm*. Obtenido de npm: <https://www.npmjs.com/package/multer>

Expressjs. (s.f.). *npm - cors*. Obtenido de npm: <https://www.npmjs.com/package/cors>

Expressjs. (s.f.). *Utilización del middleware de Express*. Obtenido de Expressjs: <https://expressjs.com/es/guide/using-middleware.html>

Fazt. (2023). *Express Framework de Nodejs, Curso para principiantes (Javascript en el backend)*. Obtenido de Youtube:

https://www.youtube.com/watch?v=JmJ1WUoUIK4&list=PLwM_P5aKq8oy2aKsISPYXRpLDU_RcsPcE&index=2

Fazt. (2023). *Nodejs Curso Práctico desde Cero (Javascript en el Backend)*. Obtenido de Youtube:
https://www.youtube.com/watch?v=i3OdKwuBjeM&list=PLwM_P5aKq8oy2aKsISPYXRpLDU_RcsPcE

Fowler, M. (2002). Patterns of Enterprise Application Architecture. En M. Fowler, *Patterns of Enterprise Application Architecture* (págs. 184-189). Addison-Wesley.

Gesualdo, Martín. (2024). *Callbacks, Promesas y Async Await en Javascript*. Obtenido de Youtube:
https://www.youtube.com/watch?v=a91rU4Jk1KU&list=PLwM_P5aKq8oy2aKsISPYXRpLDU_RcsPcE&index=7

GoDaddy. (26 de junio de 2025). *Metodología XP: qué es y cómo aplicarla en desarrollo ágil*. Obtenido de GoDaddy:
<https://www.godaddy.com/resources/latam/tecnologia/metodologia-xp-extreme-programming-que-es>

HACK A BOSS. (s.f.). *Cómo estructurar un proyecto Spring Boot desde cero | HACK A BOSS*. Obtenido de HACK A BOSS: <https://www.hackaboss.com/blog/como-estructurar-un-proyecto-spring-boot-desde-cero>

Hetzner. (s.f.). *Cloud-hosting provider for developers & teams*. Obtenido de Hetzner:
<https://www.hetzner.com/cloud>

IEBS Business School. (2025). *Diferencias SCRUM vs KANBAN*. Obtenido de Youtube:
<https://www.youtube.com/watch?v=Z5T0EI9M3kc&t=56s>

IEBSchool. (28 de abril de 2021). *Descubre qué es el Extreme Programming y sus características | IEBS Business School*. Obtenido de IEBSchool:
<https://www.iebschool.com/hub/que-es-el-xp-programming-agile-scrum/>

Indeed. (24 de junio de 2022). *Ventajas y desventajas de la metodología Scrum | indeed.com España*. Obtenido de Indeed: <https://es.indeed.com/orientacion-laboral/desarrollo-profesional/ventajas-desventajas-metodologia-scrum>

Inesem. (s.f.). *Los gestores de bases de datos (SGBD) más usados*. Obtenido de Inesem: <https://www.inesem.es/revistadigital/informatica-y-tics/los-gestores-de-bases-de-datos-mas-usados>

InMotion Hosting. (22 de agosto de 2025). *PHP vs Laravel: la guía completa para la toma de decisiones empresariales* | InMotion Hosting. Obtenido de InMotion Hosting: <https://www.inmotionhosting.com/blog/es/php-vs-laravel/>

JetBrains. (s.f.). *¿Cuál es el mejor marco de trabajo web de Python: Django, Flask o FastAPI?* | *The PyCharm Blog*. Obtenido de JetBrains: <https://blog.jetbrains.com/es/pycharm/2025/06/cual-es-el-mejor-marco-de-trabajo-web-de-python-django-flask-o-fastapi/>

Jonmircha. (2020). *Curso Diseño UX / UI (Videos 1-16)*. Obtenido de YouTube: https://www.youtube.com/watch?v=4WluMTM2oss&list=PLvq-jlkSeTUYzH3wNAN3wz_kM69CZWcuP

Kelektiv. (s.f.). *bcrypt - npm*. Obtenido de npm: <https://www.npmjs.com/package/bcrypt>

KevDev. (2024). *Conectando React y Node.js: Guía paso a paso para la comunicación entre Frontend y Backend*. Obtenido de Youtube: https://www.youtube.com/watch?v=qYGLiU-27TY&list=PLwM_P5aKq8oy2aKsISPYXRpLDU_RcsPcE&index=4

Kinsta. (17 de enero de 2025). *Svelte vs React: Características, Rendimiento y Más*. Obtenido de Kinsta: <https://kinsta.com/es/blog/svelte-vs-react/>

Linktree . (s.f.). *Linktree Free, Starter, Pro and Premium Pricing*. Obtenido de Linktree : <https://linktr.ee/s/pricing>

Lucidchart. (s.f.). *Tutorial de diagrama de clases UML* | *Lucidchart*. Obtenido de Lucidchart: <https://www.lucidchart.com/pages/es/tutorial-de-diagrama-de-clases-uml>

Lucidchart. (s.f.). *Tutorial de diagrama de secuencia UML* | *Lucidchart*. Obtenido de Lucidchart: <https://www.lucidchart.com/pages/es/diagrama-de-secuencia>

Material UI. (s.f.). *Installation - Material UI*. Obtenido de Material UI: <https://mui.com/material-ui/getting-started/installation/>

MDN. (27 de marzo de 2025). *Express Web Framework (Node.js/JavaScript) - Aprende desarrollo web* | *MDN*. Obtenido de MDN: https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs

MDN. (13 de octubre de 2025). *Introduction to client-side frameworks - Learn web development* | MDN. Obtenido de MDN: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction

MDN. (10 de abril de 2026). *Primeros pasos en React - Aprende desarrollo web* | MDN. Obtenido de MDN: https://developer.mozilla.org/es/docs/Learn_web_development/Core/Frameworks_libraries/React_getting_started

Meta. (2024). *Server Components – React*. Obtenido de React: <https://es.react.dev/reference/rsc/server-components>

Meta. (s.f.). *Component – React*. Obtenido de React: <https://es.react.dev/reference/react/Component>

Meta. (s.f.). *createContext – React*. Obtenido de React: <https://es.react.dev/reference/react/createContext#provider>

Microsoft . (s.f.). *Definir su aplicación de varios contenedores con docker-compose.yml - .NET* | Microsoft Learn. Obtenido de Microsoft Learn: <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/multi-container-microservice-net-applications/multi-container-applications-docker-compose>

midudev. (2025). *Curso Tailwind CSS 4 desde cero*. Obtenido de Youtube: <https://www.youtube.com/watch?v=R5EXap3vNDA&t=2487s>

midulive. (2024). *CURSO DE NODE.JS DESDE CERO: Introducción y primeros pasos*. Obtenido de Youtube: https://www.youtube.com/watch?v=yB4n_K7dZV8&list=PLUofhDIg_38qm2oPOV-IRTTEKyrVBBaU7&index=2

midulive. (2025). *Aprende Autenticación de Usuario, Sesión, Cookies y JWT con Node.js*. Obtenido de Youtube: https://www.youtube.com/watch?v=UqnnhAZxRac&list=PLwM_P5aKq8oy2aKslSPYXRpLDU_RcsPcE&index=10

Milkshake. (s.f.). *Precios — Creador de Sitios Web Gratis — Milkshake*. Obtenido de milkshake: <https://milkshake.app/es/pricing/>

Motdotla. (s.f.). *dotenv - npm*. Obtenido de npm: <https://www.npmjs.com/package/dotenv>

mytaskpanel. (s.f.). *Qué es NestJS y cuándo usarlo: características, ventajas y desventajas*. Obtenido de mytaskpanel: <https://www.mytaskpanel.com/que-es-nestjs-y-cuando-usarlo/>

NestJS . (s.f.). *Documentation | NestJS - A progressive Node.js framework*. Obtenido de NestJS : <https://docs.nestjs.com>

Nodemailer. (s.f.). *nodemailer - npm*. Obtenido de npm: <https://www.npmjs.com/package/nodemailer>

OBS Business School. (15 de noviembre de 2024). *Pros y contras de la metodología en cascada | OBS Business School*. Obtenido de OBS Business School: <https://www.obsbusiness.school/blog/pros-y-contras-de-la-metodologia-en-cascada>

OpenWebinars. (25 de marzo de 2025). *React desde cero: Todo lo que debes saber | OpenWebinars*. Obtenido de OpenWebinars: <https://openwebinars.net/blog/react-desde-cero-todo-lo-que-debes-saber/>

Remix-run. (s.f.). *react-router-dom - npm*. Obtenido de npm: <https://www.npmjs.com/package/react-router-dom>

Remy. (s.f.). *nodemon - npm*. Obtenido de npm: <https://www.npmjs.com/package/nodemon>

Staylor. (s.f.). *react-helmet-async - npm*. Obtenido de npm: <https://www.npmjs.com/package/react-helmet-async>

Svelte . (s.f.). *Overview • Svelte Docs*. Obtenido de Svelte : <https://svelte.dev/docs/svelte/overview>

The Blessing Aniedi. (s.f.). *Cómo usar la aplicación MILKSHAKE para tus redes sociales | TikTok*. Obtenido de tiktok: <https://www.tiktok.com/@theblessinghollyfield/video/7337775848222625057>

TodoPostgreSQL. (30 de agosto de 2018). *Ventajas y Desventajas de PostgreSQL - TodoPostgreSQL*. Obtenido de TodoPostgreSQL: <https://www.todopostgresql.com/ventajas-y-desventajas-de-postgresql/>

ValentinH. (s.f.). *react-easy-crop - npm*. Obtenido de npm: <https://www.npmjs.com/package/react-easy-crop>

Vida MRR - Programación web. (2024). *CURSO DE LOGIN y REGISTRO COMPLETO CON REACT*. Obtenido de Youtube:
https://www.youtube.com/watch?v=q4ywr3eZmk0&list=PLwM_P5aKq8oy2aKslSPYXRpLDU_RcsPcE&index=9

Vida MRR - Programación web. (2024). *CURSO DE LOGIN y REGISTRO COMPLETO CON REACT*. Obtenido de Youtube:
https://www.youtube.com/watch?v=q4ywr3eZmk0&list=PLwM_P5aKq8oy2aKslSPYXRpLDU_RcsPcE&index=9

Vite. (s.f.). *Getting Started | Vite*. Obtenido de Vite: <https://vite.dev/guide/>

Vue.js. (s.f.). *Vue.js - The Progressive JavaScript Framework | Vue.js*. Obtenido de Vue.js: <https://vuejs.org>

Wikipedia. (7 de abril de 2026). *Singleton pattern - Wikipedia*. Obtenido de Wikipedia:
https://en.wikipedia.org/wiki/Singleton_pattern

ZAP. (s.f.). *ZAP – Documentation*. Obtenido de ZAP: <https://www.zaproxy.org/docs/>

11 ANEXOS

11.1 PLAN DE EMPRESA FUTURO

Este es el enlace para visualizar el archivo pdf, el cual se encuentra alojado en GitHub.

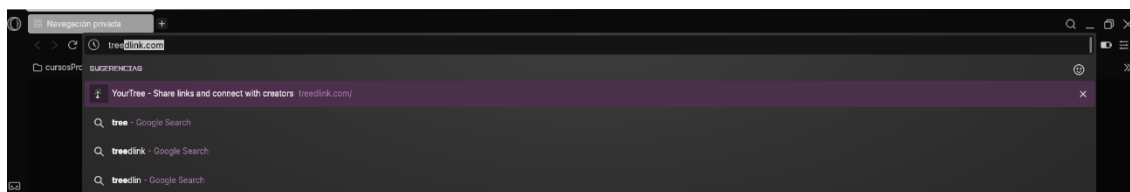
<https://github.com/nicowooow/Proyecto-FP/blob/main/documentos/Plan%20de%20empresa.pdf>

11.2 MANUAL DE USUARIO

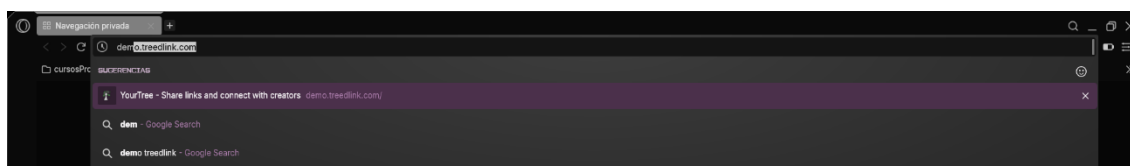
11.2.1 Ingreso a la página web

Para ingresar a la pagina web se tiene 3 maneras

- Ingresar mediante <https://treedlink.com/>, que hace una redirección al subdominio <https://demo.treedlink.com/>.



- Ingresar de forma directa mediante <https://demo.treedlink.com/>.



- Ingresar mediante la búsqueda indexada buscando treedlink en el navegador, actualmente no se puede realizar mediante esta opción por motivos de la indexación de Google.

11.2.2 Secciones que se repiten

11.2.2.1 Header

En el Header se puede encontrar los siguientes componentes:

- **Logo:** Se encuentra el logo de la página web, la cual hace una redirección a la página base.

- **Barra de búsqueda:** El usuario debe introducir el nombre del foro o nombre de usuario que desea encontrar.
- **Home:** Página de bienvenida al ingresar en la URL base.
- **Recent Pages:** Página de los perfiles que crearon al menos un enlace publico que desean compartir.
- **Forums:** Página donde se mostrarán los foros que fueron creados por los usuarios.
- **YourTree:** Página que requiere un inicio de sesión previo, en el cual se podrá modificar como se verá el perfil público.
- **Sign in:** Página en donde el usuario puede iniciar sesión con sus respectivas credenciales con un registro previo.
- **Sign up:** Página en donde el usuario se debe registrar para poder acceder a ciertas acciones que requieren una sesión activa.
- **@username:** Página de perfil privado del usuario, en donde puede cambiar el correo de verificación, tema del usuario para uso remoto, cambio de biografía y foto de perfil.
- **Toggle:** Apartado donde el usuario puede cambiar el tema actual de la página web (claro - oscuro), solo se almacenará de forma local.

11.2.2.2 Buy me a coffee

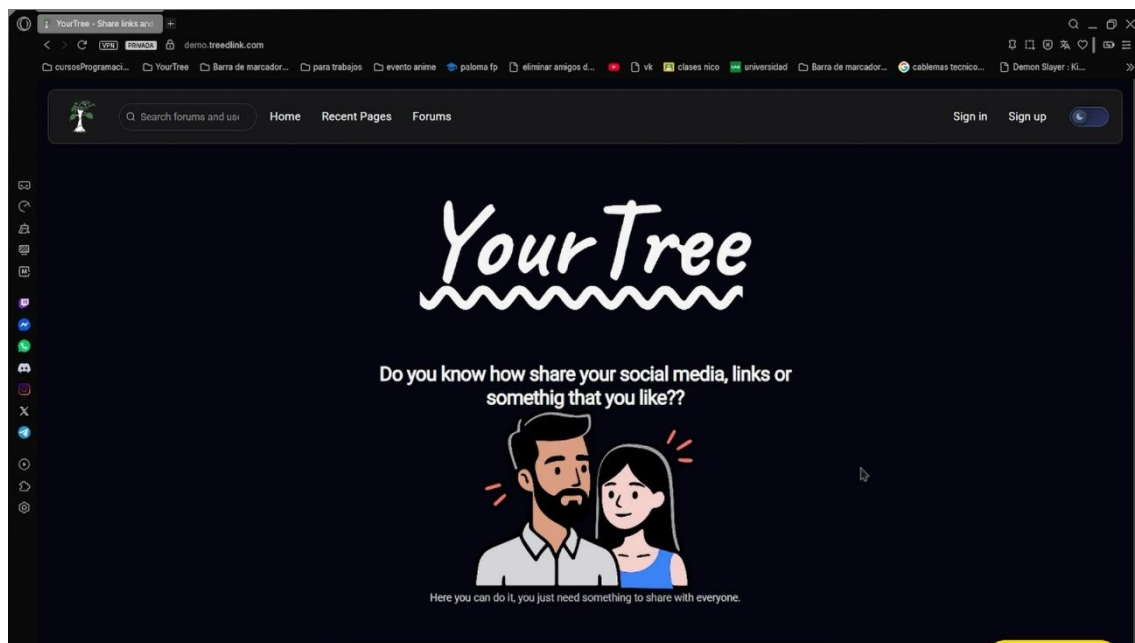
Un componente estático el cual hace una redirección a un enlace para enviar contribuciones una contribución monetaria

11.2.3 Páginas y uso respectivo

11.2.3.1 Página base o Home

Página de aterrizaje (Landing Page) diseñada para captar la atención del usuario mediante un "Hero Section" visualmente atractivo y un encabezado funcional. Presenta la propuesta de valor de YourTree, permitiendo una navegación rápida hacia las opciones de registro o visualización de perfiles destacados.

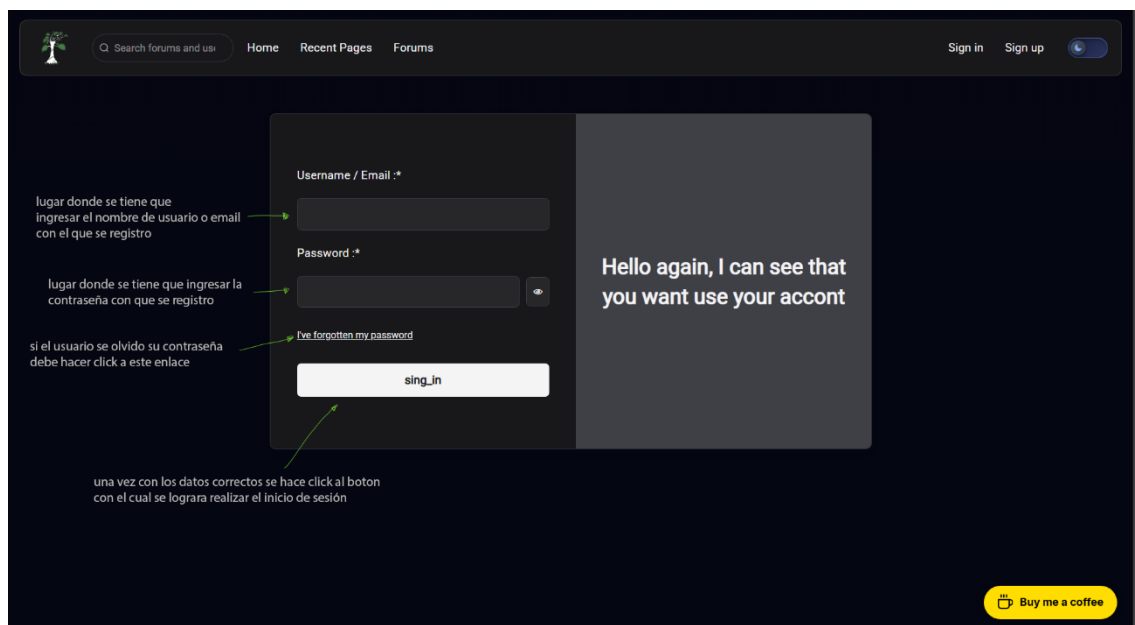
Ilustración 49 manual de usuario (home)



11.2.3.2 Sign in

Interfaz de acceso donde los usuarios registrados introducen sus credenciales (email o nombre de usuario y contraseña). El sistema valida los datos contra la base de datos para generar un token de sesión seguro, permitiendo al usuario acceder a sus herramientas de edición personalizadas.

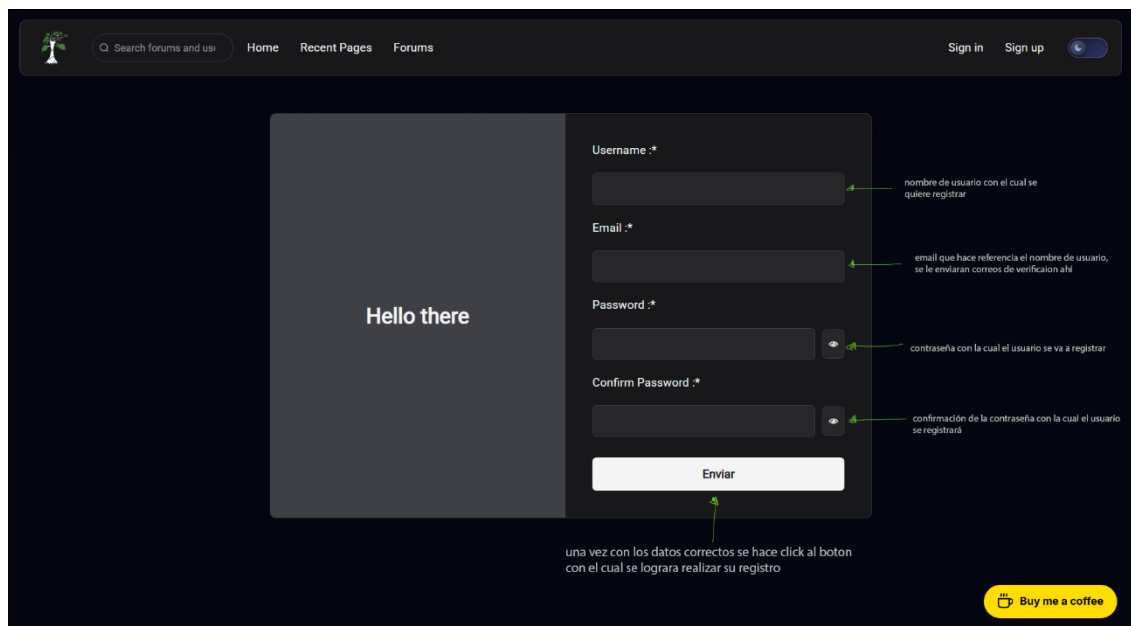
Ilustración 50 manual de usuario (Sign in)



11.2.3.3 Sign up

Formulario de registro destinado a nuevos usuarios. Solicita los datos esenciales para crear una identidad única en la plataforma, como el nombre de usuario (que servirá de URL personalizada) y el correo electrónico. Incluye validaciones en tiempo real para asegurar que el nombre elegido no esté duplicado en el sistema.

Ilustración 51 manual de usuario (Sign up)



The screenshot shows a web interface for signing up. On the left, a dark grey box contains the text "Hello there". On the right, a registration form is displayed with the following fields and labels:

- Username :***: A text input field. Annotation: "nombre de usuario con el cual se quiere registrar".
- Email :***: A text input field. Annotation: "email que hace referencia el nombre de usuario, se le enviaran correos de verificación ahí".
- Password :***: A text input field with a toggle for visibility. Annotation: "contraseña con la cual el usuario se va a registrar".
- Confirm Password :***: A text input field with a toggle for visibility. Annotation: "confirmación de la contraseña con la cual el usuario se registrará".
- Enviar**: A button at the bottom of the form. Annotation: "una vez con los datos correctos se hace click al boton con el cual se lograra realizar su registro".

At the top of the interface, there is a navigation bar with a search bar, links for "Home", "Recent Pages", and "Forums", and buttons for "Sign in" and "Sign up". A "Buy me a coffee" button is located in the bottom right corner.

11.2.3.4 YourTree

Este es el "dashboard" o panel de control privado del usuario. Requiere autenticación activa y permite la gestión integral del perfil público: subida y cambio de imagen de perfil, edición de la biografía y la creación/edición de los enlaces sociales que aparecerán en su página personal.

11.2.3.4.1 Interfaz de Gestión y Configuración

Esta sección constituye el núcleo del perfil público del usuario, permitiendo la personalización en tiempo real de su identidad digital. La interfaz se divide en dos bloques principales de configuración:

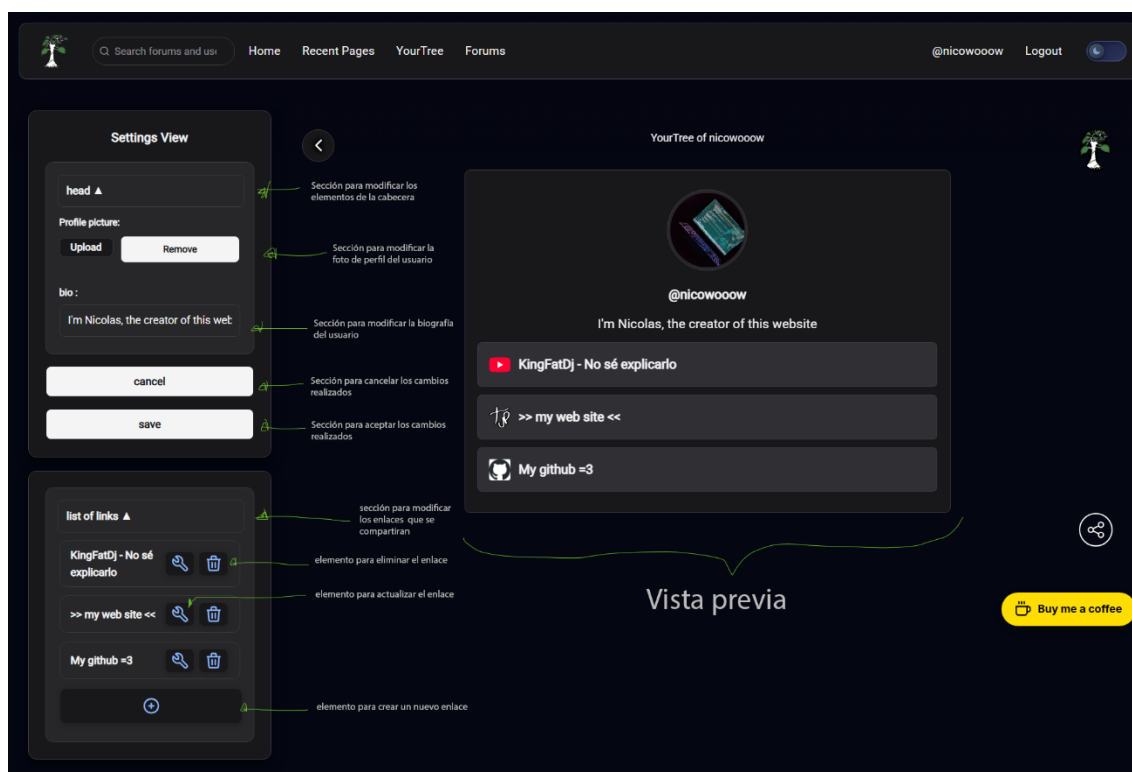
- **Configuración de Cabecera (Head)**
 - **Gestión de Imagen de Perfil:** Permite al usuario subir una nueva fotografía (Upload) o eliminar la actual (Remove) para mantener su identidad actualizada.

- **Edición de Biografía (bio):** Un campo de texto destinado a la presentación personal, permitiendo una descripción breve que se reflejará directamente en el perfil público.
- **Controles de persistencia:** Los botones Save y Cancel gestionan el envío de los cambios al servidor o el descarte de las modificaciones respectivamente.
- **Gestión de Enlaces (List of links)**
 - **Listado Dinámico:** Muestra todos los enlaces sociales activos del usuario.
 - **Acciones de CRUD:** Cada enlace dispone de iconos intuitivos para la edición (actualizar URL o título) y eliminación (borrado del registro).
 - **Adición de contenido:** Mediante el botón con el icono de suma (+), el usuario puede integrar nuevos elementos a su lista de enlaces de forma rápida.

11.2.3.4.2 Módulo de Vista Previa (Preview)

A la derecha del panel de edición, se ha implementado un componente de Vista Previa dinámica. Este módulo cumple una función crítica de experiencia de usuario (UX), ya que permite al propietario visualizar exactamente cómo se mostrará su "YourTree" a terceros antes de finalizar las ediciones. Refleja instantáneamente los cambios en la imagen, el nombre de usuario (ej. @nicowooow), la biografía y la jerarquía de los botones de enlace.

Ilustración 52 manual de usuario (YourTree)



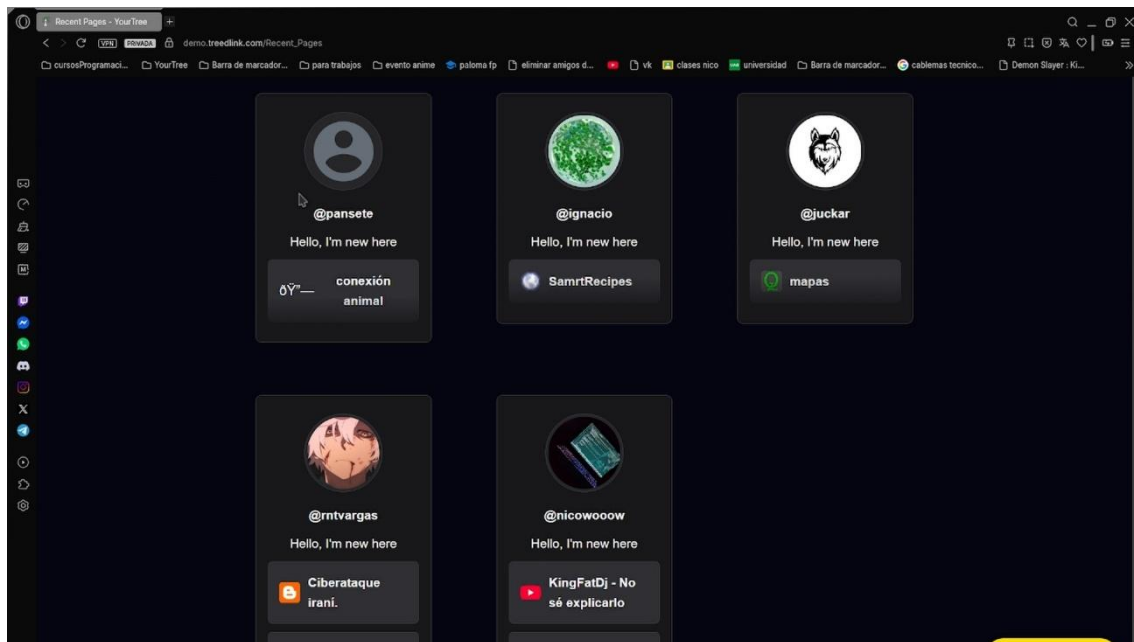
11.2.3.5 Recent Pages

Sección de descubrimiento que actúa como un feed dinámico. Muestra los últimos perfiles públicos creados por la comunidad, fomentando el tráfico entre usuarios. Funciona como un escaparate de la actividad reciente de la plataforma, permitiendo explorar cómo otros usuarios han configurado sus árboles de enlaces.

11.2.3.5.1 Sin perfil público

Estado de la página cuando un usuario nuevo aún no ha configurado sus enlaces o biografía. Muestra un diseño base con el nombre de usuario (ej. @tryacc) a la espera de contenido.

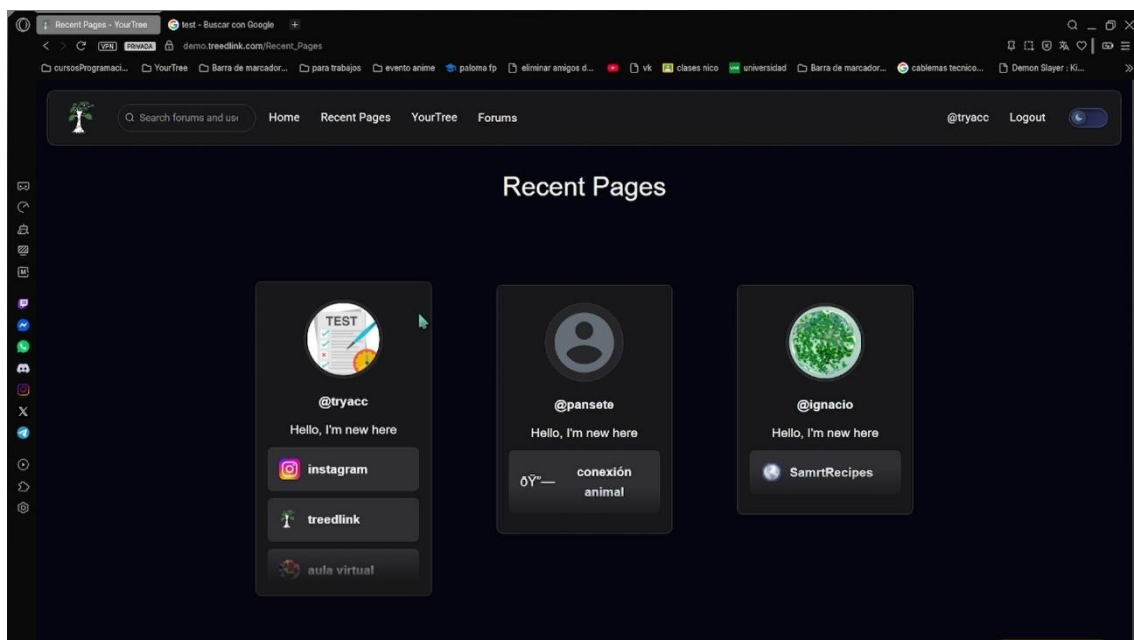
Ilustración 53 manual de usuario (Recent page sin perfil publico)



11.2.3.5.2 Con perfil público

Estado final de la página una vez que el usuario ha poblado su perfil con enlaces y contenido multimedia, mostrando la estructura completa y funcional del "Link-in-bio".

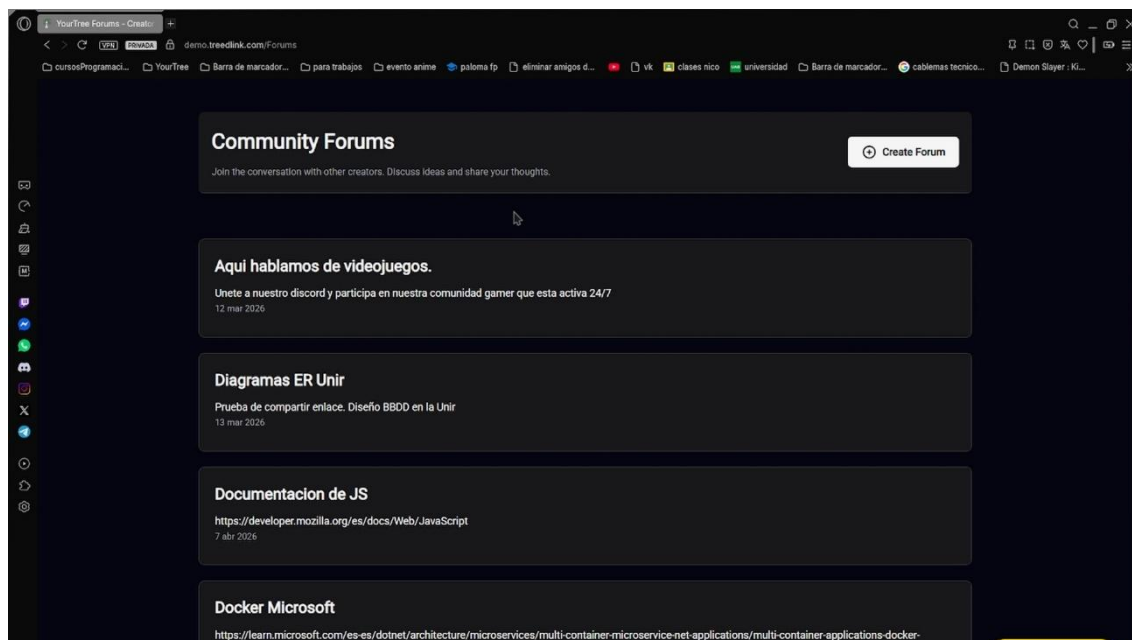
Ilustración 54 manual de usuario (Recent page con perfil publico)



11.2.3.6 Forums

Plataforma de interacción comunitaria donde los usuarios pueden debatir, compartir consejos sobre SEO y personalización, o solicitar ayuda técnica. Está estructurada en hilos de discusión que permiten organizar el conocimiento colectivo de los creadores de **YourTree**.

Ilustración 55 manual de usuario (Forum)

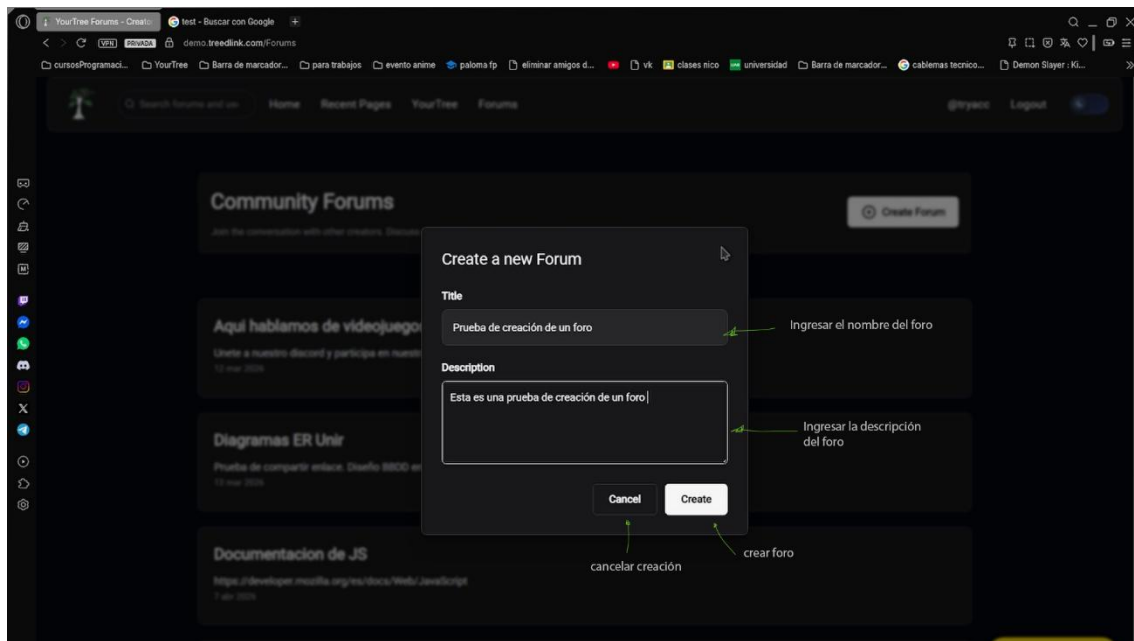


11.2.3.6.1 Creación de un foro

Para facilitar la generación de nuevos espacios de debate, se ha implementado una ventana modal que se superpone a la vista principal. Este componente permite al usuario definir la temática de su comunidad mediante los siguientes elementos funcionales:

- **Botón "Create Forum":** Ubicado en la parte superior derecha del contenedor principal (Hero section del foro), este botón destaca por su diseño con un icono de suma (+), lo que indica visualmente la capacidad de añadir contenido nuevo.
- **Título (Title):** Campo de entrada de texto destinado a capturar el nombre identificativo del foro.
- **Descripción (Description):** Área de texto donde el autor puede detallar el propósito del hilo, proporcionando contexto adicional a los futuros participantes.
- **Botón de cancelar (Cancel):** Permite al usuario abortar el proceso de creación y cerrar la ventana emergente sin realizar cambios en el sistema.
- **Botón de crear (Create):** Acción principal que valida la información introducida y ejecuta la petición al servidor para persistir el nuevo foro en la base de datos.

Ilustración 56 manual de usuario (Creación de un foro)



y descripción.

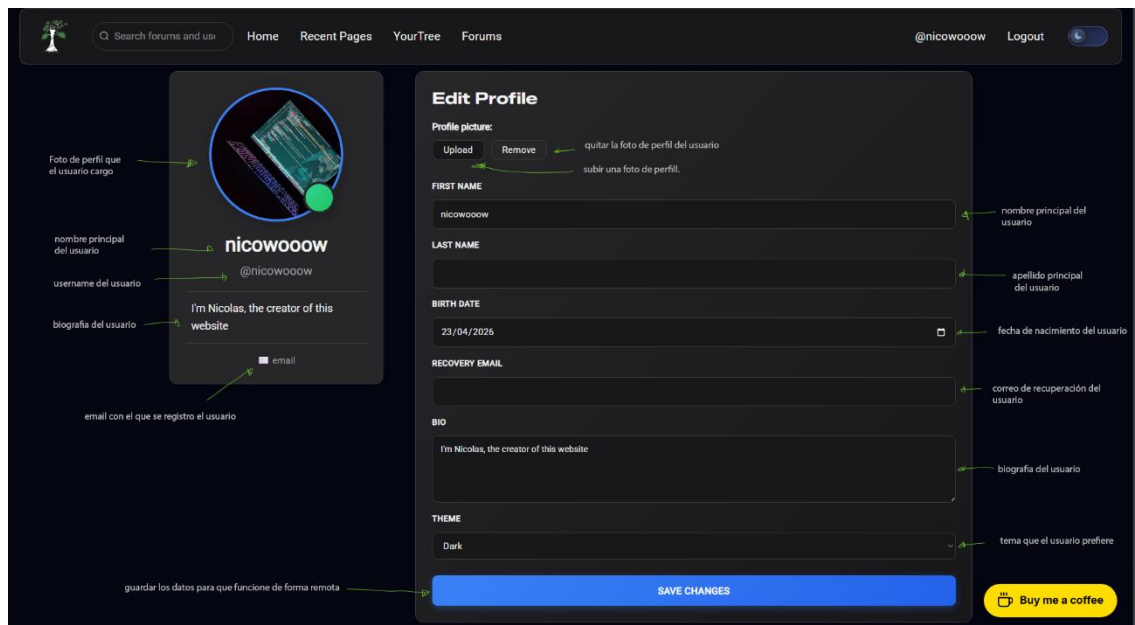
11.2.3.7 @username o perfil

Este módulo permite la gestión de los datos personales y de seguridad del usuario, ofreciendo un control total sobre su identidad dentro de la plataforma. La interfaz se divide visualmente en un panel de previsualización lateral y un formulario central de edición:

- **Gestión de Identidad Visual:**
 - **Foto de perfil:** El usuario puede cargar una nueva imagen (**Upload**) o eliminar la actual (**Remove**). El panel izquierdo muestra en tiempo real cómo luce la foto seleccionada con un indicador de estado activo.
- **Campos de Información Personal:**
 - **First Name / Last Name:** Espacios destinados al nombre y apellidos legales del usuario.
 - **Username / Bio:** Campos vinculados directamente a la identidad pública. La biografía permite una descripción personalizada que se refleja instantáneamente en la tarjeta de presentación de la izquierda.
 - **Birth Date:** Selector de fecha para el registro de la edad del usuario.

- **Seguridad y Preferencias:**
 - **Recovery Email:** Campo para establecer un correo alternativo de recuperación de cuenta.
 - **Theme Selector:** Permite al usuario alternar entre diferentes esquemas visuales (ej. Dark/Light mode) para personalizar su experiencia de uso.
- **Persistencia de Datos:**
 - **Botón "Save Changes":** Este control es el encargado de enviar el objeto con los nuevos datos al Backend para su actualización remota en la base de datos

Ilustración 57 manual de usuario (Profile o @username)



11.2.3.8 Log out

Función de cierre de sesión que garantiza la privacidad del usuario. Al activarse, se destruye el token de autenticación en el cliente y se redirige la navegación a la página de inicio, impidiendo que terceras personas modifiquen el perfil desde el mismo dispositivo.

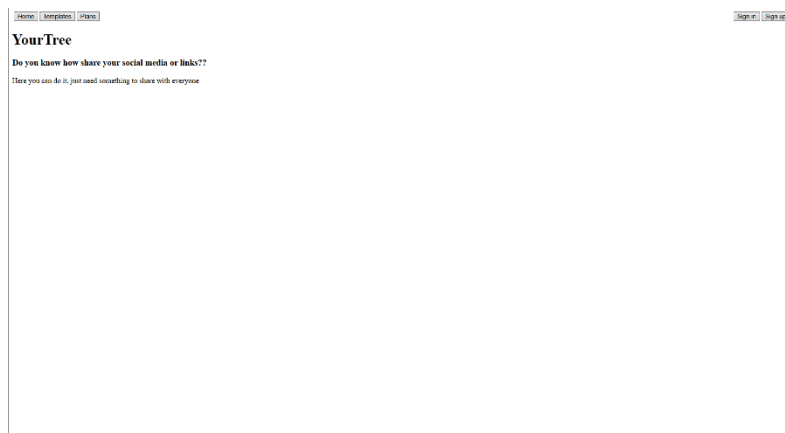
11.3 HISTORIAL DE VERSIONES

El proyecto se realizó tanto con los lenguajes vanilla (html, css y js) y en la versión del proyecto 1.0.0 se empezó a usar react

11.3.1 Versión 0.1.0

Al recién iniciarse el proyecto se comenzó con un HTML básico. En el cual aun no se tenía el maquetado realizado

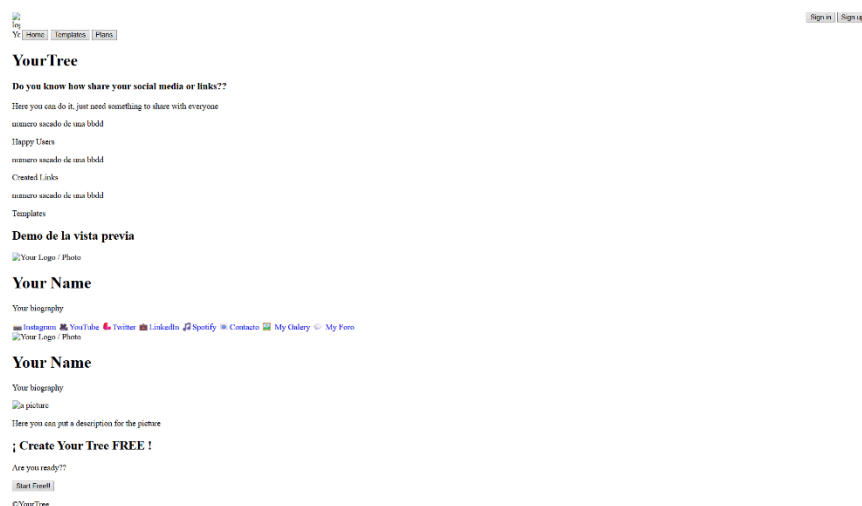
Ilustración 58 inicio del proyecto



11.3.2 Versión 0.1.1

En la segunda versión se comenzó a realizar la estructura de cómo se vería el Home.

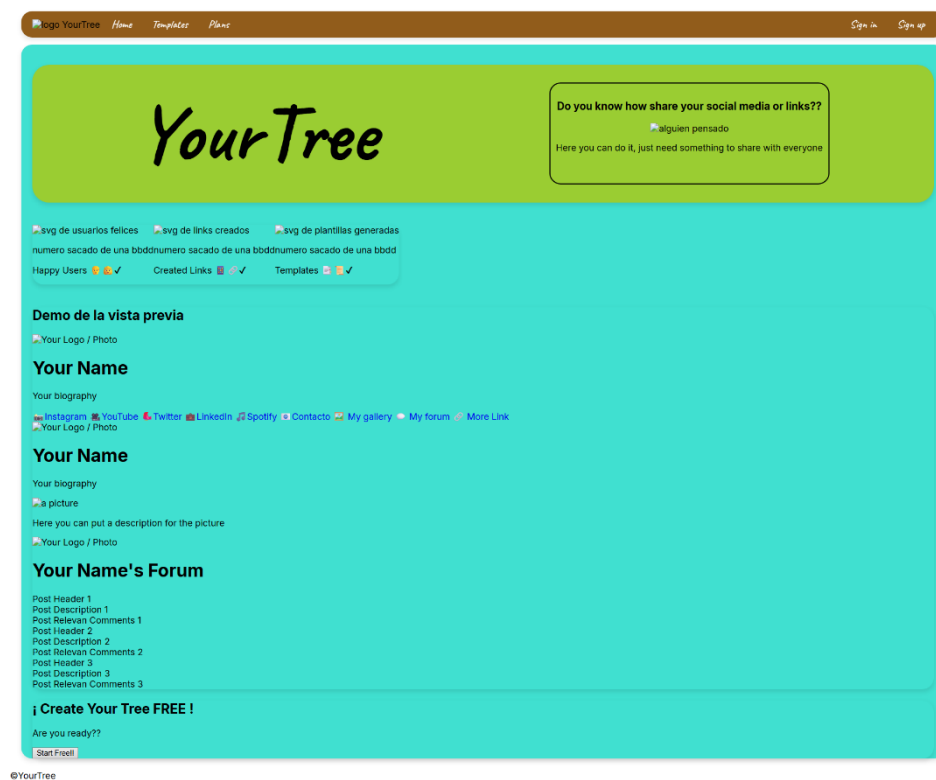
Ilustración 59 estructura básica del home



11.3.3 Versión 0.1.2

La tercera versión ya se tenía el maquetado de cómo se realizaría, aunque en ese momento se pusieron más cosas de las que estaban definidas.

Ilustración 60 estilos básicos implementados



11.3.4 Versión 0.1.3

En la cuarta versión se escogió una paleta de colores más acorde al título “YourTree” y se puso un pequeño loader.

Ilustración 61 loader de inicio

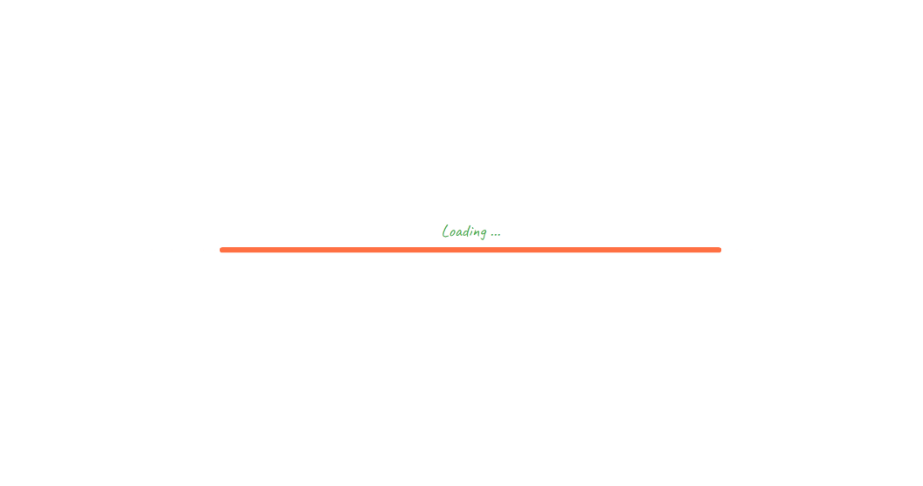
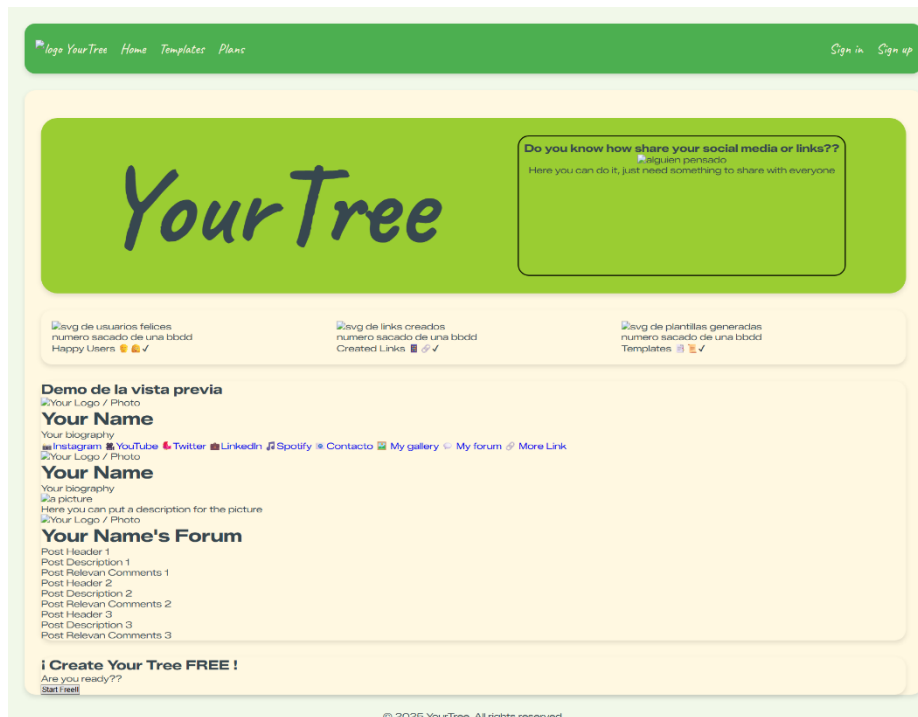


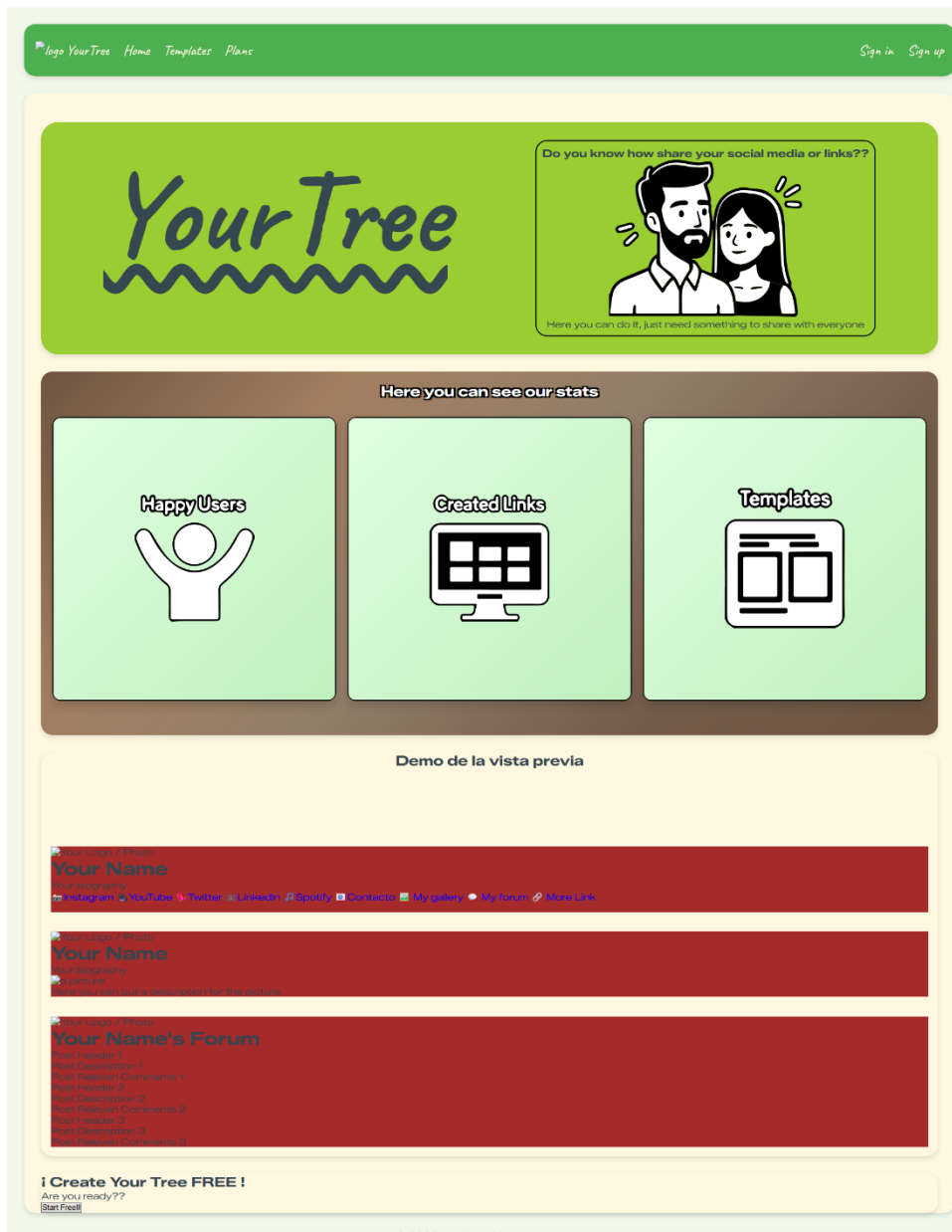
Ilustración 62 vista del home con estilos mas acorde



11.3.5 Versión 0.1.4

En la quinta versión se implementaron las imágenes de referencia que se crearon mediante Illustrator. En la parte de las estadísticas se implementaron una rotación en 3D.

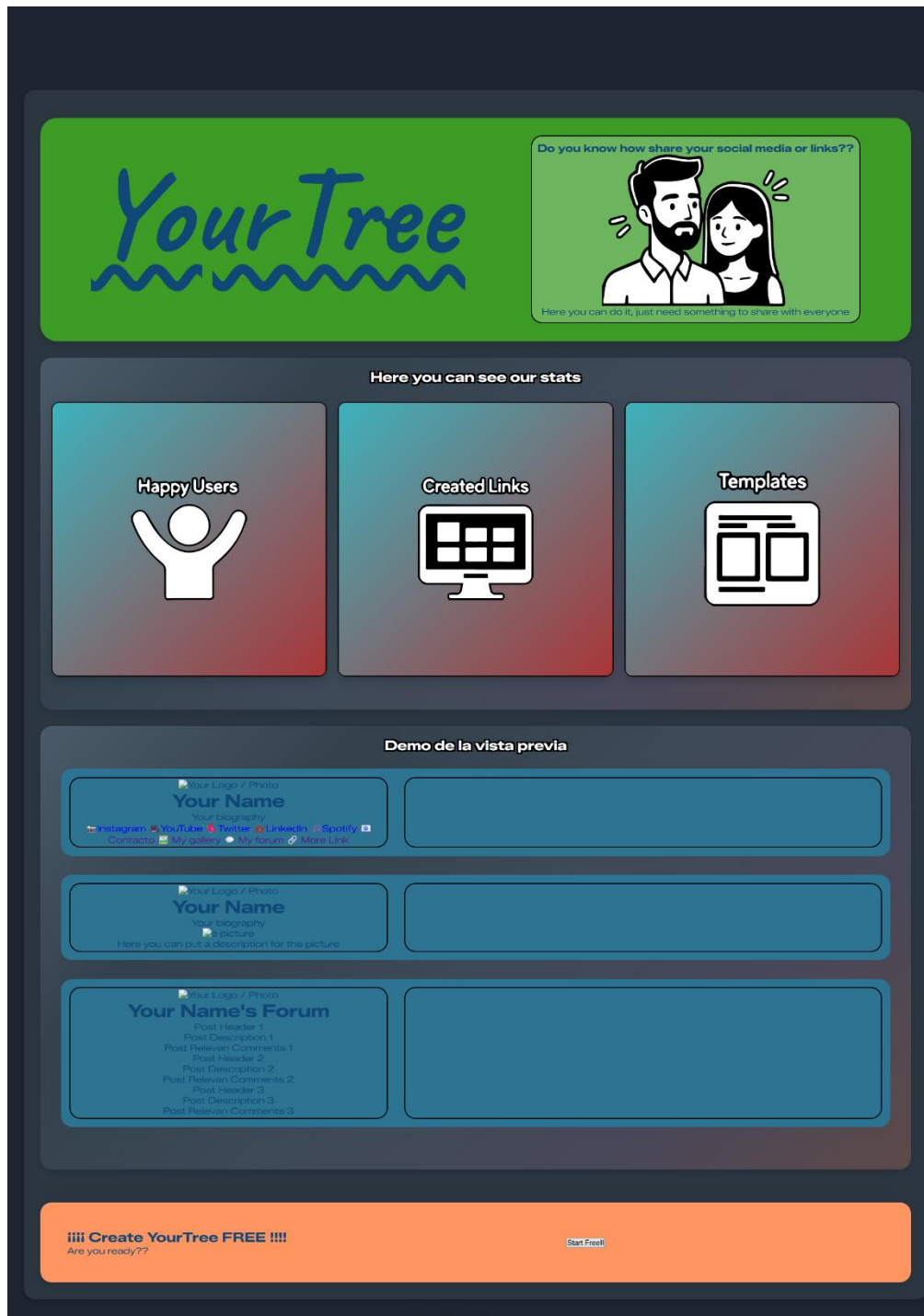
Ilustración 63 vista con imágenes del home



11.3.6 Versión 0.2.0

En la sexta versión se implementó tanto el tema claro y oscuro, mediante el atributo theme incluido en la etiqueta html.

Ilustración 64 tema oscuro del home



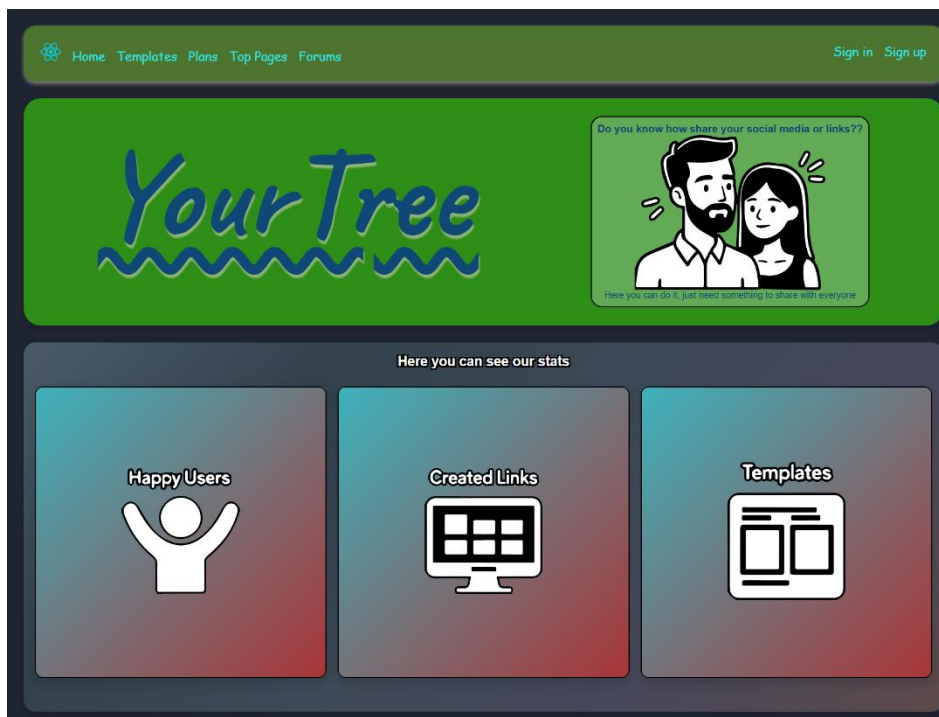
11.3.7 Versión 1.0.0

Desde este punto se empezó a implementar react con una estructura ya definida.

Ilustración 65 estructura del frontend



Ilustración 66 home realizado con React



11.3.8 Versión 1.0.1

Desde este punto se empezó a crear el backend mediante express, su construcción fue gradual y no tuvo cambios drásticos a mencionar.

YourTree – Proyecto FP

Nicolas Pezo Ccama

Ilustración 67 creacion del backend con express



11.3.9 Versión 1.3.0

Desde este punto se hicieron distintas modificaciones a la base de datos, en la cual se almacenarían los datos. Anteriormente se utilizaba el almacenamiento en memoria.

Ilustración 68 primera versión de la base de datos

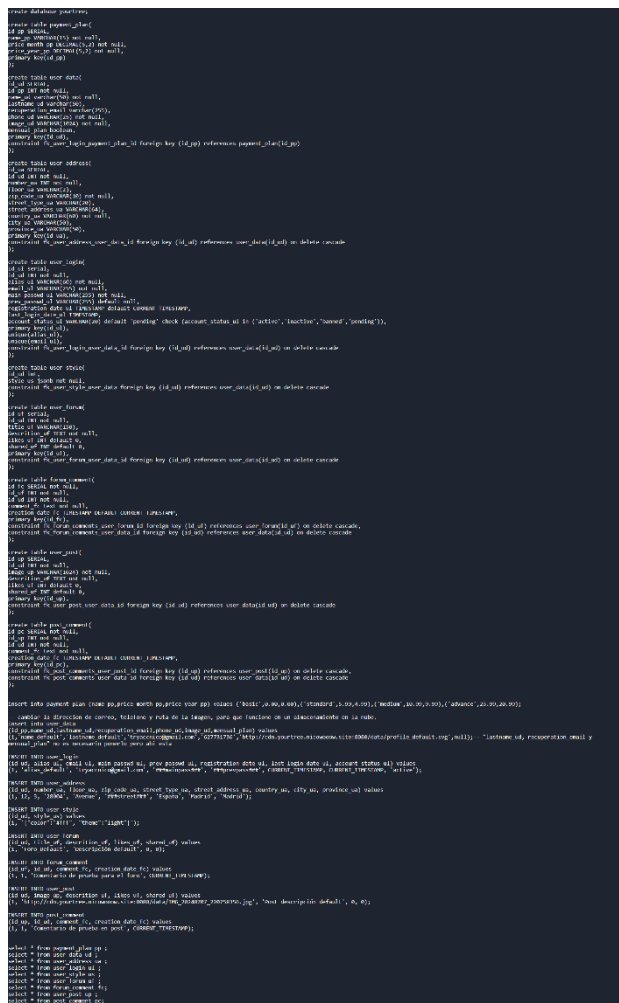


Ilustración 69 segunda versión de la base de datos

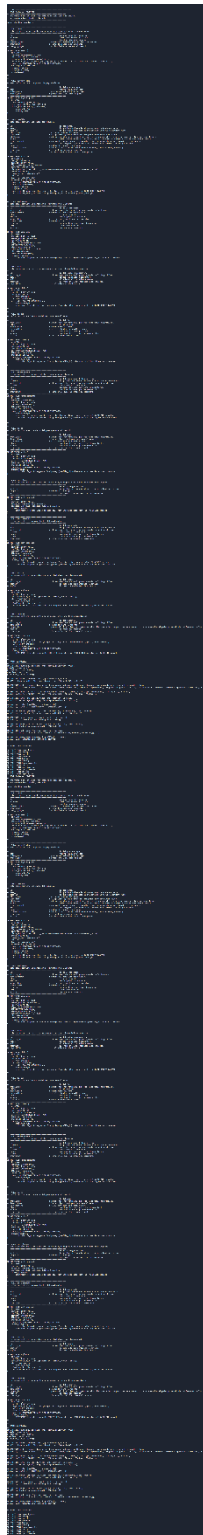


Ilustración 70 tercera versión de la base de datos

```

-- WORK IN DAYS: your tree
-- create database yourtree;
-- drop database yourtree;

-- TABLES roles
-- =====
create table
roles (
  id SERIAL,
  name VARCHAR(50) not null unique,
  description TEXT,
  primary key (id)
);

-- TABLE users
-- =====
create table
users (
  id SERIAL,
  username VARCHAR(50) not null,
  email VARCHAR(100) not null,
  password_hash VARCHAR(255) not null,
  status VARCHAR(20) default 'pending' check (
    status in ('active', 'inactive', 'banned', 'pending')
  ),
  created_at TIMESTAMPTZ default CURRENT_TIMESTAMP,
  last_login_at TIMESTAMPTZ,
  role_id int default 1 not null,
  token_version int default 0 not null,
  verify_code VARCHAR(7) default null,
  primary key (id),
  unique (username),
  unique (email),
  constraint fk_users_role_id foreign key (role_id) references roles (id)
);

-- =====
-- TABLE profiles
-- =====
create table
profiles (
  id SERIAL,
  user_id int not null,
  first_name VARCHAR(50) not null,
  last_name VARCHAR(50),
  birth_date date check (
    birth_date <= CURRENT_DATE
  ),
  recovery_email VARCHAR(255),
  bio TEXT,
  image_url VARCHAR(500),
  theme VARCHAR(50) default 'light' check (theme in ('dark', 'light')),
  created_at TIMESTAMPTZ default CURRENT_TIMESTAMP,
  primary key (id),
  unique (user_id),
  constraint fk_profiles_user_id foreign key (user_id) references users (id) on delete cascade
);

CREATE INDEX idx_profiles_user_id on profiles (user_id);

-- TABLE categories
-- =====
create table
categories (
  id SERIAL,
  name VARCHAR(50) not null,
  theme VARCHAR(50) not null,
  primary key (id),
  unique (name)
);

-- TABLE links
-- =====
create table
links (
  id SERIAL,
  profile_id int not null,
  title VARCHAR(100),
  url TEXT not null,
  url_image TEXT default '' not null,
  position int not null,
  is_active BOOLEAN default true,
  primary key (id),
  constraint fk_links_profile_id foreign key (profile_id) references profiles (id) on delete cascade
);

CREATE INDEX idx_links_profile_id on links (profile_id);

-- TABLE forum
-- =====
create table
forum (
  id SERIAL,
  profile_id int not null,
  title VARCHAR(100),
  description TEXT not null,
  is_pinned BOOLEAN default false,
  status VARCHAR(20) default 'active' check (status in ('active', 'hidden', 'deleted')),
  likes int default 0,
  likes int default 0,
  created_at TIMESTAMPTZ default CURRENT_TIMESTAMP,
  primary key (id),
  constraint fk_forum_profile_id foreign key (profile_id) references profiles (id) on delete cascade
);

CREATE INDEX idx_forum_profile_id on forum (profile_id);

-- TABLE forum comments
-- =====
create table
forum_comments (
  id SERIAL,
  forum_id int not null,
  profile_id int not null,
  content TEXT not null,
  status VARCHAR(20) default 'active' check (status in ('active', 'hidden', 'deleted')),
  likes int default 0,
  likes int default 0,
  created_at TIMESTAMPTZ default CURRENT_TIMESTAMP,
  primary key (id),
  constraint fk_forum_comments_forum_id foreign key (forum_id) references forum (id) on delete cascade,
  constraint fk_forum_comments_profile_id foreign key (profile_id) references profiles (id) on delete cascade
);

CREATE INDEX idx_forum_comments_forum_id on forum_comments (forum_id);
CREATE INDEX idx_forum_comments_profile_id on forum_comments (profile_id);

```

11.3.10 Versión 1.5.0

A partir de esta versión ya se tuvo el conocimiento de como poder hacer la parte de rutas dinámicas en React, así pudiendo tener las siguientes páginas disponibles y por otro lado se quiso intentar un estilo distinto para el frontend (gradiente y efecto glass).

Ilustración 71 ruta /

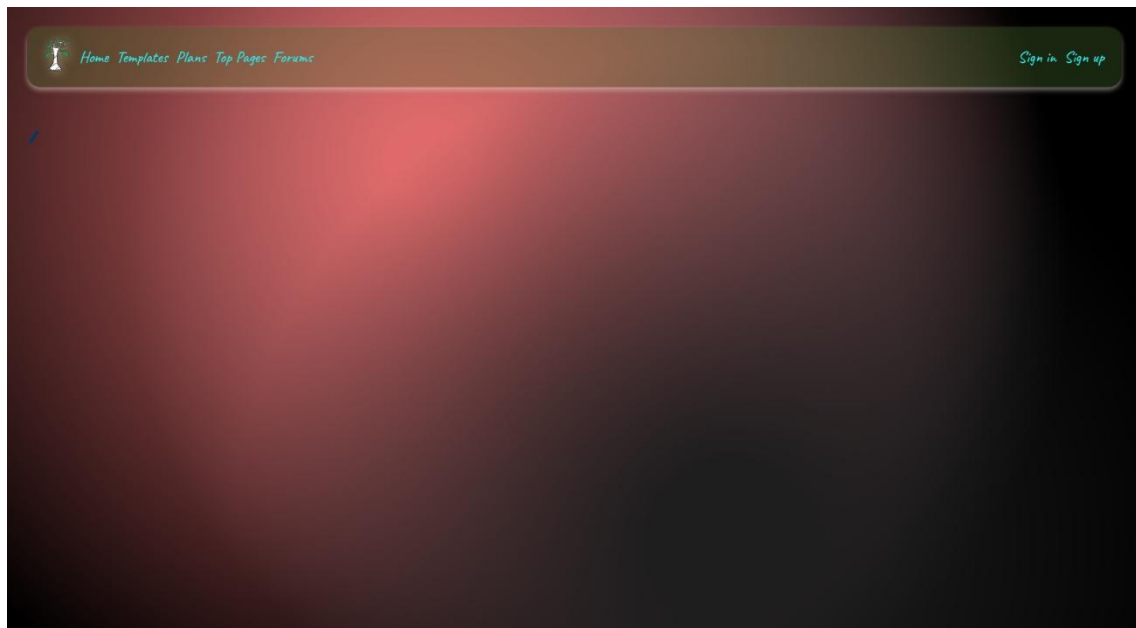


Ilustración 72 ruta templates

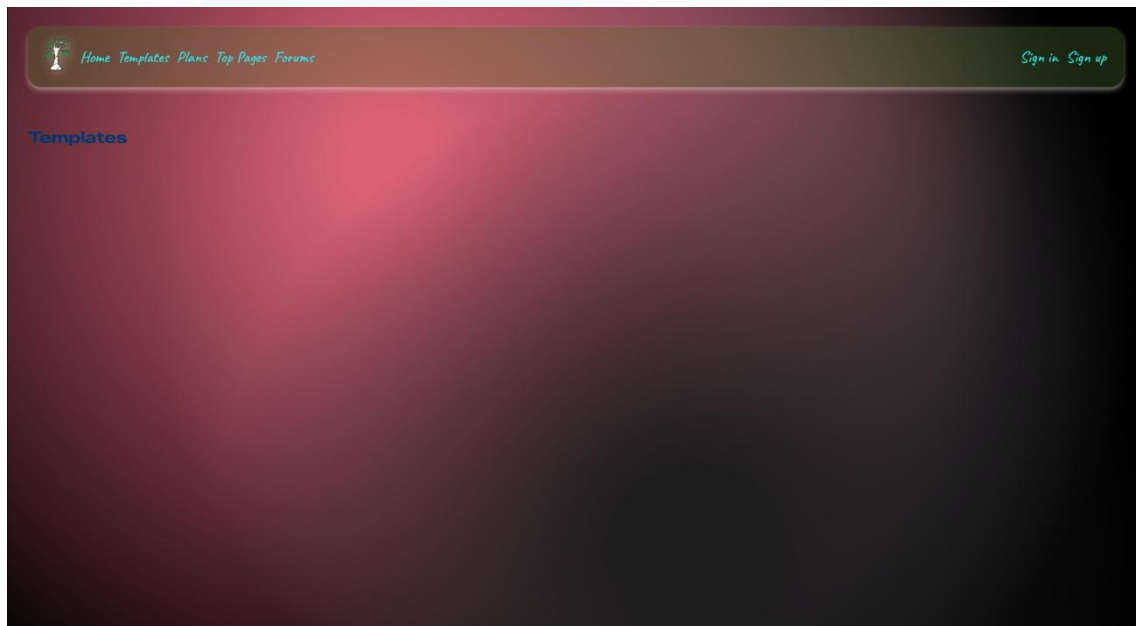


Ilustración 73 ruta plans

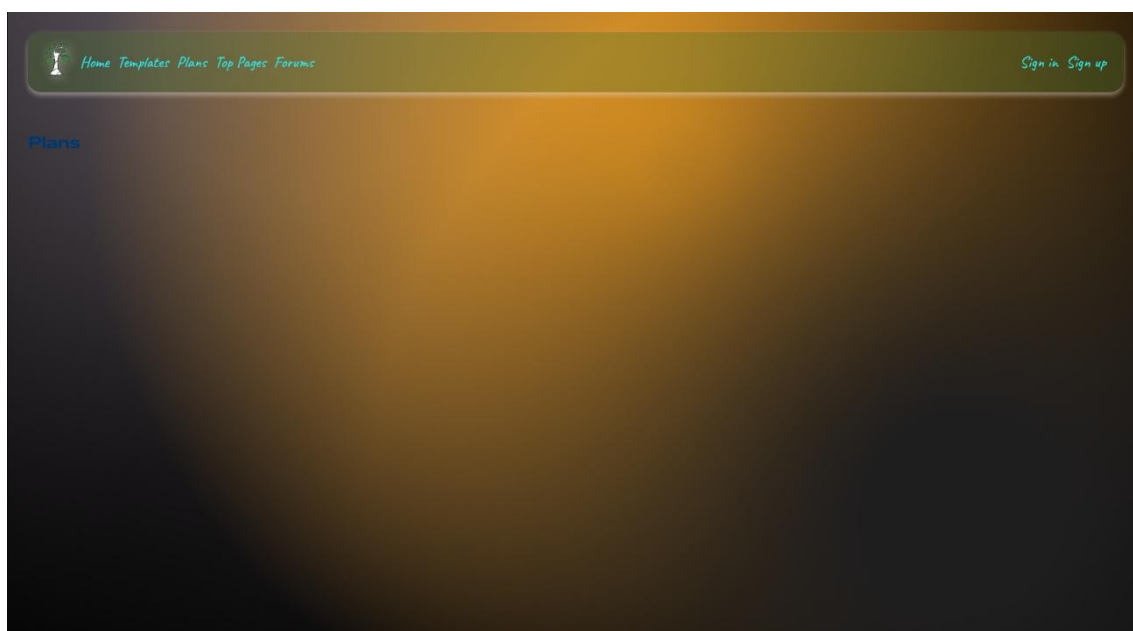


Ilustración 74 ruta top pages

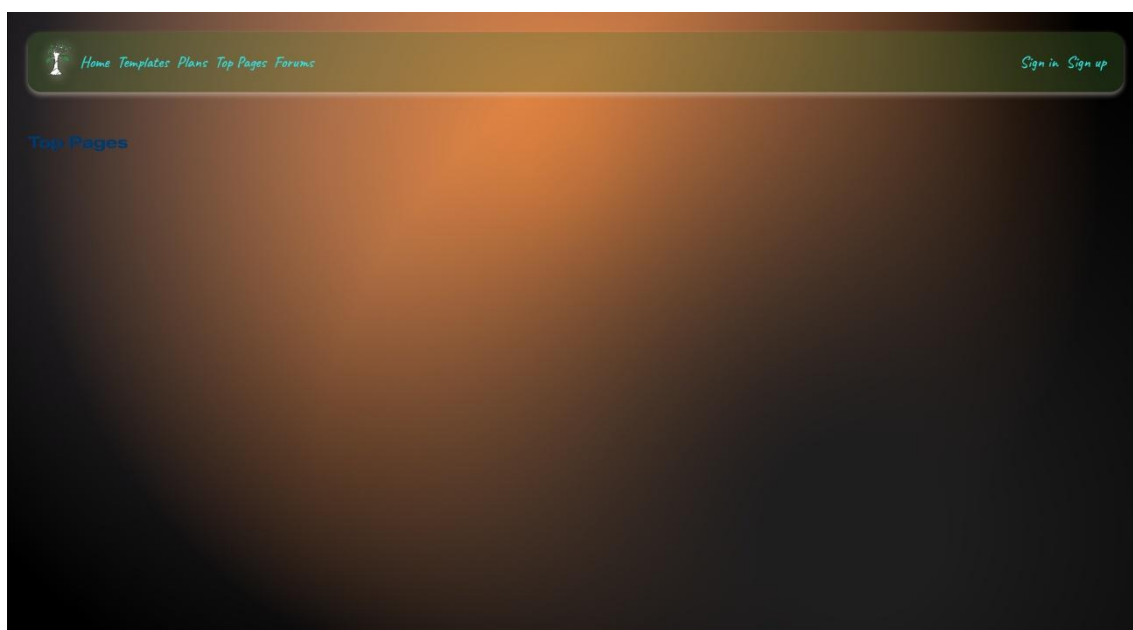


Ilustración 75 ruta fóruns

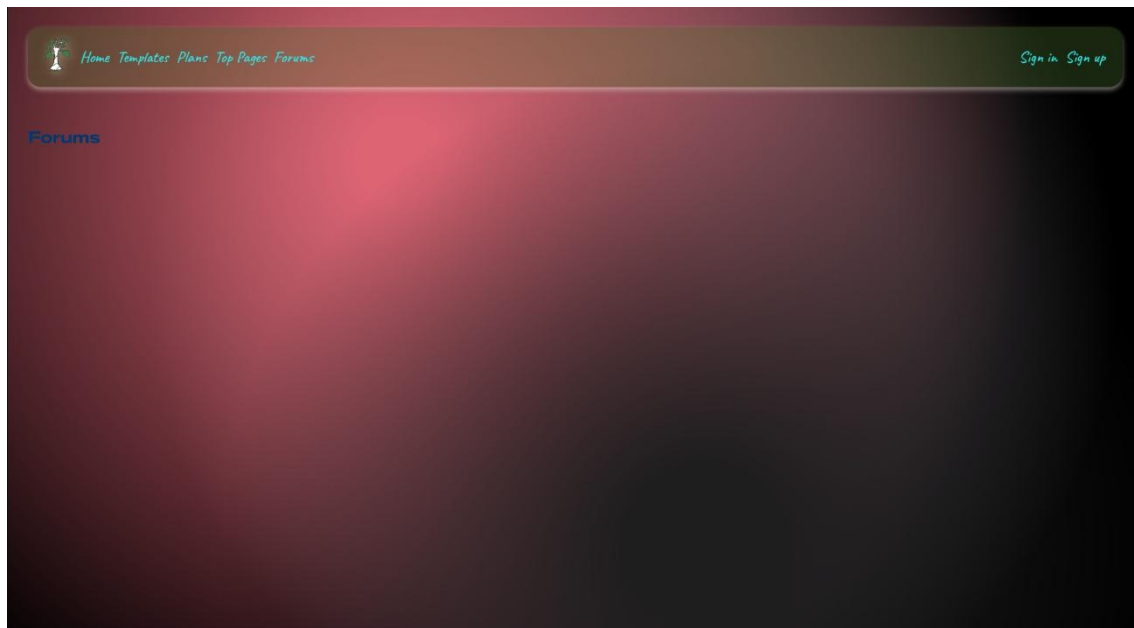


Ilustración 76 ruta sign up

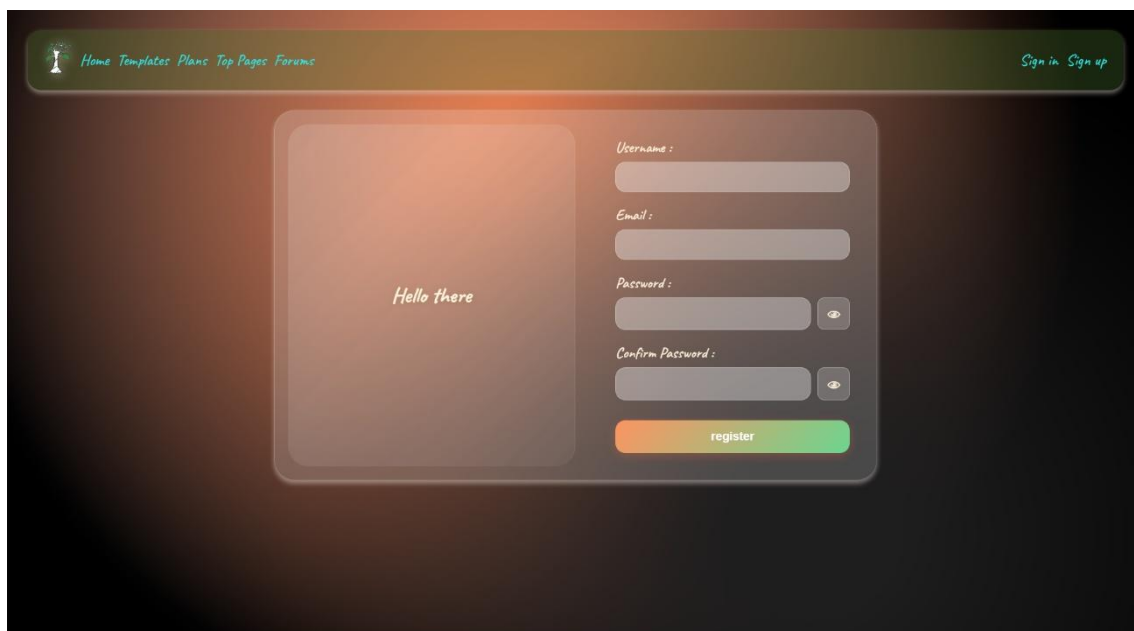


Ilustración 77 ruta sign in

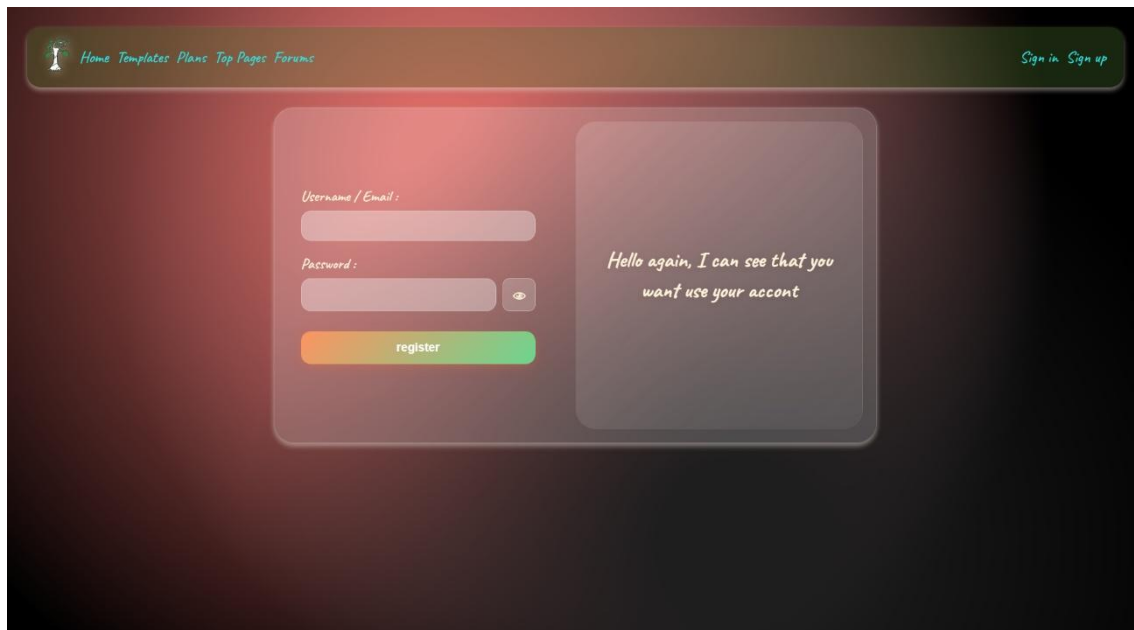
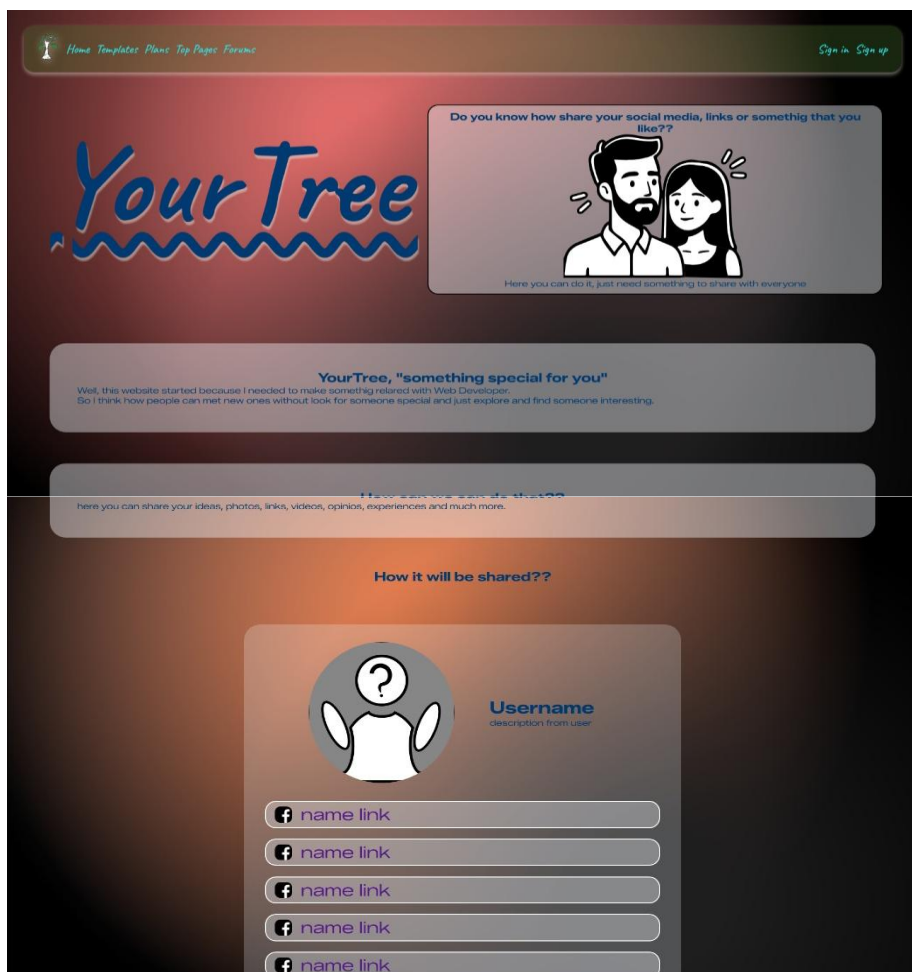


Ilustración 78 ruta home



11.4 INFORME DE AUDITORÍA DE SEGURIDAD (OWASP ZAP)

YourTree – Proyecto FP

Nicolas Pezo Ccama

Para garantizar la integridad y la protección de los datos de los usuarios en YourTree, se ha realizado una auditoría de seguridad automatizada utilizando OWASP ZAP. Este análisis permite identificar vulnerabilidades críticas como inyección SQL, Cross-Site Scripting (XSS) y configuraciones de seguridad de red débiles.

11.4.1.1 Acceso al reporte detallado

El informe completo con los resultados detallados de la auditoría se encuentra disponible en formato HTML a través del siguiente enlace al repositorio oficial del proyecto:

Enlace directo: [Reporte ZAP - YourTree](#)

11.4.1.2 Descarga y ejecución local

Para una revisión técnica profunda o ejecución en entornos locales, se pueden descargar exclusivamente los archivos de auditoría del repositorio de GitHub mediante la técnica de sparse-checkout, siguiendo estos comandos en la terminal:

1. Clonar el repositorio sin descargar los archivos aún

```
git clone --filter=blob:none --no-checkout  
https://github.com/nicowooow/Proyecto-FP.git  
cd Proyecto-FP
```

2. Configurar el modo sparse-checkout

```
git sparse-checkout init --cone
```

3. Definir la carpeta que quieres

```
git sparse-checkout set "auditoria de prueba"
```

4. Descargar los archivos

```
git checkout
```

11.5 FRAGMENTOS DE CÓDIGO IMPORTANTES

11.5.1 PutLinks

Componente que se utiliza para poner los enlaces creados mediante React.

```
export const PutLinks = React.memo(({ username, children, ...rest }) => {
  let [links, setLinks] = React.useState([]);
  let [profileId, setProfileId] = React.useState(null);

  useEffect(() => {
    async function fetchLinks() {
      try {
        let profileRes = await fetch(`/youtree/api/profile/${username}`);
        if (!profileRes.ok) return;
        let profile = await profileRes.json();
        if (profile.id) {
          setProfileId(profile.id);
          let linksRes = await fetch(`/youtree/api/links/${profile.id}`);
          if (linksRes.ok) {
            let data = await linksRes.json();
            setLinks(data);
          }
        }
      } catch (e) {
        console.error("Error fetching links:", e);
      }
    }
    if (username) fetchLinks();
  }, [username]);

  const list_links = useMemo(
    () =>
      (links || []).map((link, i) => {
        const actions = React.Children.map(children, (child) =>
          React.isValidElement(child)
            ? React.cloneElement(child, { linkId: link.id, profileId })
            : child,
        );
        const isImage =
          typeof link.url === "string" &&
          link.urlImage.startsWith("https://");

        let iconUrl = "";
        // console.log(link);

        if (isImage) {
          const url = link.urlImage.split('https://')[1].split('/');
          // console.log(url);

          // con esto hacemos que la imagen de url , si no tiene un .(imagen) en el primer corte de
          // la url se descarte
          const isImageFile = /\.svg|png|jpe?g|webp|gif|ico$/i.test(url[link.length - 1]);

          if (isImageFile) {
            iconUrl = link.urlImage;
          } else {
            const hostname = new URL(link.urlImage).hostname;
            try {
              iconUrl = `https://www.google.com/s2/favicons?domain=https://${hostname}&sz=64`;
            } catch (_) {}
          }
        }

        const liStyle = isImage
          ? { "--link-icon": `url(${iconUrl})` }
          : {};

        return (
          <li
            key={link.id ?? i}
            className={isImage ? "link-image" : "link-text"}
            style={liStyle}
          >
            <section className="links">
              <a href={link.url ?? "#"} target="_blank" rel="noopener noreferrer">
                {link.title ?? link.name ?? "name link"}
              </a>
            </section>
          </li>
        );
      })
  );
});
```

```

        <section className="links_actions">{actions}</section>
      </li>
    );
  }},
  [links, children],
);
return <ul>{list_links}</ul>;
});

```

11.5.2 Collapsible

Componente que se utiliza para hacer los menus desplegables.

```

import React, { useState, useEffect, useMemo } from "react";

export const Collapsible = React.memo(({ title, section, children }) => {
  let [open, setOpen] = useState(false);

  const handleToggle = () => {
    setOpen((v) => !v);
  };

  const buttonTitle = useMemo(() => `${title} ${open ? "▲" : "▼"}`, [title, open]);

  return (
    <section className={`collapsible_${section} ${open ? "open" : ""}`>
      <button type="button" onClick={handleToggle}>
        {buttonTitle}
      </button>
      <div className={`collapsible_body_${section}`}>{children}</div>
    </section>
  );
});

```

11.5.3 verifyToken

Función de autenticación, que se encarga de verificar el token almacenado en las cookies.

```

export const verifyToken = async () => {
  try {
    //si tenemos un token activo lo buscamos
    const token = getToken();

    //si no lo tenemos el valor es false ara el header y ahí queda
    if (!token) return false;
    // enviamos al back el token con la clave de authorization
    // por el header de la web enviamos el token
    let res = await fetch("/yourtree/api/authenticate", {
      headers: {
        Authorization: `Bearer ${token}`,
      },
    });
    // console.log(res);
    // console.log(res.body);

    // si no lo encuentra pasamos al generar el token de nuevo
    if (res.status === 401) {
      // Llamamos al metodo que tenemos arriba el cual se encarga del proceso de enviar datos
      const newToken = await refreshToken();
      // si lo nos regresa nada entonces no se puede regenerar el token y por ende no está logueado
      // osea que el header es falso y nos muestra dichas opciones
      if (!newToken) return false;
      //si no es un valor vacío entonces lo enviamos al back como un nuevo token
      res = await fetch("/yourtree/api/authenticate", {
        headers: { Authorization: `Bearer ${newToken}` },
      });
      // console.log("se refresco el token");
    }
    // si nos dice que la respuesta HTTP está entre los 400 a 500 retornamos falso
    if (!res.ok) {
      // logout();
    }
  }
};

```



```

        return false;
    }
    // sino esperamos la respuesta del backend
    let data = await res.json();
    // console.log(data);
    // retornamos los datos del back y en especifico si esta logueado o no
    // ponemos el !! para asegurarnos que el valor sea un booleano por la doble negacion
    return !!data.isLogged;
} catch (error) {
    console.error(error);
    return false;
}
};

```

11.5.4 refreshAccessToken

Función del backend que se encarga de refrescar los tokens que llegan desde el cliente.

```

import jwt from "jsonwebtoken";
import { JWT_REFRESH_SECRET } from "../config/config.js";
import { createAccessToken } from "../utils/jwt.utils.js";

// Validar refresh token y generar nuevo access token
export const refreshAccessToken = async (refreshToken) => {
    if (!refreshToken) {
        throw new Error("REFRESH_REQUIRED");
    }

    let payload;

    try {
        payload = jwt.verify(refreshToken, JWT_REFRESH_SECRET);
    } catch (error) {
        throw new Error("INVALID_REFRESH");
    }

    const newAccessToken = createAccessToken(
        payload.id,
        payload.username,
        payload.role
    );

    return newAccessToken;
};

```